Part IV

Nonparametric Models

16 Exemplar-based Methods

So far in this book, we have mostly focused on **parametric models**, either unconditional $p(\boldsymbol{y}|\boldsymbol{\theta})$ or conditional $p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is a fixed-dimensional vector of parameters. The parameters are estimated from a variable-sized dataset, $\mathcal{D} = \{(\boldsymbol{x}_n, \boldsymbol{y}_n) : n = 1 : N\}$, but after model fitting, the data is thrown away.

In this section we consider various kinds of **nonparametric models**, that keep the training data around. Thus the effective number of parameters of the model can grow with $|\mathcal{D}|$. We focus on models that can be defined in terms of the **similarity** between a test input, $\boldsymbol{x}$, and each of the training inputs, $\boldsymbol{x}_n$. Alternatively, we can define the models in terms of a dissimilarity or distance function $d(\boldsymbol{x}, \boldsymbol{x}_n)$. Since the models keep the training examples around at test time, we call them **exemplar-based models**. (This approach is also called **instance-based learning** [AKA91], or **memory-based learning**.)

16.1 K nearest neighbor (KNN) classification

In this section, we discuss one of the simplest kind of classifier, known as the **K nearest neighbor** (**KNN**) classifier. The idea is as follows: to classify a new input $\boldsymbol{x}$, we find the K closest examples to $\boldsymbol{x}$ in the training set, denoted $N_K(\boldsymbol{x}, \mathcal{D})$, and then look at their labels, to derive a distribution over the outputs for the local region around $\boldsymbol{x}$. More precisely, we compute

$$p(y = c|\boldsymbol{x}, \mathcal{D}) = \frac{1}{K} \sum_{n \in N_K(\boldsymbol{x}, \mathcal{D})} \mathbb{I}(y_n = c) \tag{16.1}$$

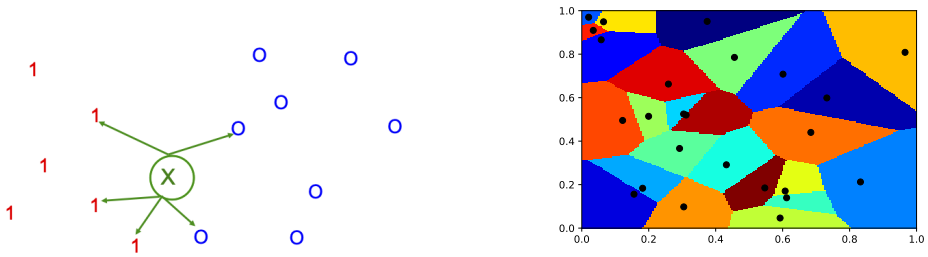
We can then return this distribution, or the majority label.

The two main parameters in the model are the size of the neighborhood, K, and the distance metric $d(\boldsymbol{x}, \boldsymbol{x}')$. For the latter, it is common to use the **Mahalanobis distance**

$$d_{\mathbf{M}}(\boldsymbol{x}, \boldsymbol{\mu}) = \sqrt{(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}} \mathbf{M} (\boldsymbol{x} - \boldsymbol{\mu})} \tag{16.2}$$

where $\mathbf{M}$ is a positive definite matrix. If $\mathbf{M} = \mathbf{I}$, this reduces to Euclidean distance. We discuss how to learn the distance metric in Section 16.2.

Despite the simplicity of KNN classifiers, it can be shown that this approach gets within a factor of 2 of the Bayes error (which measures the performance of the best possible classifier) as $N \to \infty$ [CH67; CD14]. (Of course the convergence rate to this optimal performance may be poor in practice, for reasons we discuss in Section 16.1.2.)



(a) *(b)*

Figure 16.1: (a) Illustration of a K-nearest neighbors classifier in 2d for $K = 5$. The nearest neighbors of test point $\boldsymbol{x}$ have labels $\{1, 1, 1, 0, 0\}$, so we predict $p(y = 1|\boldsymbol{x}, \mathcal{D}) = 3/5$. (b) Illustration of the Voronoi tessellation induced by 1-NN. Adapted from Figure 4.13 of [DHS01]. Generated by knn_voronoi_plot.ipynb.

16.1.1 Example

We illustrate the KNN classifier in 2d in Figure 16.1(a) for $K = 5$. The test point is marked as an "x". 3 of the 5 nearest neighbors have label 1, and 2 of the 5 have label 0. Hence we predict $p(y = 1|\boldsymbol{x}, \mathcal{D}) = 3/5 = 0.6$.

If we use $K = 1$, we just return the label of the nearest neighbor, so the predictive distribution becomes a delta function. A KNN classifier with $K = 1$ induces a **Voronoi tessellation** of the points (see Figure 16.1(b)). This is a partition of space which associates a region $V(\boldsymbol{x}_n)$ with each point $\boldsymbol{x}_n$ in such a way that all points in $V(\boldsymbol{x}_n)$ are closer to $\boldsymbol{x}_n$ than to any other point. Within each cell, the predicted label is the label of the corresponding training point. Thus the training error will be 0 when $K = 1$. However, such a model is usually overfitting the training set, as we show below.

Figure 16.2 gives an example of KNN applied to a 2d dataset, in which we have three classes. We see how, with $K = 1$, the method makes zero errors on the training set. As K increases, the decision boundaries become smoother (since we are averaging over larger neighborhoods), so the training error increases, as we start to underfit. This is shown in Figure 16.2(d). The test error shows the usual U-shaped curve.

16.1.2 The curse of dimensionality

The main statistical problem with KNN classifiers is that they do not work well with high dimensional inputs, due to the **curse of dimensionality**.

The basic problem is that the volume of space grows exponentially fast with dimension, so you might have to look quite far away in space to find your nearest neighbor. To make this more precise, consider this example from [HTF09, p22]. Suppose we apply a KNN classifier to data where the inputs are uniformly distributed in the D-dimensional unit cube. Suppose we estimate the density of class labels around a test point $\boldsymbol{x}$ by "growing" a hyper-cube around $\boldsymbol{x}$ until it contains a desired fraction p of the data points. The expected edge length of this cube will be $e_D(s) \triangleq p^{1/D}$; this function is plotted in Figure 16.3(b). If $D = 10$, and we want to base our estimate on 10% of the

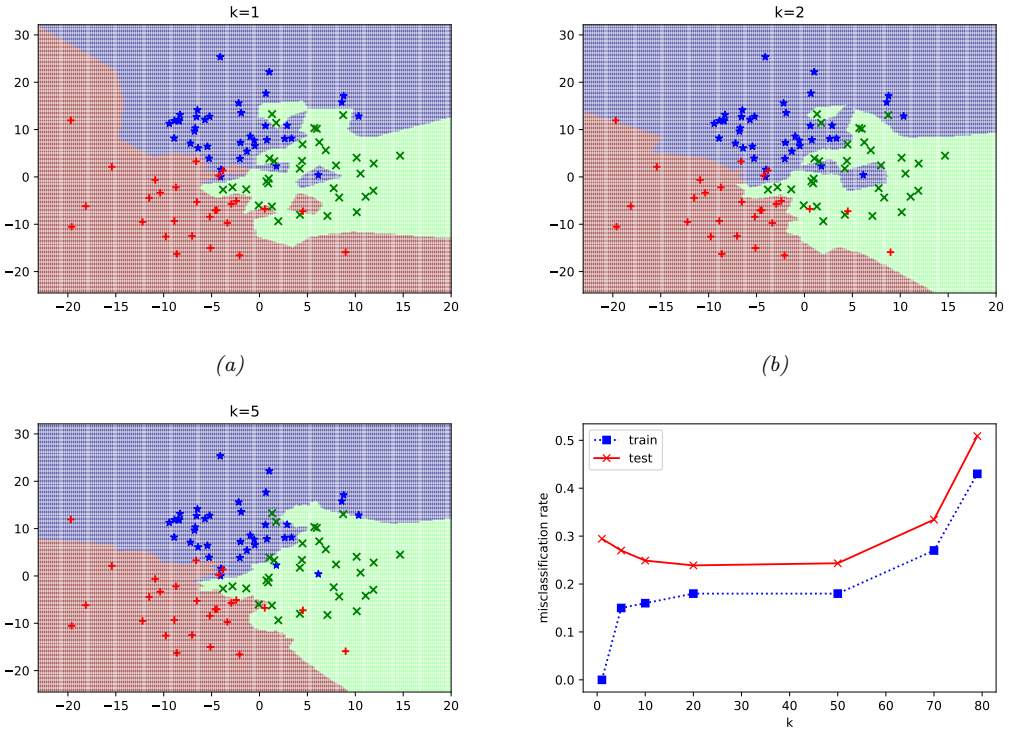


Figure 16.2: Decision boundaries induced by a KNN classifier. (a) $K = 1$. (b) $K = 2$. (c) $K = 5$. (d) Train and test error vs K. Generated by knn_classify_demo.ipynb.

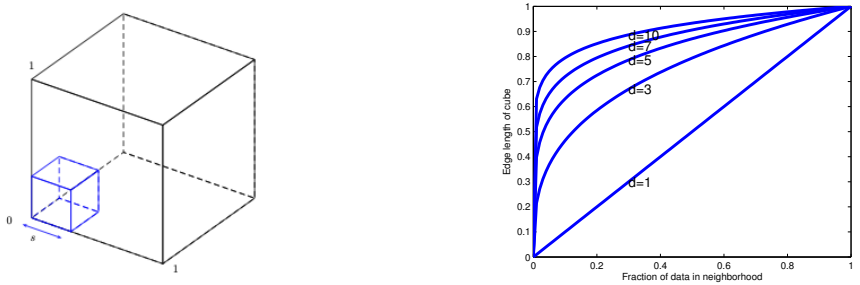


Figure 16.3: Illustration of the curse of dimensionality. (a) We embed a small cube of side s inside a larger unit cube. (b) We plot the edge length of a cube needed to cover a given volume of the unit cube as a function of the number of dimensions. Adapted from Figure 2.6 from [HTF09]. Generated by curse_dimensionality_plot.ipynb.

data, we have $e_{10}(0.1) = 0.8$, so we need to extend the cube 80% along each dimension around $\boldsymbol{x}$. Even if we only use 1% of the data, we find $e_{10}(0.01) = 0.63$. Since the range of the data is only 0 to 1 along each dimension, we see that the method is no longer very local, despite the name "nearest neighbor". The trouble with looking at neighbors that are so far away is that they may not be good predictors about the behavior of the function at a given point.

There are two main solutions to the curse: make some assumptions about the form of the function (i.e., use a parametric model), and/or use a metric that only cares about a subset of the dimensions (see Section 16.2).

16.1.3 Reducing the speed and memory requirements

KNN classifiers store all the training data. This is obviously very wasteful of space. Various heuristic pruning techniques have been proposed to remove points that do not affect the decision boundaries, see e.g., [WM00]. In Section 17.4, we discuss a more principled approach based on a sparsity promoting prior; the resulting method is called a sparse kernel machine, and only keeps a subset of the most useful exemplars.

In terms of running time, the challenge is to find the K nearest neighbors in less than $O(N)$ time, where N is the size of the training set. Finding exact nearest neighbors is computationally intractable when the dimensionality of the space goes above about 10 dimensions, so most methods focus on finding the approximate nearest neighbors. There are two main classes of techniques, based on partitioning space into regions, or using hashing.

For partitioning methods, one can either use some kind of **k-d tree**, which divides space into axis-parallel regions, or some kind of clustering method, which uses anchor points. For hashing methods, **locality sensitive hashing** (**LSH**) [GIM99] is widely used, although more recent methods learn the hashing function from data (see e.g., [Wan+15]). See [LRU14] for a good introduction to hashing methods.

An open-source library called **FAISS**, for efficient exact and approximate nearest neighbor search (and K-means clustering) of dense vectors, is available at https://github.com/facebookresearch/faiss, and described in [JDJ17].

16.1.4 Open set recognition

> Ask not what this is called, ask what this is like. — Moshe Bar.[Bar09]

In all of the classification problems we have considered so far, we have assumed that the set of classes $\mathcal{C}$ is fixed. (This is an example of the **closed world assumption**, which assumes there is a fixed number of (types of) things.) However, many real world problems involve test samples that come from new categories. This is called **open set recognition**, as we discuss below.

16.1.4.1 Online learning, OOD detection and open set recognition

For example, suppose we train a face recognition system to predict the identity of a person from a fixed set or **gallery** of face images. Let $\mathcal{D}_t = \{(\boldsymbol{x}_n, y_n) : \boldsymbol{x}_n \in \mathcal{X}, y_n \in \mathcal{C}_t, n = 1 : N_t\}$ be the labeled dataset at time t, where $\mathcal{X}$ is the set of (face) images, and $\mathcal{C}_t = \{1, \ldots, C_t\}$ is the set of people known to the system at time t (where $C_t \leq t$). At test time, the system may encounter a new person that it has not seen before. Let $\boldsymbol{x}_{t+1}$ be this new image, and $y_{t+1} = C_{t+1}$ be its new label. The system

needs to recognize that the input is from a new category, and not accidentally classify it with a label from $\mathcal{C}_t$. This is called **novelty detection**. In this case, the input is being generated from the distribution $p(\boldsymbol{x}|y = C_{t+1})$, where $C_{t+1} \notin \mathcal{C}_t$ is the new "class label". Detecting that $\boldsymbol{x}_{t+1}$ is from a novel class may be hard if the appearance of this new image is similar to the appearance of any of the existing images in $\mathcal{D}_t$.

If the system is successful at detecting that $\boldsymbol{x}_{t+1}$ is novel, then it may ask for the id of this new instance, call it C_{t+1}. It can then add the labeled pair $(\boldsymbol{x}_{t+1}, C_{t+1})$ to the dataset to create $\mathcal{D}_{t+1}$, and can grow the set of unique classes by adding C_{t+1} to $\mathcal{C}_t$ (c.f., [JK13]). This is called **incremental learning**, **online learning**, **life-long learning**, or **continual learning**. At future time points, the system may encounter an image sampled from $p(\boldsymbol{x}|y = c)$, where c is an existing class, or where c is a new class, or the image may be sampled from some entirely different kind of distribution $p'(\boldsymbol{x})$ unrelated to faces (e.g., someone uploads a photo of their dog). (Detecting this latter kind of event is called **out-of-distribution** or **OOD** detection.)

In this online setting, we often only get a few (sometimes just one) example of each class. Prediction in this setting is known as **few-shot classification**, and is discussed in more detail in Section 19.6. KNN classifiers are well-suited to this task. For example, we can just store all the instances of each class in a gallery of examples, as we explained above. At time $t + 1$, when we get input $\boldsymbol{x}_{t+1}$, rather than predicting a label for $\boldsymbol{x}_{t+1}$ by comparing it to some parametric model for each class, we just find the example in the gallery that is nearest (most similar) to $\boldsymbol{x}_{t+1}$, call it $\boldsymbol{x}'$. We then need to determine if $\boldsymbol{x}'$ and $\boldsymbol{x}_{t+1}$ are sufficiently similar to constitute a match. (In the context of person classification, this is known as **person re-identification** or **face verification**, see e.g., [WSH16]).) If there is no match, we can declare the input to be novel or OOD.

The key ingredient for all of the above problems is the (dis)similarity metric between inputs. We discuss ways to learn this in Section 16.2.

16.1.4.2 Other open world problems

The problem of open-set recognition, and incremental learning, are just examples of problems that require the **open world assumption** c.f., [Rus15]. There are many other examples of such problems.

For example, consider the problem of **entity resolution**, called **entity linking**. In this problem, we need to determine if different strings (e.g., "John Smith" and "Jon Smith") refer to the same entity or not. See e.g. [SHF15] for details.

Another important application is in **multi-object tracking**. For example, when a radar system detects a new "blip", is it due to an existing missile that is being tracked, or is it a new objective that has entered the airspace? An elegant mathematical framework for dealing with such problems, known as **random finite sets**, is described in [Mah07; Mah13; Vo+15].

16.2 Learning distance metrics

Being able to compute the "semantic distance" between a pair of points, $d(\boldsymbol{x}, \boldsymbol{x}') \in \mathbb{R}^+$ for $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{X}$, or equivalently their similarity $s(\boldsymbol{x}, \boldsymbol{x}') \in \mathbb{R}^+$, is of crucial importance to tasks such as nearest neighbor classification (Section 16.1), self-supervised learning (Section 19.2.4.4), similarity-based clustering (Section 21.5), content-based retrieval, visual tracking, etc.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

When the input space is $\mathcal{X} = \mathbb{R}^D$, the most common distance metric is the Mahalanobis distance

$$d_{\mathbf{M}}(\boldsymbol{x}, \boldsymbol{x}') = \sqrt{(\boldsymbol{x} - \boldsymbol{x}')^{\mathsf{T}} \mathbf{M} (\boldsymbol{x} - \boldsymbol{x}')} \tag{16.3}$$

We discuss some methods to learn the matrix $\mathbf{M}$ in Section 16.2.1. For high dimensional inputs, or structured inputs, it is better to first learn an embedding $\boldsymbol{e} = f(\boldsymbol{x})$, and then to compute distances in embedding space. When f is a DNN, this is called **deep metric learning**; we discuss this in Section 16.2.2.

16.2.1 Linear and convex methods

In this section, we discuss some methods that try to learn the Mahalanobis distance matrix $\mathbf{M}$, either directly (as a convex problem), or indirectly via a linear projection. For other approaches to metric learning, see e.g., [Kul13; Kim19] for more details.

16.2.1.1 Large margin nearest neighbors

In [WS09], they propose to learn the Mahalanobis matrix $\mathbf{M}$ so that the resulting distance metric works well when used by a nearest neighbor classifier. The resulting method is called **large margin nearest neighbor** or **LMNN**.

This works as follows. For each example data point i, let N_i be a set of **target neighbors**; these are usually chosen to be the set of K points with the same class label that are closest in Euclidean distance. We now optimize $\mathbf{M}$ so that we minimize the distance between each point i and all of its target neighbors $j \in N_i$:

$$\mathcal{L}_{\text{pull}}(\mathbf{M}) = \sum_{i=1}^{N} \sum_{j \in N_i} d_{\mathbf{M}}(\boldsymbol{x}_i, \boldsymbol{x}_j)^2 \tag{16.4}$$

We also want to ensure that examples with incorrect labels are far away. To do this, we ensure that each example i is closer (by some margin $m \geq 0$) to its target neighbors j than to other points l with different labels (so-called **impostors**). We can do this by minimizing

$$\mathcal{L}_{\text{push}}(\mathbf{M}) = \sum_{i=1}^{N} \sum_{j \in N_i} \sum_{l=1}^{N} \mathbb{I}\left(y_i \neq y_l\right) \left[m + d_{\mathbf{M}}(\boldsymbol{x}_i, \boldsymbol{x}_j)^2 - d_{\mathbf{M}}(\boldsymbol{x}_i, \boldsymbol{x}_l)^2\right]_+ \tag{16.5}$$

where $[z]_+ = \max(z, 0)$ is the hinge loss function (Section 4.3.2). The overall objective is $\mathcal{L}(\mathbf{M}) = (1 - \lambda)\mathcal{L}_{\text{pull}}(\mathbf{M}) + \lambda \mathcal{L}_{\text{push}}(\mathbf{M})$, where $0 < \lambda < 1$. This is a convex function defined over a convex set, which can be minimized using **semidefinite programming**. Alternatively, we can parameterize the problem using $\mathbf{M} = \mathbf{W}^{\mathsf{T}}\mathbf{W}$, and then minimize wrt $\mathbf{W}$ using unconstrained gradient methods. This is no longer convex, but allows us to use a low-dimensional mapping $\mathbf{W}$.

For large datasets, we need to tackle the $O(N^3)$ cost of computing Equation (16.5). We discuss some speedup tricks in Section 16.2.5.

16.2.1.2 Neighborhood components analysis

Another way to learn a linear mapping $\mathbf{W}$ such that $\mathbf{M} = \mathbf{W}^{\mathsf{T}}\mathbf{W}$ is known as **neighborhood components analysis** or **NCA** [Gol+05]. This defines the probability that sample $\boldsymbol{x}_i$ has $\boldsymbol{x}_j$ as its

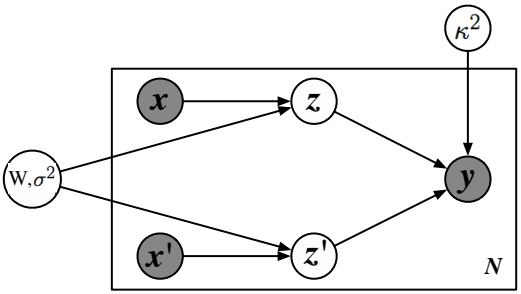


Figure 16.4: Illustration of latent coincidence analysis (LCA) as a directed graphical model. The inputs $\boldsymbol{x}, \boldsymbol{x}' \in \mathbb{R}^D$ are mapped into Gaussian latent variables $\boldsymbol{z}, \boldsymbol{z}' \in \mathbb{R}^L$ via a linear mapping $\mathbf{W}$. If the two latent points coincide (within length scale κ) then we set the similarity label to $y = 1$, otherwise we set it to $y = 0$. From Figure 1 of [DS12]. Used with kind permission of Lawrence Saul.

nearest neighbor using the linear softmax function

$$p_{ij}^{\mathbf{W}} = \frac{\exp(-||\mathbf{W}\boldsymbol{x}_i - \mathbf{W}\boldsymbol{x}_j||_2^2)}{\sum_{l \neq i} \exp(-||\mathbf{W}\boldsymbol{x}_i - \mathbf{W}\boldsymbol{x}_l||_2^2)} \tag{16.6}$$

(This is a supervised version of stochastic neighborhood embeddings discussed in Section 20.4.10.1.) The expected number of correctly classified examples according for a 1NN classifier using distance $\mathbf{W}$ is given by $J(\mathbf{W}) = \sum_{i=1}^N \sum_{j \neq i: y_j = y_i} p_{ij}^{\mathbf{W}}$. Let $\mathcal{L}(\mathbf{W}) = 1 - J(\mathbf{W})/N$ be the leave one out error. We can minimize $\mathcal{L}$ wrt $\mathbf{W}$ using gradient methods.

16.2.1.3 Latent coincidence analysis

Yet another way to learn a linear mapping $\mathbf{W}$ such that $\mathbf{M} = \mathbf{W}^\mathsf{T}\mathbf{W}$ is known as **latent coincidence analysis** or **LCA** [DS12]. This defines a conditional latent variable model for mapping a pair of inputs, $\boldsymbol{x}$ and $\boldsymbol{x}'$, to a label $y \in \{0, 1\}$, which specifies if the inputs are similar (e.g., have same class label) or dissimilar. Each input $\boldsymbol{x} \in \mathbb{R}^D$ is mapped to a low dimensional latent point $\boldsymbol{z} \in \mathbb{R}^L$ using a stochastic mapping $p(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|\mathbf{W}\boldsymbol{x}, \sigma^2\mathbf{I})$, and $p(\boldsymbol{z}'|\boldsymbol{x}') = \mathcal{N}(\boldsymbol{z}'|\mathbf{W}\boldsymbol{x}', \sigma^2\mathbf{I})$. (Compare this to factor analysis, discussed in Section 20.2.) We then define the probability that the two inputs are similar using $p(y = 1|\boldsymbol{z}, \boldsymbol{z}') = \exp(-\frac{1}{2\kappa^2}||\boldsymbol{z} - \boldsymbol{z}'||)$. See Figure 16.4 for an illustration of the modeling assumptions.

We can maximize the log marginal likelihood $\ell(\mathbf{W}, \sigma^2, \kappa^2) = \sum_n \log p(y_n|\boldsymbol{x}_n, \boldsymbol{x}'_n)$ using the EM algorithm (Section 8.7.2). (We can set $\kappa = 1$ WLOG, since it just changes the scale of $\mathbf{W}$.) More precisely, in the E step, we compute the posterior $p(\boldsymbol{z}, \boldsymbol{z}'|\boldsymbol{x}, \boldsymbol{x}', y)$ (which can be done in closed form), and in the M step, we solve a weighted least squares problem (c.f., Section 13.6.2). EM will monotonically increase the objective, and does not need step size adjustment, unlike the gradient based methods used in NCA (Section 16.2.1.2). (It is also possible to use variational Bayes (Section 4.6.8.3) to fit this model, as well as various sparse and nonlinear extensions, as discussed in [ZMY19].)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

16.2.2 Deep metric learning

When measuring the distance between high-dimensional or structured inputs, it is very useful to first learn an embedding to a lower dimensional "semantic" space, where distances are more meaningful, and less subject to the curse of dimensionality (Section 16.1.2). Let $\boldsymbol{e} = f(\boldsymbol{x}; \boldsymbol{\theta}) \in \mathbb{R}^L$ be an embedding of the input that preserves the "relevant" semantic aspects of the input, and let $\hat{\boldsymbol{e}} = \boldsymbol{e}/||\boldsymbol{e}||_2$ be the ℓ_2-normalized version. This ensures that all points lie on a hyper-sphere. We can then measure the distance between two points using the normalized Euclidean distance

$$d(\boldsymbol{x}_i, \boldsymbol{x}_j; \boldsymbol{\theta}) = ||\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_j||_2^2 \tag{16.7}$$

where smaller values means more similar, or the cosine similarity

$$d(\boldsymbol{x}_i, \boldsymbol{x}_j; \boldsymbol{\theta}) = \hat{\boldsymbol{e}}_i^{\mathsf{T}} \hat{\boldsymbol{e}}_j \tag{16.8}$$

where larger values means more similar. (Cosine similarity measures the angle between the two vectors, as illustrated in Figure 20.43.) These quantities are related via

$$||\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_j||_2^2 = (\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_j)^{\mathsf{T}}(\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_j) = 2 - 2\hat{\boldsymbol{e}}_i^{\mathsf{T}} \hat{\boldsymbol{e}}_j \tag{16.9}$$

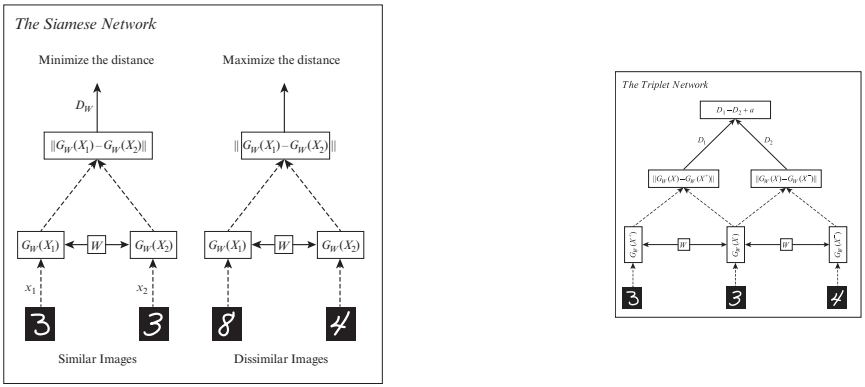
This overall approach is called **deep metric learning** or DML.

The basic idea in DML is to learn the embedding function such that similar examples are closer than dissimilar examples. More precisely, we assume we have a labeled dataset, $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) : i = 1 : N\}$, from which we can derive a set of similar pairs, $\mathcal{S} = \{(i, j) : y_i = y_j\}$. If $(i, j) \in \mathcal{S}$ but $(i, k) \notin S$, then we assume that $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ should be close in embedding space, whereas $\boldsymbol{x}_i$ and $\boldsymbol{x}_k$ should be far. We discuss various ways to enforce this property below. Note that these methods also work when we do not have class labels, provided we have some other way of defining similar pairs. For example, in Section 19.2.4.3, we discuss self-supervised approaches to representation learning, that automatically create semantically similar pairs, and learn embeddings to force these pairs to be closer than unrelated pairs.

Before discussing DML in more detail, it is worth mentioning that many recent approaches to DML are not as good as they claim to be, as pointed out in [MBL20; Rot+20]. (The claims in some of these papers are often invalid due to improper experimental comparisons, a common flaw in contemporary ML research, as discussed in e.g., [BLV19; LS19b].) We therefore focus on (slightly) older and simpler methods, that tend to be more robust.

16.2.3 Classification losses

Suppose we have labeled data with C classes. Then we can fit a classification model in $O(NC)$ time, and then reuse the hidden features as an embedding function. (It is common to use the second-to-last layer, since it generalizes better to new classes than the final layer.) This approach is simple and scalable. However, it only learns to embed examples on the correct side of a decision boundary, which does not necessarily result in similar examples being placed close together and dissimilar examples being placed far apart. In addition, this method cannot be used if we do not have labeled training data.



(a) (b)

Figure 16.5: Networks for deep metric learning. (a) Siamese network. (b) Triplet network. Adapted from Figure 5 of [KB19].

16.2.4 Ranking losses

In this section, we consider minimizing **ranking loss**, to ensure that similar examples are closer than dissimilar examples. Most of these methods do not need class labels (although we sometimes assume that labels exist as a notationally simple way to define similarity).

16.2.4.1 Pairwise (contrastive) loss and Siamese networks

One of the earliest approaches to representation learning from similar/dissimilar pairs was based on minimizing the following **contrastive loss** [CHL05]:

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{x}_i, \boldsymbol{x}_j) = \mathbb{I}(y_i = y_j)\, d(\boldsymbol{x}_i, \boldsymbol{x}_j)^2 + \mathbb{I}(y_i \neq y_j)\, [m - d(\boldsymbol{x}_i, \boldsymbol{x}_j)]_+^2 \tag{16.10}$$

where $[z]_+ = \max(0, z)$ is the hinge loss and $m > 0$ is a margin parameter. Intuitively, we want to force positive pairs (with the same label) to be close, and negative pairs (with different labels) to be further apart than some minimal safety margin. We minimize this loss over all pairs of data. Naively this takes $O(N^2)$ time; see Section 16.2.5 for some speedups.

Note that we use the same feature extractor $\boldsymbol{f}(\cdot; \boldsymbol{\theta})$ for both inputs, $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$. when computing the distance, as illustrated in Figure 16.5a. The resulting network is therefore called a **Siamese network** (named after Siamese twins).

16.2.4.2 Triplet loss

One disadvantage of pairwise losses is that the optimization of the positive pairs is independent of the negative pairs, which can make their magnitudes incomparable. A solution to this is to use the **triplet loss** [SKP15]. This is defined as follows. For each example i (known as an **anchor**), we find a similar (positive) example $\boldsymbol{x}_i^+$ and a dissimilar (negative) example $\boldsymbol{x}_i^-$. We then minimize the following loss, averaged overall all triples:

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{x}_i, \boldsymbol{x}_i^+, \boldsymbol{x}_i^-) = [d_{\boldsymbol{\theta}}(\boldsymbol{x}_i, \boldsymbol{x}_i^+)^2 - d_{\boldsymbol{\theta}}(\boldsymbol{x}_i, \boldsymbol{x}_i^-)^2 + m]_+ \tag{16.11}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Intuitively this says we want the distance from the anchor to the positive to be less (by some safety margin m) than the distance from the anchor to the negative. We can compute the triplet loss using a triplet network as shown in Figure 16.5b.

Naively minimizing triplet loss takes $O(N^3)$ time. In practice we compute the loss on a minibatch (chosen so that there is at least one similar and one dissimilar example for the anchor point, often taken to be the first entry in the minibatch). Nevertheless the method can be slow. We discuss some speedups in Section 16.2.5.

16.2.4.3 N-pairs loss

One problem with the triplet loss is that each anchor is only compared to one negative example at a time. This might not provide a strong enough learning signal. One solution to this is to create a multi-class classification problem in which we create a set of $N-1$ negatives and 1 positive for every anchor. This is called the **N-pairs loss** [Soh16]. More precisely, we define the following loss for each set:

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{x}^+, \{\boldsymbol{x}_k^-\}_{k=1}^{N-1}) = \log\left(1 + \sum_{k=1}^{N-1} \exp\left[\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}_k^-) - \hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}^+)\right]\right) \tag{16.12}$$

$$= -\log \frac{\exp(\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}^+))}{\exp(\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}^+)) + \sum_{k=1}^{N-1}\exp(\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}_k^-))} \tag{16.13}$$

Note that the N-pairs loss is the same as the **InfoNCE** loss used in the CPC paper [OLV18]. In [Che+20a], they propose a version where they scale the similarities by a temperature term; they call this the **NT-Xent** (normalized temperature-scaled cross-entropy) loss. We can view the temperature parameter as scaling the radius of the hypersphere on which the data lives.

When $N = 2$, the loss reduces to the logistic loss

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{x}^+, \boldsymbol{x}^-) = \log\left(1 + \exp(\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}^-) - \hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}^+))\right) \tag{16.14}$$

Compare this to the margin loss used by triplet learning (when $m = 1$):

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{x}, \boldsymbol{x}^+, \boldsymbol{x}^-) = \max\left(0, \hat{\boldsymbol{e}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}(\boldsymbol{x}^-) - \hat{\boldsymbol{e}}(\boldsymbol{x})^\mathsf{T}\hat{\boldsymbol{e}}(\boldsymbol{x}^+) + 1\right) \tag{16.15}$$

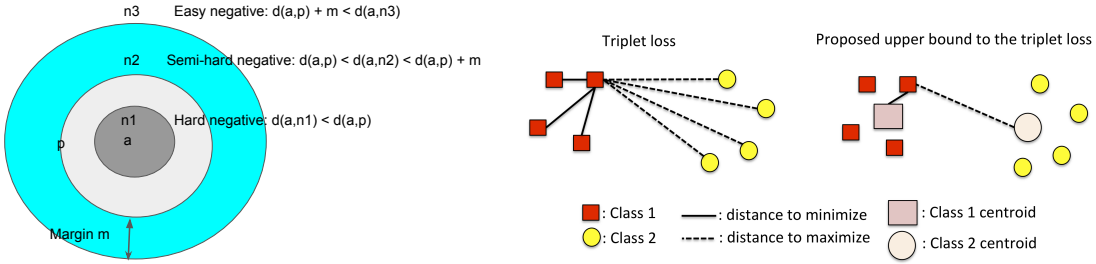
See Figure 4.2 for a comparison of these two functions.

16.2.5 Speeding up ranking loss optimization

The main disadvantage of ranking loss is the $O(N^2)$ or $O(N^3)$ cost of computing the loss function, due to the need to compare all pairs or triples of examples. In this section, we discuss various speedup tricks.

16.2.5.1 Mining techniques

A key insight is that we don't need to consider all negative examples for each anchor, since most will be uninformative (i.e., will incur zero loss). Instead we can focus attention on negative examples which are closer to the anchor than its nearest positive example. These are called **hard negatives**, and are particularly useful for speeding up triplet loss.



(a) *(b)*

Figure 16.6: Speeding up triplet loss minimization. (a) Illustration of hard vs easy negatives. Here a is the anchor point, p is a positive point, and n_i are negative points. Adapted from Figure 4 of [KB19]. (b) Standard triplet loss would take $8 \times 3 \times 4 = 96$ calculations, whereas using a proxy loss (with one proxy per class) takes $8 \times 2 = 16$ calculations. From Figure 1 of [Do+19]. Used with kind permission of Gustavo Cerneiro.

More precisely, if a is an anchor and p is its nearest positive example, we say that n is a hard negative (for a) if $d(\boldsymbol{x}_a, \boldsymbol{x}_n) < d(\boldsymbol{x}_a, \boldsymbol{x}_p)$ and $y_n \neq y_a$. Sometimes an anchor may not have any hard negatives. We can therefore increase the pool of candidates by considering **semi-hard negatives**, for which

$$d(\boldsymbol{x}_a, \boldsymbol{x}_p) < d(\boldsymbol{x}_a, \boldsymbol{x}_n) < d(\boldsymbol{x}_a, \boldsymbol{x}_p) + m \tag{16.16}$$

where $m > 0$ is a margin parameter. See Figure 16.6a for an illustration. This is the technique used by Google's **FaceNet** model [SKP15], which learns an embedding function for faces, so it can cluster similar looking faces together, to which the user can attach a name.

In practice, the hard negatives are usually chosen from within the minibatch. This therefore requires large batch sizes to ensure sufficient diversity. Alternatively, we can have a separate process that continually updates the set of candidate hard negatives, as the distance measure evolves during training.

16.2.5.2 Proxy methods

Triplet loss minimization is expensive even with hard negative mining (Section 16.2.5.1). Ideally we can find a method that is $O(N)$ time, just like classification loss.

One such method, proposed in [MA+17], measures the distance between each anchor and a set of P **proxies** that represent each class, rather than directly measuring distance between examples. These proxies need to be updated online as the distance metric evolves during learning. The overall procedure takes $O(NP^2)$ time, where $P \sim C$.

More recently, [Qia+19] proposed to represent each class with multiple prototypes, while still achieving linear time complexity, using a **soft triple** loss.

16.2.5.3 Optimizing an upper bound

[Do+19] proposed a simple and fast method for optimizing the triplet loss. The key idea is to define one *fixed* proxy or centroid per class, and then to use distance to the proxy as an upper bound on

the triplet loss.

More precisely, consider a simplified form of the triplet loss, without the margin term:

$$\ell_t(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{x}_k) = ||\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_j|| - ||\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_k|| \tag{16.17}$$

where $\hat{\boldsymbol{e}}_i = \hat{\boldsymbol{e}}_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$, etc. Using the triangle inequality we have

$$||\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_j|| \leq ||\hat{\boldsymbol{e}}_i - \boldsymbol{c}_{y_i}|| + ||\hat{\boldsymbol{e}}_j - \boldsymbol{c}_{y_i}|| \tag{16.18}$$

$$||\hat{\boldsymbol{e}}_i - \hat{\boldsymbol{e}}_k|| \geq ||\hat{\boldsymbol{e}}_i - \boldsymbol{c}_{y_k}|| - ||\hat{\boldsymbol{e}}_k - \boldsymbol{c}_{y_k}|| \tag{16.19}$$

Hence

$$\ell_t(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{x}_k) \leq \ell_u(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{x}_k) \triangleq ||\hat{\boldsymbol{e}}_i - \boldsymbol{c}_{y_i}|| - ||\hat{\boldsymbol{e}}_i - \boldsymbol{c}_{y_k}|| + ||\hat{\boldsymbol{e}}_j - \boldsymbol{c}_{y_i}|| + ||\hat{\boldsymbol{e}}_k - \boldsymbol{c}_{y_k}|| \tag{16.20}$$

We can use this to derive a tractable upper bound on the triplet loss as follows:

$$\mathcal{L}_t(\mathcal{D}, \mathcal{S}) = \sum_{(i,j)\in\mathcal{S}, (i,k)\notin\mathcal{S}, i,j,k\in\{1,\ldots,N\}} \ell_t(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{x}_k) \leq \sum_{(i,j)\in\mathcal{S}, (i,k)\notin\mathcal{S}, i,j,k\in\{1,\ldots,N\}} \ell_u(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{x}_k) \tag{16.21}$$

$$= C' \sum_{i=1}^{N} \left(||\boldsymbol{x}_i - \boldsymbol{c}_{y_i}|| - \frac{1}{3(C-1)} \sum_{m=1, m\neq y_i}^{C} ||\boldsymbol{x}_i - \boldsymbol{c}_m|| \right) \triangleq \mathcal{L}_u(\mathcal{D}, \mathcal{S}) \tag{16.22}$$

where $C' = 3(C-1)(\frac{N}{C} - 1)\frac{N}{C}$ is a constant, and we assume that $\sum_{i=1}^{N} \mathbb{I}(y_i = c) = N/C$ for each c. It is clear that $\mathcal{L}_u$ can be computed in $O(NC)$ time. See Figure 16.6b for an illustration.

In [Do+19], they show that $0 \leq \mathcal{L}_t - \mathcal{L}_u \leq \frac{N^3}{C^2} K$, where K is some constant that depends on the spread of the centroids. To ensure the bound is tight, the centroids should be as far from each other as possible, and the distances between them should be as similar as possible. An easy way to ensure is to define the $\boldsymbol{c}_m$ vectors to be one-hot vectors, one per class. These vectors already have unit norm, and are orthogonal to each other. The distance between each pair of centroids is $\sqrt{2}$, which ensures the upper bound is fairly tight.

The downside of this approach is that it assumes the embedding layer is $L = C$ dimensional. There are two solutions to this. First, after training, we can add a linear projection layer to map from C to $L \neq C$, or we can take the second-to-last layer of the embedding network. The second approach is to sample a large number of points on the L-dimensional unit hyper-sphere (which we can do by sampling from the standard normal, and then normalizing [Mar72]), and then running K-means clustering (Section 21.3) with $K = C$. In the experiments reported in [Do+19], these two approaches give similar results.

Interestingly, in [Rot+20], they show that increasing $\pi_{\text{intra}}/\pi_{\text{inter}}$ results in improved downstream performance on various retrieval tasks, where

$$\pi_{\text{intra}} = \frac{1}{Z_{\text{intra}}} \sum_{c=1}^{C} \sum_{i\neq j: y_i = y_j = c} d(\boldsymbol{x}_i, \boldsymbol{x}_j) \tag{16.23}$$

is the average intra-class distance, and

$$\pi_{\text{inter}} = \frac{1}{Z_{\text{inter}}} \sum_{c=1}^{C} \sum_{c'=1}^{C} d(\boldsymbol{\mu}_c, \boldsymbol{\mu}_{c'}) \tag{16.24}$$

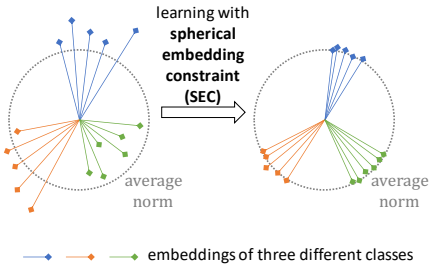


Figure 16.7: Adding spherical embedding constraint to a deep metric learning method. Used with kind permission of Dingyi Zhang.

is the average inter-class distance, where $\boldsymbol{\mu}_c = \frac{1}{Z_c} \sum_{i:y_i=c} \hat{\boldsymbol{e}}_i$ is the mean embedding for examples from class c. This suggests that we should not only keep the centroids far apart (in order to maximize the numerator), but we should also prevent examples from getting too close to their centroids (in order to minimize the denominator); this latter term is not captured in the method of [Do+19].

16.2.6 Other training tricks for DML

Besides the speedup tricks in Section 16.2.5, there are a lot of other details that are important to get right in order to ensure good DML performance. Many of these details are discussed in [MBL20; Rot+20]. Here we just briefly mention a few.

One important issue is how the minibatches are created. In classification problems (at least with balanced classes), selecting examples at random from the training set is usually sufficient. However, for DML, we need to ensure that each example has some other examples in the minibatch that are similar to it, as well as some others that are dissimilar to it. One approach is to use hard mining techniques (Section 16.2.5.1). Another idea is to use coreset methods applied to previously learned embeddings to select a diverse minibatch at each step [Sin+20]. However, [Rot+20] show that the following simple strategy also works well for creating each batch: pick B/n classes, and then pick N_c examples randomly from each class, where B is the batch size, and $N_c = 2$ is a tuning parameter.

Another important issue is avoiding overfitting. Since most datasets used in the DML literature are small, it is standard to use an image classifier, such as GoogLeNet (Section 14.3.3) or ResNet (Section 14.3.4), which has been pre-trained on ImageNet, and then to fine-tune the model using the DML loss. (See Section 19.2 for more details on this kind of transfer learning.) In addition, it is standard to use data augmentation (see Section 19.1). (Indeed, with some self-supervised learning methods, data aug is the only way to create similar pairs.)

In [ZLZ20], they propose to add a **spherical embedding constraint** (SEC), which is an additional batchwise regularization term, which encourages all the examples to have the same norm. That is, the regularizer is just the empirical variance of the norms of the (unnormalized) embeddings in that batch. See Figure 16.7 for an illustration. This regularizer can be added to any of the existing DML losses to modestly improve training speed and stability, as well as final performance, analogously to how batchnorm (Section 14.2.4.1) is used.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

16.3 Kernel density estimation (KDE)

In this section, we consider a form of non-parametric density estimation known as **kernel density estimation** or **KDE**. This is a form of generative model, since it defines a probability distribution $p(\boldsymbol{x})$ that can be evaluated pointwise, and which can be sampled from to generate new data.

16.3.1 Density kernels

Before explaining KDE, we must define what we mean by a "kernel". This term has several different meanings in machine learning and statistics.[1] In this section, we use a specific kind of kernel which we refer to as a **density kernel**. This is a function $\mathcal{K} : \mathbb{R} \to \mathbb{R}_+$ such that $\int \mathcal{K}(x)dx = 1$ and $\mathcal{K}(-x) = \mathcal{K}(x)$. This latter symmetry property implies the $\int x\mathcal{K}(x)dx = 0$, and hence

$$\int x\mathcal{K}(x - x_n)dx = x_n \tag{16.25}$$

A simple example of such a kernel is the **boxcar kernel**, which is the uniform distribution within the unit interval around the origin:

$$\mathcal{K}(x) \triangleq 0.5\mathbb{I}\left(|x| \leq 1\right) \tag{16.26}$$

Another example is the **Gaussian kernel**:

$$\mathcal{K}(x) = \frac{1}{(2\pi)^{\frac{1}{2}}} e^{-x^2/2} \tag{16.27}$$

We can control the width of the kernel by introducing a **bandwidth** parameter h:

$$\mathcal{K}_h(x) \triangleq \frac{1}{h}\mathcal{K}(\frac{x}{h}) \tag{16.28}$$

We can generalize to vector valued inputs by defining a **radial basis function** or **RBF** kernel:

$$\mathcal{K}_h(\boldsymbol{x}) \propto \mathcal{K}_h(||\boldsymbol{x}||) \tag{16.29}$$

In the case of the Gaussian kernel, this becomes

$$\mathcal{K}_h(\boldsymbol{x}) = \frac{1}{h^D(2\pi)^{D/2}} \prod_{d=1}^{D} \exp(-\frac{1}{2h^2}x_d^2) \tag{16.30}$$

Although Gaussian kernels are popular, they have unbounded support. Some alternative kernels, which have compact support (which can be computationally faster), are listed in Table 16.1. See Figure 16.8 for a plot of these kernel functions.

1. For a good blog post on this, see https://francisbach.com/cursed-kernels/.

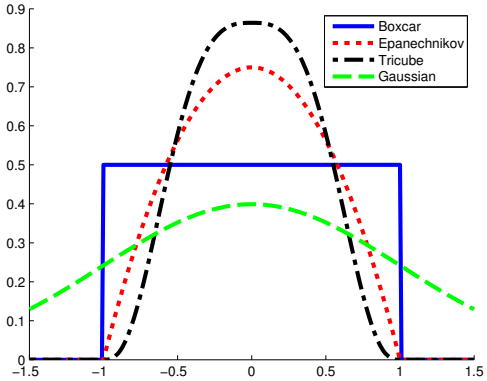


Figure 16.8: A comparison of some popular normalized kernels. Generated by smoothingKernelPlot.ipynb.

Name	Definition	Compact	Smooth	Boundaries
Gaussian	$\mathcal{K}(x) = (2\pi)^{-\frac{1}{2}} e^{-x^2/2}$	0	1	1
Boxcar	$\mathcal{K}(x) = \frac{1}{2}\mathbb{I}(\lvert x\rvert \leq 1)$	1	0	0
Epanechnikov kernel	$\mathcal{K}(x) = \frac{3}{4}(1-x^2)\mathbb{I}(\lvert x\rvert \leq 1)$	1	1	0
Tri-cube kernel	$\mathcal{K}(x) = \frac{70}{81}(1-\lvert x\rvert^3)^3\mathbb{I}(\lvert x\rvert \leq 1)$	1	1	1

Table 16.1: List of some popular normalized kernels in 1d. Compact=1 means the function is non-zero for a finite range of inputs. Smooth=1 means the function is differentiable over the range of its support. Boundaries=1 means the function is also differentiable at the boundaries of its support.

16.3.2 Parzen window density estimator

To explain how to use kernels to define a nonparametric density estimate, recall the form of the Gaussian mixture model from Section 3.5.1. If we assume a fixed spherical Gaussian covariance and uniform mixture weights, we get

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = \frac{1}{K}\sum_{k=1}^{K}\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \sigma^2\mathbf{I}) \tag{16.31}$$

One problem with this model is that it requires specifying the number K of clusters, as well as their locations $\boldsymbol{\mu}_k$. An alternative to estimating these parameters is to allocate one cluster center per data point. In this case, the model becomes

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{N}(\boldsymbol{x}|\boldsymbol{x}_n, \sigma^2\mathbf{I}) \tag{16.32}$$

We can generalize Equation (16.32) by writing

$$p(\boldsymbol{x}|\mathcal{D}) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_n) \tag{16.33}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

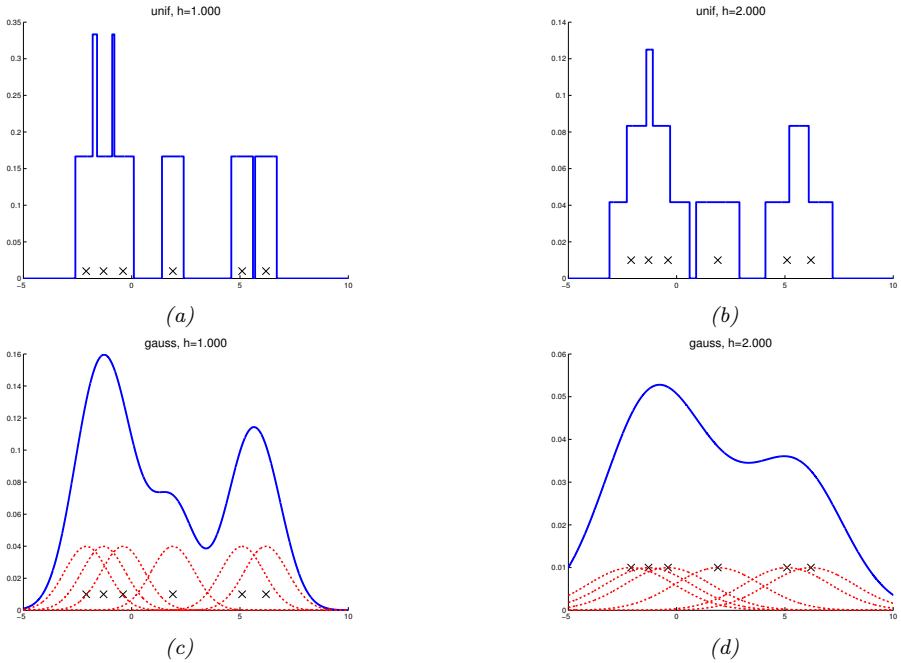


Figure 16.9: A nonparametric (Parzen) density estimator in 1d estimated from 6 data points, denoted by x. Top row: uniform kernel. Bottom row: Gaussian kernel. Left column: bandwidth parameter $h = 1$. Right column: bandwidth parameter $h = 2$. Adapted from http://en.wikipedia.org/wiki/Kernel_density_estimation*. Generated by parzen_window_demo2.ipynb.*

where $\mathcal{K}_h$ is a density kernel. This is called a **Parzen window density estimator**, or **kernel density estimator** (**KDE**).

The advantage over a parametric model is that no model fitting is required (except for choosing h, discussed in Section 16.3.3), and there is no need to pick the number of cluster centers. The disadvantage is that the model takes a lot of memory (you need to store all the data) and a lot of time to evaluate.

Figure 16.9 illustrates KDE in 1d for two kinds of kernel. On the top, we use a boxcar kernel; the resulting model just counts how many data points land within an interval of size h around each x_n to get a piecewise constant density. On the bottom, we use a Gaussian kernel, which results in a smoother density.

16.3.3 How to choose the bandwidth parameter

We see from Figure 16.9 that the bandwidth parameter h has a large effect on the learned distribution. We can view this as controlling the complexity of the model.

In the case of 1d data, where the "true" data generating distribution is assumed to be a Gaussian, one can show [BA97a] that the optimal bandwidth for a Gaussian kernel (from the point of view of

minimizing frequentist risk) is given by $h = \sigma \left(\frac{4}{3N}\right)^{1/5}$. We can compute a robust approximation to the standard deviation by first computing the **median absolute deviation**, $\text{median}(|\boldsymbol{x} - \text{median}(\boldsymbol{x})|)$, and then using $\hat{\sigma} = 1.4826$ MAD. If we have D dimensions, we can estimate h_d separately for each dimension, and then set $h = (\prod_{d=1}^{D} h_d)^{1/D}$.

16.3.4 From KDE to KNN classification

In Section 16.1, we discussed the K nearest neighbor classifier as a heuristic approach to classification. Interestingly, we can derive it as a generative classifier in which the class conditional densities $p(\boldsymbol{x}|y = c)$ are modeled using KDE. Rather than using a fixed bandwidth and counting how many data points fall within the hyper-cube centered on a datapoint, we will allow the bandwidth or volume to be different for each data point. Specifically, we will "grow" a volume around $\boldsymbol{x}$ until we encounter K data points, regardless of their class label. This is called a **balloon kernel density estimator** [TS92]. Let the resulting volume have size $V(\boldsymbol{x})$ (this was previously h^D), and let there be $N_c(\boldsymbol{x})$ examples from class c in this volume. Then we can estimate the class conditional density as follows:

$$p(\boldsymbol{x}|y = c, \mathcal{D}) = \frac{N_c(\boldsymbol{x})}{N_c V(\boldsymbol{x})} \tag{16.34}$$

where N_c is the total number of examples in class c in the whole data set. If we take the class prior to be $p(y = c) = N_c/N$, then the class posterior is given by

$$p(y = c|\boldsymbol{x}, \mathcal{D}) = \frac{\frac{N_c(\boldsymbol{x})}{N_c V(\boldsymbol{x})} \frac{N_c}{N}}{\sum_{c'} \frac{N_{c'}(\boldsymbol{x})}{N_{c'} V(\boldsymbol{x})} \frac{N_{c'}}{N}} = \frac{N_c(\boldsymbol{x})}{\sum_{c'} N_{c'}(\boldsymbol{x})} = \frac{N_c(\boldsymbol{x})}{K} = \frac{1}{K} \sum_{n \in N_K(\boldsymbol{x}, \mathcal{D})} \mathbb{I}(y_n = c) \tag{16.35}$$

where we used the fact that $\sum_c N_c(\boldsymbol{x}) = K$, since we choose a total of K points (regardless of class) around every point. This matches Equation (16.1).

16.3.5 Kernel regression

Just as KDE can be used for generative classifiers (see Section 16.1), it can also be used for generative models for regression, as we discuss below.

16.3.5.1 Nadaraya-Watson estimator for the mean

In regression, our goal is to compute the conditional expectation

$$\mathbb{E}[y|\boldsymbol{x}, \mathcal{D}] = \int y \; p(y|\boldsymbol{x}, \mathcal{D}) dy = \frac{\int y \; p(\boldsymbol{x}, y|\mathcal{D}) dy}{\int p(\boldsymbol{x}, y|\mathcal{D}) dy} \tag{16.36}$$

If we use an MVN for $p(y, \boldsymbol{x}|\mathcal{D})$, we derive a result which is equivalent to linear regression, as we showed in Section 11.2.3.5. However, the assumption that $p(y, \boldsymbol{x}|\mathcal{D})$ is Gaussian is rather limiting. We can use KDE to more accurately approximate the joint density $p(\boldsymbol{x}, y|\mathcal{D})$ as follows:

$$p(y, \boldsymbol{x}|\mathcal{D}) \approx \frac{1}{N} \sum_{n=1}^{N} \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n) \mathcal{K}_h(y - y_n) \tag{16.37}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

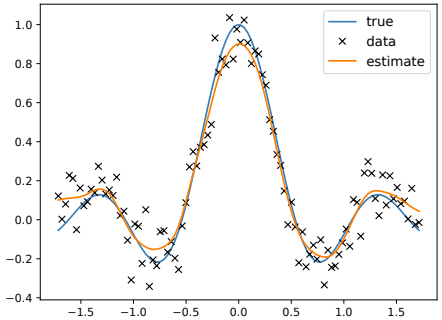


Figure 16.10: An example of kernel regression in 1d using a Gaussian kernel. Generated by kernelRegressionDemo.ipynb.

Hence

$$\mathbb{E}\left[y|\boldsymbol{x},\mathcal{D}\right] = \frac{\frac{1}{N}\sum_{n=1}^{N}\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_n)\int y\mathcal{K}_h(y-y_n)dy}{\frac{1}{N}\sum_{n'=1}^{N}\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_{n'})\int \mathcal{K}_h(y-y_{n'})dy} \tag{16.38}$$

We can simplify the numerator using the fact that $\int y\mathcal{K}_h(y-y_n)dy = y_n$ (from Equation (16.25)). We can simplify the denominator using the fact that density kernels integrate to one, i.e., $\int \mathcal{K}_h(y-y_n)dy = 1$. Thus

$$\mathbb{E}\left[y|\boldsymbol{x},\mathcal{D}\right] = \frac{\sum_{n=1}^{N}\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_n)y_n}{\sum_{n'=1}^{N}\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_{n'})} = \sum_{n=1}^{N} y_n w_n(\boldsymbol{x}) \tag{16.39}$$

$$w_n(\boldsymbol{x}) \triangleq \frac{\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_n)}{\sum_{n'=1}^{N}\mathcal{K}_h(\boldsymbol{x}-\boldsymbol{x}_{n'})} \tag{16.40}$$

We see that the prediction is just a weighted sum of the outputs at the training points, where the weights depend on how similar $\boldsymbol{x}$ is to the stored training points. This method is called **kernel regression**, **kernel smoothing**, or the **Nadaraya-Watson** (N-W) model. See Figure 16.10 for an example, where we use a Gaussian kernel.

In Section 17.2.3, we discuss the connection between kernel regression and Gaussian process regression.

16.3.5.2 Estimator for the variance

Sometimes it is useful to compute the predictive variance, as well as the predictive mean. We can do this by noting that

$$\mathbb{V}\left[y|\boldsymbol{x},\mathcal{D}\right] = \mathbb{E}\left[y^2|\boldsymbol{x},\mathcal{D}\right] - \mu(\boldsymbol{x})^2 \tag{16.41}$$

where $\mu(\boldsymbol{x}) = \mathbb{E}[y|\boldsymbol{x}, \mathcal{D}]$ is the N-W estimate. If we use a Gaussian kernel with variance σ^2, we can compute $\mathbb{E}\left[y^2|\boldsymbol{x}, \mathcal{D}\right]$ as follows:

$$\mathbb{E}\left[y^2|\boldsymbol{x}, \mathcal{D}\right] = \frac{\sum_{n=1}^{N} \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n) \int y^2 \mathcal{K}_h(y - y_n) dy}{\sum_{n'=1}^{N} \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_{n'}) \int \mathcal{K}_h(y - y_{n'}) dy} \tag{16.42}$$

$$= \frac{\sum_{n=1}^{N} \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n)(\sigma^2 + y_n^2)}{\sum_{n'=1}^{N} \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_{n'})} \tag{16.43}$$

where we used the fact that

$$\int y^2 \mathcal{N}(y|y_n, \sigma^2) dy = \sigma^2 + y_n^2 \tag{16.44}$$

Combining Equation (16.43) with Equation (16.41) gives

$$\mathbb{V}[y|\boldsymbol{x}, \mathcal{D}] = \sigma^2 + \sum_{n=1}^{N} w_n(\boldsymbol{x}) y_n^2 - \mu(\boldsymbol{x})^2 \tag{16.45}$$

This matches Eqn. 8 of [BA10] (modulo the initial σ^2 term).

16.3.5.3 Locally weighted regression

We can drop the normalization term from Equation (16.39) to get

$$\mu(\boldsymbol{x}) = \sum_{n=1}^{N} y_n \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n) \tag{16.46}$$

This is just a weighted sum of the observed responses, where the weights depend on how similar the test input $\boldsymbol{x}$ is to the training points $\boldsymbol{x}_n$.

Rather than just interpolating the stored responses y_n, we can fit a locally linear model around each training point:

$$\mu(\boldsymbol{x}) = \min_{\boldsymbol{\beta}} \sum_{n=1}^{N} [y_n - \boldsymbol{\beta}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}_n)]^2 \, \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n) \tag{16.47}$$

where $\boldsymbol{\phi}(\boldsymbol{x}) = [1, \boldsymbol{x}]$. This is called **locally linear regression** (LRR) or **locally-weighted scatterplot smoothing**, and is commonly known by the acronym **LOWESS** or **LOESS** [CD88]. This is often used when annotating scatter plots with local trend lines.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17 Kernel Methods *

In this chapter, we consider **nonparametric methods** for regression and classification. Such methods do not assume a fixed parametric form for the prediction function, but instead try to estimate the function itself (rather than the parameters) directly from data. The key idea is that we observe the function value at a fixed set of N points, namely $y_n = f(\boldsymbol{x}_n)$ for $n = 1 : N$, where f is the unknown function, so to predict the function value at a new point, say $\boldsymbol{x}_*$, we just have to compare how "similar" $\boldsymbol{x}_*$ is to each of the N training points, $\{\boldsymbol{x}_n\}$, and then we can predict that $f(\boldsymbol{x}_*)$ is some weighted combination of the $\{f(\boldsymbol{x}_n)\}$ values. Thus we may need to "remember" the entire training set, $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}$, in order to make predictions at test time — we cannot "compress" $\mathcal{D}$ into a fixed-sized parameter vector.

The weights that are used for prediction are determined by the similarity between $\boldsymbol{x}_*$ and each $\boldsymbol{x}_n$, which is computed using a special kind of function known as kernel function, $\mathcal{K}(\boldsymbol{x}_n, \boldsymbol{x}_*) \geq 0$, which we explain in Section 17.1. This approach is similar to RBF networks (Section 13.6.1), except we use the datapoints $\{\boldsymbol{x}_n\}$ themselves as the "anchors", rather than learning the RBF centroids $\{\boldsymbol{\mu}_k\}$.

In Section 17.2, we discuss an approach called Gaussian processes, which allows us to use the kernel to define a *prior over functions*, which we can update given data to get a *posterior over functions*. Alternatively we can use the same kernel with a method called Support Vector Machines to compute a MAP estimate of the function, as we explain in Section 17.3.

17.1 Mercer kernels

The key to nonparametric methods is that we need a way to encode prior knowledge about the similarity of two input vectors. If we know that $\boldsymbol{x}_i$ is similar to $\boldsymbol{x}_j$, then we can encourage the model to make the predicted output at both locations (i.e., $f(\boldsymbol{x}_i)$ and $f(\boldsymbol{x}_j)$) to be similar.

To define similarity, we introduce the notion of a **kernel function**. The word "kernel" has many different meanings in mathematics, including density kernels (Section 16.3.1), transition kernels of a Markov chain (Section 3.6.1.2), and convolutional kernels (Section 14.1). Here we consider a **Mercer kernel**, also called a **positive definite kernel**. This is any symmetric function $\mathcal{K} : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^+$ such that

$$\sum_{i=1}^{N} \sum_{j=1}^{N} \mathcal{K}(\boldsymbol{x}_i, \boldsymbol{x}_j) c_i c_j \geq 0 \tag{17.1}$$

for any set of N (unique) points $\boldsymbol{x}_i \in \mathcal{X}$, and any choice of numbers $c_i \in \mathbb{R}$. (We assume $\mathcal{K}(\boldsymbol{x}_i, \boldsymbol{x}_j) > 0$, so that we can only achieve equality in the above equation if $c_i = 0$ for all i.)

Another way to understand this condition is the following. Given a set of N datapoints, let us define the **Gram matrix** as the following $N \times N$ similarity matrix:

$$\mathbf{K} = \begin{pmatrix} \mathcal{K}(\boldsymbol{x}_1, \boldsymbol{x}_1) & \cdots & \mathcal{K}(\boldsymbol{x}_1, \boldsymbol{x}_N) \\ & \vdots & \\ \mathcal{K}(\boldsymbol{x}_N, \boldsymbol{x}_1) & \cdots & \mathcal{K}(\boldsymbol{x}_N, \boldsymbol{x}_N) \end{pmatrix} \tag{17.2}$$

We say that $\mathcal{K}$ is a Mercer kernel iff the Gram matrix is positive definite for any set of (distinct) inputs $\{\boldsymbol{x}_i\}_{i=1}^N$.

The most widely used kernel for real-valued inputs is the **squared exponential kernel** (SE), also called the **exponentiated quadratic kernel** (EQ), **Gaussian kernel**, or **RBF kernel**. It is defined by

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\frac{||\boldsymbol{x} - \boldsymbol{x}'||^2}{2\ell^2}\right) \tag{17.3}$$

Here ℓ corresponds to the length scale of the kernel, i.e., the distance over which we expect differences to matter. This is known as the **bandwidth** parameter. The RBF kernel measures similarity between two vectors in $\mathbb{R}^D$ using (scaled) Euclidean distance. In Section 17.1.2, we will discuss several other kinds of kernel.

In Section 17.2, we show how to use kernels to define priors and posteriors over functions. The basic idea is this: if $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}')$ is large, meaning the inputs are similar, then we expect the output of the function to be similar as well, so $f(\boldsymbol{x}) \approx f(\boldsymbol{x}')$. More precisely, information we learn about $f(\boldsymbol{x})$ will help us predict $f(\boldsymbol{x}')$ for all $\boldsymbol{x}'$ which are correlated with $\boldsymbol{x}$, and hence for which $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}')$ is large.

In Section 17.3, we show how to use kernels to generalize from Euclidean distance to a more general notion of distance, so that we can use geometric methods such as linear discriminant analysis in an implicit feature space instead of input space.

17.1.1 Mercer's theorem

Recall from Section 7.4 that any positive definite matrix $\mathbf{K}$ can be represented using an eigendecomposition of the form $\mathbf{K} = \mathbf{U}^\mathsf{T}\mathbf{\Lambda}\mathbf{U}$, where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues $\lambda_i > 0$, and $\mathbf{U}$ is a matrix containing the eigenvectors. Now consider element (i, j) of $\mathbf{K}$:

$$k_{ij} = (\mathbf{\Lambda}^{\frac{1}{2}}\mathbf{U}_{:i})^\mathsf{T}(\mathbf{\Lambda}^{\frac{1}{2}}\mathbf{U}_{:j}) \tag{17.4}$$

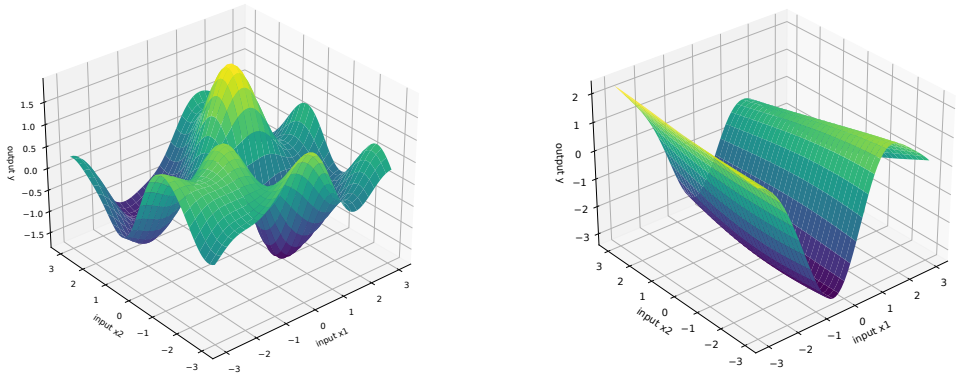
where $\mathbf{U}_{:i}$ is the i'th column of $\mathbf{U}$. If we define $\boldsymbol{\phi}(\boldsymbol{x}_i) = \mathbf{\Lambda}^{\frac{1}{2}}\mathbf{U}_{:i}$, then we can write

$$k_{ij} = \boldsymbol{\phi}(\boldsymbol{x}_i)^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}_j) = \sum_m \phi_m(\boldsymbol{x}_i)\phi_m(\boldsymbol{x}_j) \tag{17.5}$$

Thus we see that the entries in the kernel matrix can be computed by performing an inner product of some feature vectors that are implicitly defined by the eigenvectors of the kernel matrix. This idea can be generalized to apply to kernel functions, not just kernel matrices; this result is known as **Mercer's theorem**.

For example, consider the **quadratic kernel** $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \langle \boldsymbol{x}, \boldsymbol{x}' \rangle^2$. In 2d, we have

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = (x_1 x_1' + x_2 x_2')^2 = x_1^2 (x_1')^2 + 2x_1 x_2 x_1' x_2' + x_2^2 (x_2')^2 \tag{17.6}$$



(a) (b)

Figure 17.1: Function samples from a GP with an ARD kernel. (a) $\ell_1 = \ell_2 = 1$. Both dimensions contribute to the response. (b) $\ell_1 = 1$, $\ell_2 = 5$. The second dimension is essentially ignored. Adapted from Figure 5.1 of [RW06]. Generated by gprDemoArd.ipynb.

We can write this as $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{\phi}(\boldsymbol{x})^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x})$ if we define $\boldsymbol{\phi}(x_1, x_2) = [x_1^2, \sqrt{2}x_1x_2, x_2^2] \in \mathbb{R}^3$. So we embed the 2d inputs $\boldsymbol{x}$ into a 3d feature space $\boldsymbol{\phi}(\boldsymbol{x})$.

Now consider the RBF kernel. In this case, the corresponding feature representation is infinite dimensional (see Section 17.2.9.3 for details). However, by working with kernel functions, we can avoid having to deal with infinite dimensional vectors.

17.1.2 Some popular Mercer kernels

In the sections below, we describe some popular Mercer kernels. More details can be found at [Wil14] and https://www.cs.toronto.edu/~duvenaud/cookbook/.

17.1.2.1 Stationary kernels for real-valued vectors

For real-valued inputs, $\mathcal{X} = \mathbb{R}^D$, it is common to use **stationary kernels**, which are functions of the form $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \mathcal{K}(||\boldsymbol{x} - \boldsymbol{x}'||)$; thus the value only depends on the elementwise difference between the inputs. The RBF kernel is a stationary kernel. We give some other examples below.

ARD kernel

We can generalize the RBF kernel by replacing Euclidean distance with Mahalanobis distance, as follows:

$$\mathcal{K}(\boldsymbol{r}) = \sigma^2 \exp\left(-\frac{1}{2}\boldsymbol{r}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\boldsymbol{r}\right) \tag{17.7}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 17.2: Functions sampled from a GP with a Matern kernel. (a) $\nu = 5/2$. (b) $\nu = 1/2$. Generated by gpKernelPlot.ipynb.

where $\boldsymbol{r} = \boldsymbol{x} - \boldsymbol{x}'$. If $\boldsymbol{\Sigma}$ is diagonal, this can be written as

$$\mathcal{K}(\boldsymbol{r}; \boldsymbol{\ell}, \sigma^2) = \sigma^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{D}\frac{1}{\ell_d^2}r_d^2\right) = \prod_{d=1}^{D}\mathcal{K}(r_d; \ell_d, \sigma^{2/D}) \tag{17.8}$$

where

$$\mathcal{K}(r; \ell, \tau^2) = \tau^2 \exp\left(-\frac{1}{2}\frac{1}{\ell^2}r^2\right) \tag{17.9}$$

We can interpret σ^2 as the overall variance, and ℓ_d as defining the **characteristic length scale** of dimension d. If d is an irrelevant input dimension, we can set $\ell_d = \infty$, so the corresponding dimension will be ignored. This is known as **automatic relevancy determination** or **ARD** (Section 11.7.7). Hence the corresponding kernel is called the **ARD kernel**. See Figure 17.1 for an illustration of some 2d functions sampled from a GP using this prior.

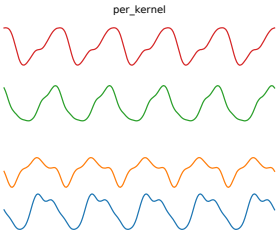
Matern kernels

The SE kernel gives rise to functions that are infinitely differentiable, and therefore are very smooth. For many applications, it is better to use the **Matern kernel**, which gives rise to "rougher" functions, which can better model local "wiggles" without having to make the overall length scale very small.

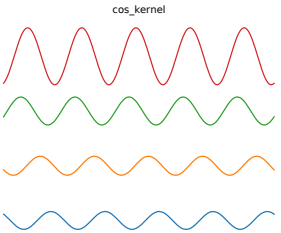
The Matern kernel has the following form:

$$\mathcal{K}(r; \nu, \ell) = \frac{2^{1-\nu}}{\Gamma(\nu)}\left(\frac{\sqrt{2\nu}r}{\ell}\right)^{\nu} K_\nu\left(\frac{\sqrt{2\nu}r}{\ell}\right) \tag{17.10}$$

where K_ν is a modified Bessel function and ℓ is the length scale. Functions sampled from this GP are k-times differentiable iff $\nu > k$. As $\nu \to \infty$, this approaches the SE kernel.



(a) Periodic kernel.



(b) Cosine kernel.

Figure 17.3: Functions sampled from a GP using various stationary periodic kernels. Generated by gpKernelPlot.ipynb.

For values $\nu \in \{\frac{1}{2}, \frac{3}{2}, \frac{5}{2}\}$, the function simplifies as follows:

$$\mathcal{K}(r; \frac{1}{2}, \ell) = \exp(-\frac{r}{\ell}) \tag{17.11}$$

$$\mathcal{K}(r; \frac{3}{2}, \ell) = \left(1 + \frac{\sqrt{3}r}{\ell}\right) \exp\left(-\frac{\sqrt{3}r}{\ell}\right) \tag{17.12}$$

$$\mathcal{K}(r; \frac{5}{2}, \ell) = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right) \tag{17.13}$$

The value $\nu = \frac{1}{2}$ corresponds to the **Ornstein-Uhlenbeck process**, which describes the velocity of a particle undergoing Brownian motion. The corresponding function is continuous but not differentiable, and hence is very "jagged". See Figure 17.2b for an illustration.

Periodic kernels

The **periodic kernel** captures repeating structure, and has the form

$$\mathcal{K}_{\text{per}}(r; \ell, p) = \exp\left(-\frac{2}{\ell^2} \sin^2(\pi \frac{r}{p})\right) \tag{17.14}$$

where p is the period. See Figure 17.3a for an illustration.

A related kernel is the **cosine kernel**:

$$\mathcal{K}(r; p) = \cos\left(2\pi \frac{r}{p}\right) \tag{17.15}$$

See Figure 17.3b for an illustration.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17.1.2.2 Making new kernels from old

Given two valid kernels $\mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}')$ and $\mathcal{K}_2(\boldsymbol{x}, \boldsymbol{x}')$, we can create a new kernel using any of the following methods:

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = c\mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}'), \text{ for any constant } c > 0 \tag{17.16}$$

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = f(\boldsymbol{x})\mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}')f(\boldsymbol{x}'), \text{ for any function } f \tag{17.17}$$

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = q(\mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}')) \text{ for any function polynomial } q \text{ with nonneg. coef.} \tag{17.18}$$

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \exp(\mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}')) \tag{17.19}$$

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}', \text{ for any psd matrix } \mathbf{A} \tag{17.20}$$

For example, suppose we start with the linear kernel $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^\mathsf{T}\boldsymbol{x}'$. We know this is a valid Mercer kernel, since the corresponding Gram matrix is just the (scaled) covariance matrix of the data. From the above rules, we can see that the polynomial kernel $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x}^\mathsf{T}\boldsymbol{x}')^M$ is a valid Mercer kernel. This contains all monomials of order M. For example, if $M = 2$ and the inputs are 2d, we have

$$(\boldsymbol{x}^\mathsf{T}\boldsymbol{x}')^2 = (x_1x_1' + x_2x_2')^2 = (x_1x_1')^2 + (x_2x_2)^2 + 2(x_1x_1')(x_2x_2') \tag{17.21}$$

We can generalize this to contain all terms up to degree M by using the kernel $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x}^\mathsf{T}\boldsymbol{x}' + c)^M$. For example, if $M = 2$ and the inputs are 2d, we have

$$\begin{aligned}(\boldsymbol{x}^\mathsf{T}\boldsymbol{x}' + 1)^2 = &(x_1x_1')^2 + (x_1x_1')(x_2x_2') + (x_1x_1') \\ &+ (x_2x_2)(x_1x_1') + (x_2x_2')^2 + (x_2x_2') \\ &+ (x_1x_1') + (x_2x_2') + 1\end{aligned} \tag{17.22}$$

We can also use the above rules to establish that the Gaussian kernel is a valid kernel. To see this, note that

$$||\boldsymbol{x} - \boldsymbol{x}'||^2 = \boldsymbol{x}^\mathsf{T}\boldsymbol{x} + (\boldsymbol{x}')^\mathsf{T}\boldsymbol{x}' - 2\boldsymbol{x}^\mathsf{T}\boldsymbol{x}' \tag{17.23}$$

and hence

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \exp(-||\boldsymbol{x} - \boldsymbol{x}'||^2/2\sigma^2) = \exp(-\boldsymbol{x}^\mathsf{T}\boldsymbol{x}/2\sigma^2)\exp(\boldsymbol{x}^\mathsf{T}\boldsymbol{x}'/\sigma^2)\exp(-(\boldsymbol{x}')^\mathsf{T}\boldsymbol{x}'/2\sigma^2) \tag{17.24}$$

is a valid kernel.

17.1.2.3 Combining kernels by addition and multiplication

We can also combine kernels using addition or multiplication:

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}') + \mathcal{K}_2(\boldsymbol{x}, \boldsymbol{x}') \tag{17.25}$$

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \mathcal{K}_1(\boldsymbol{x}, \boldsymbol{x}') \times \mathcal{K}_2(\boldsymbol{x}, \boldsymbol{x}') \tag{17.26}$$

Multiplying two positive-definite kernels together always results in another positive definite kernel. This is a way to get a conjunction of the individual properties of each kernel, as illustrated in Figure 17.4.

In addition, adding two positive-definite kernels together always results in another positive definite kernel. This is a way to get a disjunction of the individual properties of each kernel, as illustrated in Figure 17.5.

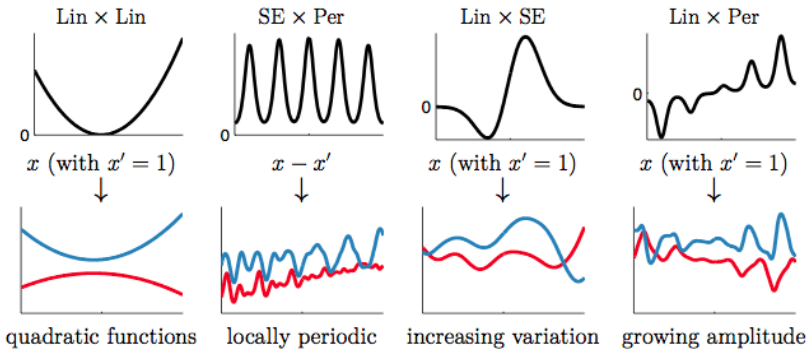


Figure 17.4: Examples of 1d structures obtained by multiplying elementary kernels. Top row shows $\mathcal{K}(x, x' = 1)$. Bottom row shows some functions sampled from $GP(f|0, \mathcal{K})$. From Figure 2.2 of [Duv14]. Used with kind permission of David Duvenaud.

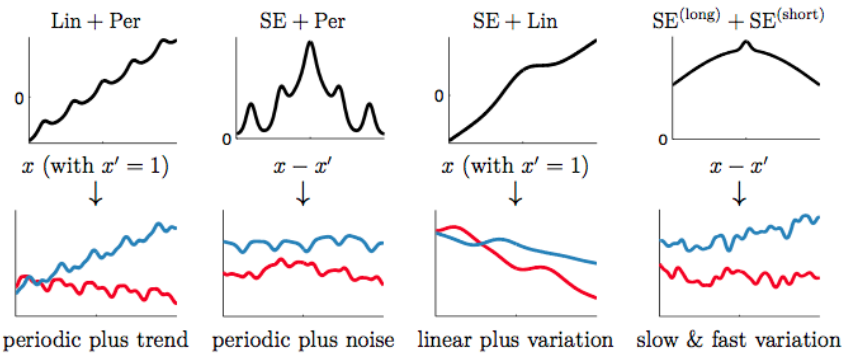


Figure 17.5: Examples of 1d structures obtained by adding elementary kernels. Here $SE^{(short)}$ and $SE^{(long)}$ are two SE kernels with different length scales. From Figure 2.4 of [Duv14]. Used with kind permission of David Duvenaud.

17.1.2.4 Kernels for structured inputs

Kernels are particularly useful when the inputs are structured objects, such as strings and graphs, since it is often hard to "featurize" variable-sized inputs. For example, we can define a **string kernel** which compares strings in terms of the number of n-grams they have in common [Lod+02; BC17].

We can also define kernels on graphs [KJM19]. For example, the **random walk kernel** conceptually performs random walks on two graphs simultaneously, and then counts the number of paths that were produced by both walks. This can be computed efficiently as discussed in [Vis+10]. For more details on graph kernels, see [KJM19].

For a review of kernels on structured objects, see e.g., [Gär03].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

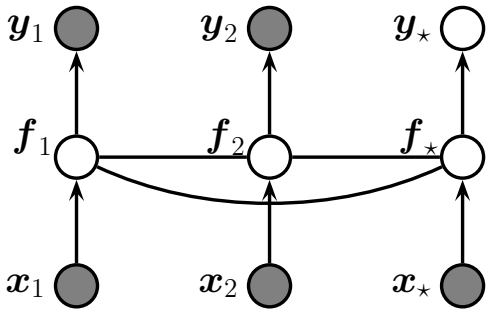


Figure 17.6: A Gaussian process for 2 training points, $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, and 1 testing point, $\boldsymbol{x}_*$, represented as a graphical model representing $p(\boldsymbol{y}, \boldsymbol{f}_X|\mathbf{X}) = \mathcal{N}(\boldsymbol{f}_X|m(\mathbf{X}), \mathcal{K}(\mathbf{X})) \prod_i p(y_i|f_i)$. The hidden nodes $f_i = f(\boldsymbol{x}_i)$ represent the value of the function at each of the data points. These hidden nodes are fully interconnected by undirected edges, forming a Gaussian graphical model; the edge strengths represent the covariance terms $\Sigma_{ij} = \mathcal{K}(\boldsymbol{x}_i, \boldsymbol{x}_j)$. If the test point $\boldsymbol{x}_*$ is similar to the training points $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, then the value of the hidden function f_* will be similar to f_1 and f_2, and hence the predicted output y_* will be similar to the training values y_1 and y_2.

17.2 Gaussian processes

In this section, we discuss **Gaussian processes**, which is a way to define distributions over functions of the form $f : \mathcal{X} \to \mathbb{R}$, where $\mathcal{X}$ is any domain. The key assumption is that the function values at a set of $M > 0$ inputs, $\boldsymbol{f} = [f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_M)]$, is jointly Gaussian, with mean $(\boldsymbol{\mu} = m(\boldsymbol{x}_1), \ldots, m(\boldsymbol{x}_M))$ and covariance $\boldsymbol{\Sigma}_{ij} = \mathcal{K}(\boldsymbol{x}_i, \boldsymbol{x}_j)$, where m is a mean function and $\mathcal{K}$ is a positive definite (Mercer) kernel. Since we assume this holds for any $M > 0$, this includes the case where $M = N + 1$, containing N training points $\boldsymbol{x}_n$ and 1 test point $\boldsymbol{x}_*$. Thus we can infer $f(\boldsymbol{x}_*)$ from knowledge of $f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_n)$ by manipulating the joint Gaussian distribution $p(f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_N), f(\boldsymbol{x}_*))$, as we explain below. We can also extend this to work with the case where we observe noisy functions of $f(\boldsymbol{x}_n)$, such as in regression or classification problems.

17.2.1 Noise-free observations

Suppose we observe a training set $\mathcal{D} = \{(\boldsymbol{x}_n, y_n) : n = 1 : N\}$, where $y_n = f(\boldsymbol{x}_n)$ is the noise-free observation of the function evaluated at $\boldsymbol{x}_n$. If we ask the GP to predict $f(\boldsymbol{x})$ for a value of $\boldsymbol{x}$ that it has already seen, we want the GP to return the answer $f(\boldsymbol{x})$ with no uncertainty. In other words, it should act as an **interpolator** of the training data.

Now we consider the case of predicting the outputs for new inputs that may not be in $\mathcal{D}$. Specifically, given a test set $\mathbf{X}_*$ of size $N_* \times D$, we want to predict the function outputs $\boldsymbol{f}_* = [f(\boldsymbol{x}_{*1}), \ldots, f(\boldsymbol{x}_{*N_*})]$. By definition of the GP, the joint distribution $p(\boldsymbol{f}_X, \boldsymbol{f}_*|\mathbf{X}, \mathbf{X}_*)$ has the following form

$$\begin{pmatrix} \boldsymbol{f}_X \\ \boldsymbol{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}_{X,X} & \mathbf{K}_{X,*} \\ \mathbf{K}_{X,*}^\mathsf{T} & \mathbf{K}_{*,*} \end{pmatrix} \right) \tag{17.27}$$

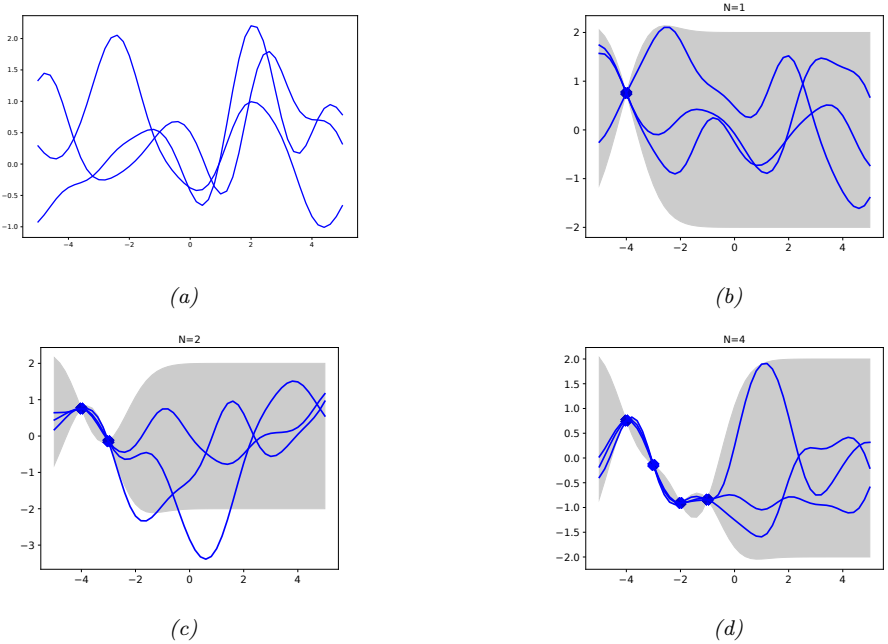


Figure 17.7: (a) some functions sampled from a GP prior with squared exponential kernel. (b-d) : some samples from a GP posterior, after conditioning on 1,2, and 4 noise-free observations. The shaded area represents $\mathbb{E}[f(\boldsymbol{x})] \pm 2\text{std}[f(\boldsymbol{x})]$. *Adapted from Figure 2.2 of [RW06]. Generated by gprDemoNoiseFree.ipynb.*

where $\boldsymbol{\mu}_X = [m(\boldsymbol{x}_1), \ldots, m(\boldsymbol{x}_N)]$, $\boldsymbol{\mu}_* = [m(\boldsymbol{x}_1^*), \ldots, m(\boldsymbol{x}_{N_*}^*)]$, $\mathbf{K}_{X,X} = \mathcal{K}(\mathbf{X}, \mathbf{X})$ is $N \times N$, $\mathbf{K}_{X,*} = \mathcal{K}(\mathbf{X}, \mathbf{X}_*)$ is $N \times N_*$, and $\mathbf{K}_{*,*} = \mathcal{K}(\mathbf{X}_*, \mathbf{X}_*)$ is $N_* \times N_*$. See Figure 17.6 for an illustration. By the standard rules for conditioning Gaussians (Section 3.2.3), the posterior has the following form

$$p(\boldsymbol{f}_*|\mathbf{X}_*, \mathcal{D}) = \mathcal{N}(\boldsymbol{f}_*|\boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \tag{17.28}$$

$$\boldsymbol{\mu}_* = m(\mathbf{X}_*) + \mathbf{K}_{X,*}^{\mathsf{T}} \mathbf{K}_{X,X}^{-1} (\boldsymbol{f}_X - m(\mathbf{X})) \tag{17.29}$$

$$\boldsymbol{\Sigma}_* = \mathbf{K}_{*,*} - \mathbf{K}_{X,*}^{\mathsf{T}} \mathbf{K}_{X,X}^{-1} \mathbf{K}_{X,*} \tag{17.30}$$

This process is illustrated in Figure 17.7. On the left we show some samples from the prior, $p(f)$, where we use an RBF kernel (Section 17.1) and a zero mean function. On the right, we show samples from the posterior, $p(f|\mathcal{D})$. We see that the model perfectly interpolates the training data, and that the predictive uncertainty increases as we move further away from the observed data.

17.2.2 Noisy observations

Now let us consider the case where what we observe is a noisy version of the underlying function, $y_n = f(\boldsymbol{x}_n) + \epsilon_n$, where $\epsilon_n \sim \mathcal{N}(0, \sigma_y^2)$. In this case, the model is not required to interpolate the data,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

but it must come "close" to the observed data. The covariance of the observed noisy responses is

$$\text{Cov}\left[y_i, y_j\right] = \text{Cov}\left[f_i, f_j\right] + \text{Cov}\left[\epsilon_i, \epsilon_j\right] = \mathcal{K}(\boldsymbol{x}_i, \boldsymbol{x}_j) + \sigma_y^2 \delta_{ij} \tag{17.31}$$

where $\delta_{ij} = \mathbb{I}\left(i = j\right)$. In other words

$$\text{Cov}\left[\boldsymbol{y}|\mathbf{X}\right] = \mathbf{K}_{X,X} + \sigma_y^2 \mathbf{I}_N \triangleq \mathbf{K}_\sigma \tag{17.32}$$

The joint density of the observed data and the latent, noise-free function on the test points is given by

$$\begin{pmatrix} \boldsymbol{y} \\ \boldsymbol{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}_\sigma & \mathbf{K}_{X,*} \\ \mathbf{K}_{X,*}^\mathsf{T} & \mathbf{K}_{*,*} \end{pmatrix} \right) \tag{17.33}$$

Hence the posterior predictive density at a set of test points $\mathbf{X}_*$ is

$$p(\boldsymbol{f}_*|\mathcal{D}, \mathbf{X}_*) = \mathcal{N}(\boldsymbol{f}_*|\boldsymbol{\mu}_{*|X}, \boldsymbol{\Sigma}_{*|X}) \tag{17.34}$$

$$\boldsymbol{\mu}_{*|X} = \boldsymbol{\mu}_* + \mathbf{K}_{X,*}^\mathsf{T} \mathbf{K}_\sigma^{-1} (\boldsymbol{y} - \boldsymbol{\mu}_X) \tag{17.35}$$

$$\boldsymbol{\Sigma}_{*|X} = \mathbf{K}_{*,*} - \mathbf{K}_{X,*}^\mathsf{T} \mathbf{K}_\sigma^{-1} \mathbf{K}_{X,*} \tag{17.36}$$

In the case of a single test input, this simplifies as follows

$$p(f_*|\mathcal{D}, \boldsymbol{x}_*) = \mathcal{N}(f_*|m_* + \boldsymbol{k}_*^\mathsf{T} \mathbf{K}_\sigma^{-1} (\boldsymbol{y} - \boldsymbol{\mu}_X),\ k_{**} - \boldsymbol{k}_*^\mathsf{T} \mathbf{K}_\sigma^{-1} \boldsymbol{k}_*) \tag{17.37}$$

where $\boldsymbol{k}_* = [\mathcal{K}(\boldsymbol{x}_*, \boldsymbol{x}_1), \ldots, \mathcal{K}(\boldsymbol{x}_*, \boldsymbol{x}_N)]$ and $k_{**} = \mathcal{K}(\boldsymbol{x}_*, \boldsymbol{x}_*)$. If the mean function is zero, we can write the posterior mean as follows:

$$\mu_{*|X} = \boldsymbol{k}_*^\mathsf{T} (\mathbf{K}_\sigma^{-1} \boldsymbol{y}) \triangleq \boldsymbol{k}_*^\mathsf{T} \boldsymbol{\alpha} = \sum_{n=1}^N \mathcal{K}(\boldsymbol{x}_*, \boldsymbol{x}_n) \alpha_n \tag{17.38}$$

This is identical to the predictions from kernel ridge regression in Equation (17.108).

17.2.3 Comparison to kernel regression

In Section 16.3.5, we discussed kernel regression, which is a generative approach to regression in which we approximate $p(y, \boldsymbol{x})$ using kernel density estimation. In particular, Equation (16.39) gives us

$$\mathbb{E}\left[y|\boldsymbol{x}, \mathcal{D}\right] = \frac{\sum_{n=1}^N \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n) y_n}{\sum_{n'=1}^N \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_{n'})} = \sum_{n=1}^N y_n w_n(\boldsymbol{x}) \tag{17.39}$$

$$w_n(\boldsymbol{x}) \triangleq \frac{\mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_n)}{\sum_{n'=1}^N \mathcal{K}_h(\boldsymbol{x} - \boldsymbol{x}_{n'})} \tag{17.40}$$

This is very similar to Equation (17.38). However, there are a few important differences. Firstly, in a GP, we use a positive definite (Mercer) kernel instead of a density kernel; Mercer kernels can be defined on structured objects, such as strings and graphs, which is harder to do for density kernels.

Second, a GP is an interpolator (at least when $\sigma^2 = 0$), so $\mathbb{E}[y|\boldsymbol{x}_n, \mathcal{D}] = y_n$. By contrast, kernel regression is not an interpolator (although it can be made into one by iteratively fitting the residuals, as in [KJ16]). Third, a GP is a Bayesian method, which means we can estimate hyperparameters (of the kernel) by maximizing the marginal likelihood; by contrast, in kernel regression we must use cross-validation to estimate the kernel parameters, such as the bandwidth. Fourth, computing the weights w_n for kernel regression takes $O(N)$ time, where $N = |\mathcal{D}|$, whereas computing the weights α_n for GP regression takes $O(N^3)$ time (although there are approximation methods that can reduce this to $O(NM^2)$, as we discuss in Section 17.2.9).

17.2.4 Weight space vs function space

In this section, we show how Bayesian linear regression is a special case of a GP.

Consider the linear regression model $y = f(\boldsymbol{x}) + \epsilon$, where $f(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x})$ and $\epsilon \sim \mathcal{N}(0, \sigma_y^2)$. If we use a Gaussian prior $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\mathbf{0}, \boldsymbol{\Sigma}_w)$, then the posterior is as follows (see Section 11.7.2 for the derivation):

$$p(\boldsymbol{w}|\mathcal{D}) = \mathcal{N}(\boldsymbol{w}|\frac{1}{\sigma_y^2}\mathbf{A}^{-1}\boldsymbol{\Phi}^T\boldsymbol{y}, \mathbf{A}^{-1}) \tag{17.41}$$

where $\boldsymbol{\Phi}$ is the $N \times D$ design matrix, and

$$\mathbf{A} = \sigma_y^{-2}\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi} + \boldsymbol{\Sigma}_w^{-1} \tag{17.42}$$

The posterior predictive distribution for $f_* = f(\boldsymbol{x}_*)$ is therefore

$$p(f_*|\mathcal{D}, \boldsymbol{x}_*) = \mathcal{N}(f_*|\frac{1}{\sigma_y^2}\boldsymbol{\phi}_*^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{y},\ \boldsymbol{\phi}_*^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{\phi}_*) \tag{17.43}$$

where $\boldsymbol{\phi}_* = \boldsymbol{\phi}(\boldsymbol{x}_*)$. This views the problem of inference and prediction in **weight space**.

We now show that this is equivalent to the predictions made by a GP using a kernel of the form $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{\phi}(\boldsymbol{x})^\mathsf{T}\boldsymbol{\Sigma}_w\boldsymbol{\phi}(\boldsymbol{x}')$. To see this, let $\mathbf{K} = \boldsymbol{\Phi}\boldsymbol{\Sigma}_w\boldsymbol{\Phi}^\mathsf{T}$, $\boldsymbol{k}_* = \boldsymbol{\Phi}\boldsymbol{\Sigma}_w\boldsymbol{\phi}_*$, and $k_{**} = \boldsymbol{\phi}_*^\mathsf{T}\boldsymbol{\Sigma}_w\boldsymbol{\phi}_*$. Using this notation, and the matrix inversion lemma, we can rewrite Equation (17.43) as follows

$$p(f_*|\mathcal{D}, \boldsymbol{x}_*) = \mathcal{N}(f_*|\boldsymbol{\mu}_{*|X}, \boldsymbol{\Sigma}_{*|X}) \tag{17.44}$$

$$\boldsymbol{\mu}_{*|X} = \boldsymbol{\phi}_*^\mathsf{T}\boldsymbol{\Sigma}_w\boldsymbol{\Phi}^\mathsf{T}(\mathbf{K} + \sigma_y^2\mathbf{I})^{-1}\boldsymbol{y} = \boldsymbol{k}_*^\mathsf{T}\mathbf{K}_\sigma^{-1}\boldsymbol{y} \tag{17.45}$$

$$\boldsymbol{\Sigma}_{*|X} = \boldsymbol{\phi}_*^\mathsf{T}\boldsymbol{\Sigma}_w\boldsymbol{\phi}_* - \boldsymbol{\phi}_*^\mathsf{T}\boldsymbol{\Sigma}_w\boldsymbol{\Phi}^\mathsf{T}(\mathbf{K} + \sigma_y^2\mathbf{I})^{-1}\boldsymbol{\Phi}\boldsymbol{\Sigma}_w\boldsymbol{\phi}_* = k_{**} - \boldsymbol{k}_*^\mathsf{T}\mathbf{K}_\sigma^{-1}\boldsymbol{k}_* \tag{17.46}$$

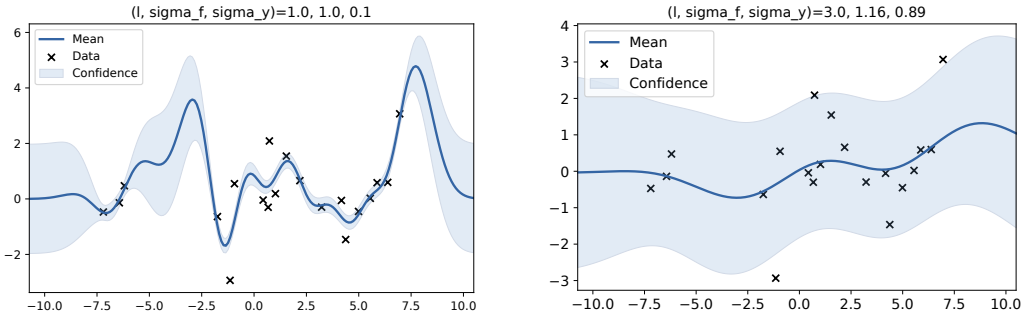
which matches the results in Equation (17.37), assuming $m(\boldsymbol{x}) = 0$. (Non-zero mean can be captured by adding a constant feature with value 1 to $\boldsymbol{\phi}(\boldsymbol{x})$.)

Thus we can derive a GP from Bayesian linear regression. Note, however, that linear regression assumes $\boldsymbol{\phi}(\boldsymbol{x})$ is a finite length vector, whereas a GP allows us to work directly in terms of kernels, which may correspond to infinite length feature vectors (see Section 17.1.1). That is, a GP works in **function space**.

17.2.5 Numerical issues

In this section, we discuss computational and numerical issues which arise when implementing the above equations. For notational simplicity, we assume the prior mean is zero, $m(\boldsymbol{x}) = 0$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 17.8: Some 1d GPs with SE kernels but different hyper-parameters fit to 20 noisy observations. The hyper-parameters $(\ell, \sigma_f, \sigma_y)$ are as follows: (a) (1,1,0.1) (b) (3.0, 1.16, 0.89). Adapted from Figure 2.5 of [RW06]. Generated by gprDemoChangeHparams.ipynb.

The posterior predictive mean is given by $\mu_* = \boldsymbol{k}_*^\mathsf{T} \mathbf{K}_\sigma^{-1} \boldsymbol{y}$. For reasons of numerical stability, it is unwise to directly invert $\mathbf{K}_\sigma$. A more robust alternative is to compute a Cholesky decomposition, $\mathbf{K}_\sigma = \mathbf{L}\mathbf{L}^\mathsf{T}$, which takes $O(N^3)$ time. Then we compute $\boldsymbol{\alpha} = \mathbf{L}^\mathsf{T} \setminus (\mathbf{L} \setminus \boldsymbol{y})$, where we have used the backslash operator to represent backsubstitution (Section 7.7.1). Given this, we can compute the posterior mean for each test case in $O(N)$ time using

$$\mu_* = \boldsymbol{k}_*^\mathsf{T} \mathbf{K}_\sigma^{-1} \boldsymbol{y} = \boldsymbol{k}_*^\mathsf{T} \mathbf{L}^{-\mathsf{T}} (\mathbf{L}^{-1} \boldsymbol{y}) = \boldsymbol{k}_*^\mathsf{T} \boldsymbol{\alpha} \tag{17.47}$$

We can compute the variance in $O(N^2)$ time for each test case using

$$\sigma_*^2 = k_{**} - \boldsymbol{k}_*^\mathsf{T} \mathbf{L}^{-T} \mathbf{L}^{-1} \boldsymbol{k}_* = k_{**} - \boldsymbol{v}^\mathsf{T} \boldsymbol{v} \tag{17.48}$$

where $\boldsymbol{v} = \mathbf{L} \setminus \boldsymbol{k}_*$.

Finally, the log marginal likelihood (needed for kernel learning, Section 17.2.6) can be computed using

$$\log p(\boldsymbol{y}|\mathbf{X}) = -\frac{1}{2} \boldsymbol{y}^\mathsf{T} \boldsymbol{\alpha} - \sum_{n=1}^{N} \log L_{nn} - \frac{N}{2} \log(2\pi) \tag{17.49}$$

17.2.6 Estimating the kernel

Most kernels have some free parameters, which can have a large effect on the predictions from the model. For example, suppose we are performing 1d regression using a GP with an RBF kernel of the form

$$\mathcal{K}(x_p, x_q) = \sigma_f^2 \exp(-\frac{1}{2\ell^2}(x_p - x_q)^2) \tag{17.50}$$

Here ℓ is the horizontal scale over which the function changes, σ_f^2 controls the vertical scale of the function. We assume observation noise with variance σ_y^2.

We sampled 20 observations from an MVN with a covariance given by $\mathbf{\Sigma} = \mathcal{K}(x_i, x_j)$ for a grid of points $\{x_i\}$, and added observation noise of value σ_y. We then fit this data using a GP with the same kernel, but with a range of hyperparmameters. Figure 17.8 illustrates the effects of changing these parameters. In Figure 17.8(a), we use $(\ell, \sigma_f, \sigma_y) = (1, 1, 0.1)$, and the result is a good fit. In Figure 17.8(b), we increase the length scale to $\ell = 3$; now the function looks overly smooth.

17.2.6.1 Empirical Bayes

To estimate the kernel parameters $\boldsymbol{\theta}$ (sometimes called hyperparameters), we could use exhaustive search over a discrete grid of values, with validation loss as an objective, but this can be quite slow. (This is the approach used by nonprobabilistic methods, such as SVMs (Section 17.3) to tune kernels.) Here we consider an empirical Bayes approach (Section 4.6.5.3), which will allow us to use gradient-based optimization methods, which are much faster. In particular, we will maximize the marginal likelihood

$$p(\boldsymbol{y}|\mathbf{X}, \boldsymbol{\theta}) = \int p(\boldsymbol{y}|\boldsymbol{f}, \mathbf{X}) p(\boldsymbol{f}|\mathbf{X}, \boldsymbol{\theta}) d\boldsymbol{f} \tag{17.51}$$

(The reason it is called the marginal likelihood, rather than just likelihood, is because we have marginalized out the latent Gaussian vector $\boldsymbol{f}$.)

For notational simplicity, we assume the mean function is 0. Since $p(\boldsymbol{f}|\mathbf{X}) = \mathcal{N}(\boldsymbol{f}|\mathbf{0}, \mathbf{K})$, and $p(\boldsymbol{y}|\boldsymbol{f}) = \prod_{n=1}^{N} \mathcal{N}(y_n|f_n, \sigma_y^2)$, the marginal likelihood is given by

$$\log p(\boldsymbol{y}|\mathbf{X}, \boldsymbol{\theta}) = \log \mathcal{N}(\boldsymbol{y}|\mathbf{0}, \mathbf{K}_\sigma) = -\frac{1}{2}\boldsymbol{y}^\mathsf{T}\mathbf{K}_\sigma^{-1}\boldsymbol{y} - \frac{1}{2}\log|\mathbf{K}_\sigma| - \frac{N}{2}\log(2\pi) \tag{17.52}$$

where the dependence of $\mathbf{K}_\sigma = \mathbf{K}_{X,X} + \sigma_y^2\mathbf{I}_N$ on $\boldsymbol{\theta}$ is implicit. The first term is a data fit term, the second term is a model complexity term, and the third term is just a constant. To understand the tradeoff between the first two terms, consider a SE kernel in 1D, as we vary the length scale ℓ and hold σ_y^2 fixed. For short length scales, the fit will be good, so $\boldsymbol{y}^\mathsf{T}\mathbf{K}_\sigma^{-1}\boldsymbol{y}$ will be small. However, the model complexity will be high: $\mathbf{K}$ will be almost diagonal, (as in Figure 13.22, top right), since most points will not be considered "near" any others, so the $\log|\mathbf{K}_\sigma|$ term will be large. For long length scales, the fit will be poor but the model complexity will be low: $\mathbf{K}$ will be almost all 1's, (as in Figure 13.22, bottom right), so $\log|\mathbf{K}_\sigma|$ will be small.

We now discuss how to maximize the marginal likelihood. One can show that

$$\frac{\partial}{\partial\theta_j}\log p(\boldsymbol{y}|\mathbf{X}, \boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{y}^\mathsf{T}\mathbf{K}_\sigma^{-1}\frac{\partial\mathbf{K}_\sigma}{\partial\theta_j}\mathbf{K}_\sigma^{-1}\boldsymbol{y} - \frac{1}{2}\mathrm{tr}(\mathbf{K}_\sigma^{-1}\frac{\partial\mathbf{K}_\sigma}{\partial\theta_j}) \tag{17.53}$$

$$= \frac{1}{2}\mathrm{tr}\left((\boldsymbol{\alpha}\boldsymbol{\alpha}^\mathsf{T} - \mathbf{K}_\sigma^{-1})\frac{\partial\mathbf{K}_\sigma}{\partial\theta_j}\right) \tag{17.54}$$

where $\boldsymbol{\alpha} = \mathbf{K}_\sigma^{-1}\boldsymbol{y}$. It takes $O(N^3)$ time to compute $\mathbf{K}_\sigma^{-1}$, and then $O(N^2)$ time per hyper-parameter to compute the gradient.

The form of $\frac{\partial\mathbf{K}_\sigma}{\partial\theta_j}$ depends on the form of the kernel, and which parameter we are taking derivatives with respect to. Often we have constraints on the hyper-parameters, such as $\sigma_y^2 \geq 0$. In this case, we can define $\theta = \log(\sigma_y^2)$, and then use the chain rule.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

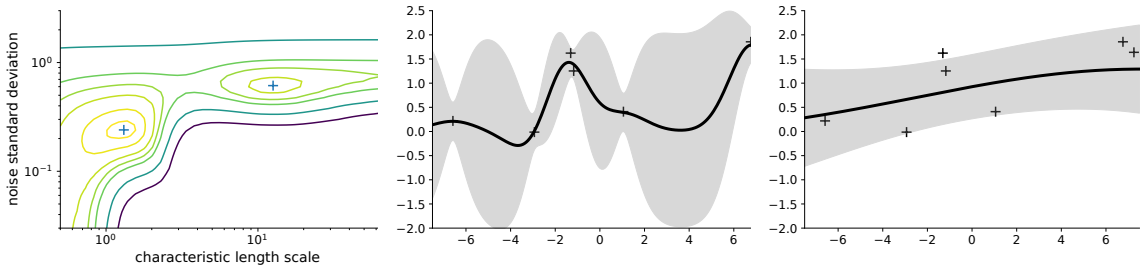


Figure 17.9: Illustration of local minima in the marginal likelihood surface. (a) We plot the log marginal likelihood vs kernel length scale ℓ and observation noise σ_y, for fixed signal level $\sigma_f = 1$, using the 7 data points shown in panels b and c. (b) The function corresponding to the lower left local minimum, $(\ell, \sigma_y) \approx (1, 0.2)$. This is quite "wiggly" and has low noise. (c) The function corresponding to the top right local minimum, $(\ell, \sigma_y) \approx (10, 0.8)$. This is quite smooth and has high noise. The data was generated using $(\ell, \sigma_f, \sigma_y) = (1, 1, 0.1)$. Adapted from Figure 5.5 of [RW06]. Generated by gpr_demo_marglik.ipynb.

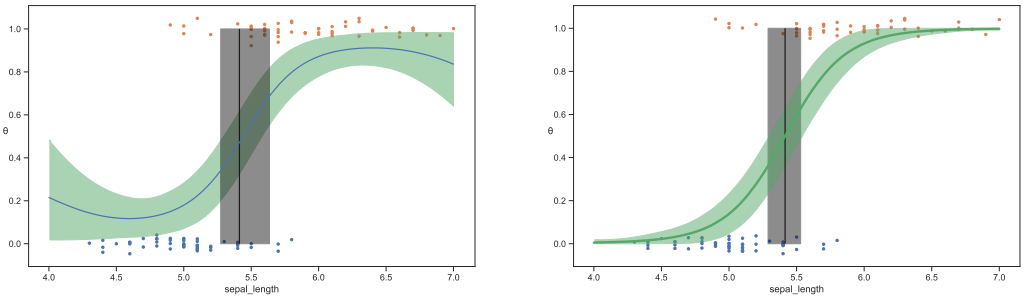
Given an expression for the log marginal likelihood and its derivative, we can estimate the kernel parameters using any standard gradient-based optimizer. However, since the objective is not convex, local minima can be a problem, as we illustrate below, so we may need to use multiple restarts.

As an example, consider the RBF in Equation (17.50) with $\sigma_f^2 = 1$. In Figure 17.9(a), we plot $\log p(\boldsymbol{y}|\mathbf{X}, \ell, \sigma_y^2)$ (where $\mathbf{X}$ and $\boldsymbol{y}$ are the 7 data points shown in panels b and c) as we vary ℓ and σ_y^2. The two local optima are indicated by $+$. The bottom left optimum corresponds to a low-noise, short-length scale solution (shown in panel b). The top right optimum corresponds to a high-noise, long-length scale solution (shown in panel c). With only 7 data points, there is not enough evidence to confidently decide which is more reasonable, although the more complex model (panel b) has a marginal likelihood that is about 60% higher than the simpler model (panel c). With more data, the more complex model would become even more preferred.

Figure 17.9 illustrates some other interesting (and typical) features. The region where $\sigma_y^2 \approx 1$ (top of panel a) corresponds to the case where the noise is very high; in this regime, the marginal likelihood is insensitive to the length scale (indicated by the horizontal contours), since all the data is explained as noise. The region where $\ell \approx 0.5$ (left hand side of panel a) corresponds to the case where the length scale is very short; in this regime, the marginal likelihood is insensitive to the noise level (indicated by the vertical contours), since the data is perfectly interpolated. Neither of these regions would be chosen by a good optimizer.

17.2.6.2 Bayesian inference

When we have a small number of datapoints (e.g., when using GPs for Bayesian optimization), using a point estimate of the kernel parameters can give poor results [Bul11; WF14]. In such cases, we may wish to approximate the posterior over the kernel parameters. Several methods can be used. For example, [MA10] shows how to use slice sampling, [Hen+15] shows how to use Hamiltonian Monte Carlo, and [BBV11] shows how to use sequential Monte Carlo.



(a) (b)

Figure 17.10: GP classifier for a binary classification problem on Iris flowers (Setosa vs Versicolor) using a single input feature (sepal length). The fat vertical line is the credible interval for the decision boundary. (a) SE kernel. (b) SE plus linear kernel. Adapted from Figures 7.11–7.12 of [Mar18]. Generated by gp_classify_iris_1d_pymc3.ipynb.

17.2.7 GPs for classification

So far, we have focused on GPs for regression using Gaussian likelihoods. In this case, the posterior is also a GP, and all computation can be performed analytically. However, if the likelihood is non-Gaussian, such as the Bernoulli likelihood for binary classification, we can no longer compute the posterior exactly.

There are various approximations we can make, some of which we discuss in the sequel to this book, [Mur23]. In this section, we use the Hamiltonian Monte Carlo method (Section 4.6.8.4), both for the latent Gaussian function $\boldsymbol{f}$ as well as the kernel hyperparameters $\boldsymbol{\theta}$. The basic idea is to specify the negative log joint

$$-\mathcal{E}(\boldsymbol{f}, \boldsymbol{\theta}) = \log p(\boldsymbol{f}, \boldsymbol{\theta}|\mathbf{X}, \boldsymbol{y}) = \log \mathcal{N}(\boldsymbol{f}|\mathbf{0}, \mathbf{K}(\mathbf{X}, \mathbf{X})) + \sum_{n=1}^{N} \log \text{Ber}(y_n|f_n(\boldsymbol{x}_n)) + \log p(\boldsymbol{\theta}) \quad (17.55)$$

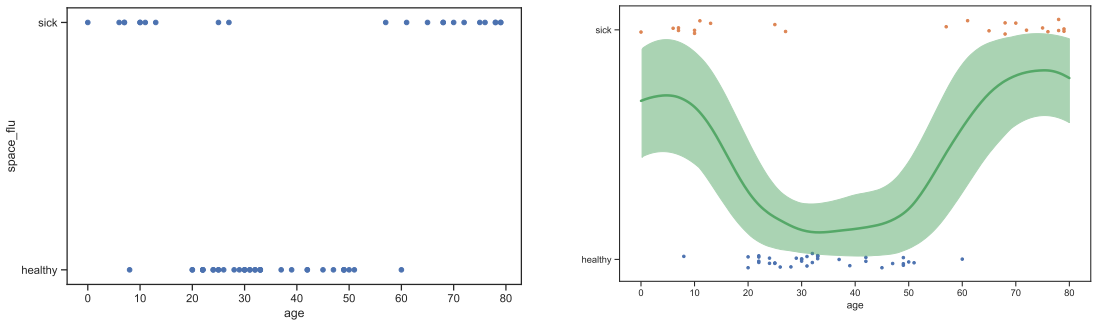
We then use autograd to compute $\nabla_{\boldsymbol{f}} \mathcal{E}(\boldsymbol{f}, \boldsymbol{\theta})$ and $\nabla_{\boldsymbol{\theta}} \mathcal{E}(\boldsymbol{f}, \boldsymbol{\theta})$, and use these gradients as inputs to a Gaussian proposal distribution.

Let us consider a 1d example from [Mar18]. This is similar to the Bayesian logistic regression example from Figure 4.20, where the goal is to classify iris flowers as being Setosa or Versicolor, $y_n \in \{0, 1\}$, given information about the sepal length, x_n. We will use an SE kernel with length scale ℓ. We put a $\text{Ga}(2, 0.5)$ prior on ℓ.

Figure 17.10a shows the results using the SE kernel. This is similar to the results of linear logistic regression (see Figure 4.20), except that at the edges (away from the data), the probability curves towards 0.5. This is because the prior mean function is $m(x) = 0$, and $\sigma(0) = 0.5$. We can eliminate this artefact by using a more flexible kernel, which encodes the prior knowledge that we expect the output to be monotonically increasing or decreasing in the input. We can do this using a **linear kernel**,

$$\mathcal{K}(x, x') = (x - c)(x' - c) \quad (17.56)$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 17.11: (a) Fictitious "space flu" binary classification problem. (b) Fit from a GP with SE kernel. Adapted from Figures 7.13–7.14 of [Mar18]. Generated by gp_classify_spaceflu_1d_pymc3.ipynb.

We can scale and add this to the SE kernel to get

$$\mathcal{K}(x, x') = \tau(x - c)(x' - c) + \exp\left[-\frac{(x - x')^2}{2\ell^2}\right] \tag{17.57}$$

The results are shown in Figure 17.10b, and look more reasonable.

One might wonder why we bothered to use a GP, when the results are no better than a simple linear logistic regression model. The reason is that the GP is much more flexible, and makes fewer a priori assumptions, beyond smoothness. For example, suppose the data looked like Figure 17.11a. In this case, a linear logistic regression model could not fit the data. We could in principle use a neural network, but it may not work well since we only have 60 data points. However, GPs are well designed to handle the small sample setting. In Figure 17.11b, we show the results of fitting a GP with an SE kernel to this data. The results look reasonable.

17.2.8 Connections with deep learning

It turns out that there are many interesting connections and similarities between GPs and deep neural networks. For example, one can show that a neural network with a single, **infinitely wide** layer of RBF units is equivalent to a GP with an RBF kernel. (This follows from the fact that the RBF kernel can be expressed as the inner product of an infinite number of features.) In fact, many kinds of DNNs (in the infinite limit) can be converted to an equivalent GP using a specific kind of kernel known as the **neural tangent kernel** [JGH18]. See the sequel to this book, [Mur23], for details.

17.2.9 Scaling GPs to large datasets

The main disadvantage of GPs (and other kernel methods, such as SVMs, which we discuss in Section 17.3) is that inverting the $N \times N$ kernel matrix takes $O(N^3)$ time, making the method too

slow for big datasets. Many different approximate schemes have been proposed to speedup GPs (see e.g., [Liu+18a] for a review). In this section, we briefly mention some of them. For more details, see the sequel to this book, [Mur23].

17.2.9.1 Sparse (inducing-point) approximations

A simple approach to speeding up GP inference is to use less data. A better approach is to try to "summarize" the N training points $\mathbf{X}$ into $M \ll N$ **inducing points** or **pseudo inputs** $\mathbf{Z}$. This lets us replace $p(\boldsymbol{f}|\boldsymbol{f}_X)$ with $p(\boldsymbol{f}|\boldsymbol{f}_Z)$, where $\boldsymbol{f}_X = \{f(\boldsymbol{x}) : \boldsymbol{x} \in \mathbf{Z}\}$ is the vector of observed function values at the training points, and $\boldsymbol{f}_Z = \{f(\boldsymbol{x}) : \boldsymbol{x} \in \mathbf{Z}\}$ is the vector of estimated function values at the inducing points. By optimizing $(\mathbf{Z}, \boldsymbol{f}_Z)$ we can learn to "compress" the training data $(\mathbf{X}, \boldsymbol{f}_X)$ into a "bottleneck" $(\mathbf{Z}, \boldsymbol{f}_Z)$, thus speeding up computation from $O(N^3)$ to $O(M^3)$. This is called a **sparse GP**. This whole process can be made rigorous using the framework of variational inference. For details, see the sequel to this book, [Mur23].

17.2.9.2 Exploiting parallelization and kernel matrix structure

It takes $O(N^3)$ time to compute the Cholesky decomposition of $\mathbf{K}_{X,X}$, which is needed to solve the linear system $\mathbf{K}_\sigma \boldsymbol{\alpha} = \boldsymbol{y}$ and to compute $|\mathbf{K}_{X,X}|$, where $\mathbf{K}_\sigma = \mathbf{K}_{X,X} + \sigma^2 \mathbf{I}_N$. An alternative to Cholesky decomposition is to use linear algebra methods, often called **Krylov subspace methods**, which are based just on **matrix vector multiplication** or **MVM**. These approaches are often much faster, since they can naturally exploit structure in the kernel matrix. Moreover, even if the kernel matrix does not have special structure, matrix multiplies are trivial to parallelize, and can thus be greatly accelerated by GPUs, unlike Cholesky based methods which are largely sequential. This is the basis of the popular **GPyTorch** package [Gar+18]. For more details, see the sequel to this book, [Mur23].

17.2.9.3 Random feature approximation

Although the power of kernels resides in the ability to avoid working with featurized representations of the inputs, such kernelized methods take $O(N^3)$ time, in order to invert the Gram matrix $\mathbf{K}$. This can make it difficult to use such methods on large scale data. Fortunately, we can approximate the feature map for many (shift invariant) kernels using a randomly chosen finite set of M basis functions, thus reducing the cost to $O(NM + M^3)$. We briefly discuss this idea below. For more details, see e.g., [Liu+20].

Random features for RBF kernel

We will focus on the case of the Gaussian RBF kernel. One can show that

$$\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}') \approx \boldsymbol{\phi}(\boldsymbol{x})^\mathsf{T} \boldsymbol{\phi}(\boldsymbol{x}') \tag{17.58}$$

where the (real-valued) feature vector is given by

$$\boldsymbol{\phi}(\boldsymbol{x}) \triangleq \frac{1}{\sqrt{T}}[(\sin(\boldsymbol{\omega}_1^\mathsf{T}\boldsymbol{x}), ..., \sin(\boldsymbol{\omega}_T^\mathsf{T}\boldsymbol{x}), \cos(\boldsymbol{\omega}_1^\mathsf{T}\boldsymbol{x}), ..., \cos(\boldsymbol{\omega}_T^\mathsf{T}\boldsymbol{x}))] \tag{17.59}$$

$$= \frac{1}{\sqrt{T}}[\sin(\boldsymbol{\Omega}\boldsymbol{x}), \cos(\boldsymbol{\Omega}\boldsymbol{x})] \tag{17.60}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $T = M/2$, and $\boldsymbol{\Omega} \in \mathbb{R}^{T \times D}$ is a random Gaussian matrix, where the entries are sampled iid from $\mathcal{N}(0, 1/\sigma^2)$, where σ is the kernel bandwidth. The bias of the approximation decreases as we increase M. In practice, we use a finite M, and compute a single sample Monte Carlo approximation to the expectation by drawing a single random matrix. The features in Equation (17.60) are called **random Fourier features** (**RFF**) [RR08] or "weighted sums of random kitchen sinks" [RR09].

We can also use positive random features, rather than trigonometric random features, which can be preferable in some applications, such as models which use attention (see Section 15.6.4). In particular, we can use

$$\boldsymbol{\phi}(\boldsymbol{x}) \triangleq e^{-||\boldsymbol{x}||^2/2} \frac{1}{\sqrt{M}} \left[(\exp(\boldsymbol{\omega}_1^\mathsf{T}\boldsymbol{x}), \cdots, (\exp(\boldsymbol{\omega}_M^\mathsf{T}\boldsymbol{x})\right] \tag{17.61}$$

where $\boldsymbol{\omega}_m$ are sampled as before. For details, see [Cho+20b].

Regardless of whether we use trigonometric or positive features, we can obtain a lower variance estimate by ensuring that the rows of $\boldsymbol{\Omega}$ are random but orthogonal; these are called **orthogonal random features**. Such sampling can be conducted efficiently via Gram-Schmidt orthogonalization of the unstructured Gaussian matrices [Yu+16], or several approximations that are even faster (see [CRW17; Cho+19]).

Fastfood approximation

Unfortunately, storing the random matrix $\boldsymbol{\Omega}$ takes $O(DM)$ space, and computing $\boldsymbol{\Omega}\boldsymbol{x}$ takes $O(DM)$ time, where D is the input dimensionality, and M is the number of random features. This can be prohibitive if $M \gg D$, which it may need to be in order to get any benefits over using the original set of features. Fortunately, we can use the **fast Hadamard transform** to reduce the memory from $O(MD)$ to $O(M)$, and reduce the time from $O(MD)$ to $O(M \log D)$. This approach has been called **fastfood** [LSS13], a reference to the original term "kitchen sinks".

Extreme learning machines

We can use the random features approximation to the kernel to convert a GP into a linear model of the form

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbf{W}\boldsymbol{\phi}(\boldsymbol{x}) = \mathbf{W}h(\boldsymbol{\Omega}\boldsymbol{x}) \tag{17.62}$$

where $h(a) = \sqrt{1/M}[\sin(a), \cos(a)]$ for RBF kernels. This is equivalent to a one-layer MLP with random (and fixed) input-to-hidden weights. When $M > N$, this corresponds to an over-parameterized model, which can perfectly interpolate the training data.

In [Cur+17], they apply this method to fit a logistic regression model of the form $f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbf{W}^\mathsf{T}h(\hat{\boldsymbol{\Omega}}\boldsymbol{x}) + \boldsymbol{b}$ using SGD; they call the resulting method "**McKernel**". We can also optimize $\boldsymbol{\Omega}$ as well as $\mathbf{W}$, as discussed in [Alb+17], although now the problem is no longer convex.

Alternatively, we can use $M < N$, but stack many such random nonlinear layers together, and just optimize the output weights. This has been called an **extreme learning machine** or **ELM** (see e.g., [Hua14]), although this work is controversial.[1]

1. The controversy has arisen because the inventor Guang-Bin Huang has been accused of not citing related prior work, such as the equivalent approach based on random feature approximations to kernels. For details, see https://en.wikipedia.org/wiki/Extreme_learning_machine#Controversy.

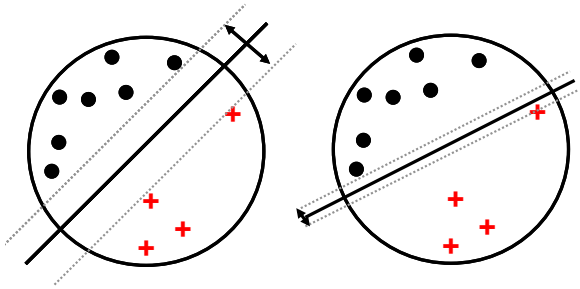


Figure 17.12: Illustration of the large margin principle. Left: a separating hyper-plane with large margin. Right: a separating hyper-plane with small margin.

17.3 Support vector machines (SVMs)

In this section, we discuss a form of (non-probabilistic) predictors for classification and regression problems which have the form

$$f(\boldsymbol{x}) = \sum_{i=1}^{N} \alpha_i \mathcal{K}(\boldsymbol{x}, \boldsymbol{x}_i) \tag{17.63}$$

By adding suitable constraints, we can ensure that many of the α_i coefficients are 0, so that predictions at test time only depend on a subset of the training points, known as "**support vectors**". Hence the resulting model is called a **support vector machine** or **SVM**. We give a brief summary below. More details, can be found in e.g., [VGS97; SS01].

17.3.1 Large margin classifiers

Consider a binary classifier of the form $h(\boldsymbol{x}) = \text{sign}(f(\boldsymbol{x}))$, where the decision boundary is given by the following linear function:

$$f(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + w_0 \tag{17.64}$$

(In the SVM literature, it is common to assume the class labels are -1 and $+1$, rather than 0 and 1. To avoid confusion, we denote such target labels by $\tilde{y}$ rather than y.) There may be many lines that separate the data. However, intuitively we would like to pick the one that has maximum **margin**, which is the distance of the closest point to the decision boundary, since this will give us the most robust solution. This idea is illustrated in Figure 17.12: the solution on the left has larger margin than the one on the right, so it will be less sensitive to perturbations of the data.

How can we compute such a **large margin classifier**? First we need to derive an expression for the distance of a point to the decision boundary. Referring to Figure 17.13(a), we see that

$$\boldsymbol{x} = \boldsymbol{x}_\perp + r\frac{\boldsymbol{w}}{||\boldsymbol{w}||} \tag{17.65}$$

where r is the distance of $\boldsymbol{x}$ from the decision boundary whose normal vector is $\boldsymbol{w}$, and $\boldsymbol{x}_\perp$ is the orthogonal projection of $\boldsymbol{x}$ onto this boundary.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

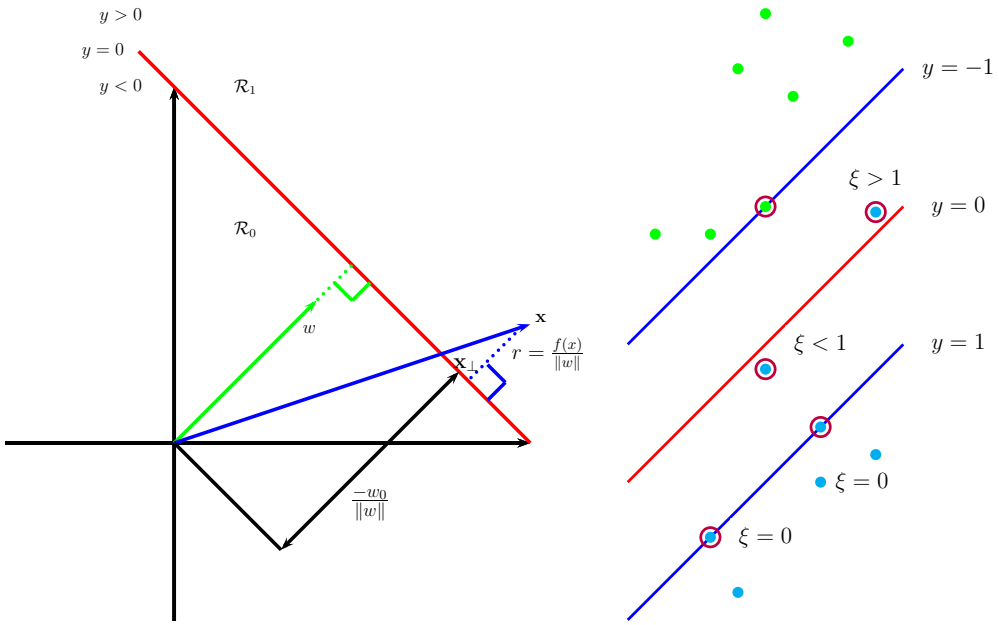


Figure 17.13: (a) Illustration of the geometry of a linear decision boundary in 2d. A point $\boldsymbol{x}$ is classified as belonging in decision region $\mathcal{R}_1$ if $f(\boldsymbol{x}) > 0$, otherwise it belongs in decision region $\mathcal{R}_0$; $\boldsymbol{w}$ is a vector which is perpendicular to the decision boundary. The term w_0 controls the distance of the decision boundary from the origin. $\boldsymbol{x}_\perp$ is the orthogonal projection of $\boldsymbol{x}$ onto the boundary. The signed distance of $\boldsymbol{x}$ from the boundary is given by $f(\boldsymbol{x})/||\boldsymbol{w}||$. Adapted from Figure 4.1 of [Bis06]. (b) Points with circles around them are support vectors, and have dual variables $\alpha_n > 0$. In the soft margin case, we associate a slack variable ξ_n with each example. If $0 < \xi_n < 1$, the point is inside the margin, but on the correct side of the decision boundary. If $\xi_n > 1$, the point is on the wrong side of the boundary. Adapted from Figure 7.3 of [Bis06].

We would like to maximize r, so we need to express it as a function of $\boldsymbol{w}$. First, note that

$$f(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + w_0 = (\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_\perp + w_0) + r\frac{\boldsymbol{w}^\mathsf{T}\boldsymbol{w}}{||\boldsymbol{w}||} = (\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_\perp + w_0) + r||\boldsymbol{w}|| \tag{17.66}$$

Since $0 = f(\boldsymbol{x}_\perp) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_\perp + w_0$, we have $f(\boldsymbol{x}) = r||\boldsymbol{w}||$ and hence $r = \frac{f(\boldsymbol{x})}{||\boldsymbol{w}||}$.

Since we want to ensure each point is on the correct side of the boundary, we also require $f(\boldsymbol{x}_n)\tilde{y}_n > 0$. We want to maximize the distance of the closest point, so our final objective becomes

$$\max_{\boldsymbol{w}, w_0} \frac{1}{||\boldsymbol{w}||} \min_{n=1}^{N} \left[\tilde{y}_n(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_0)\right] \tag{17.67}$$

Note that by rescaling the parameters using $\boldsymbol{w} \to k\boldsymbol{w}$ and $w_0 \to kw_0$, we do not change the distance of any point to the boundary, since the k factor cancels out when we divide by $||\boldsymbol{w}||$. Therefore let us define the scale factor such that $\tilde{y}_n f_n = 1$ for the point that is closest to the decision boundary.

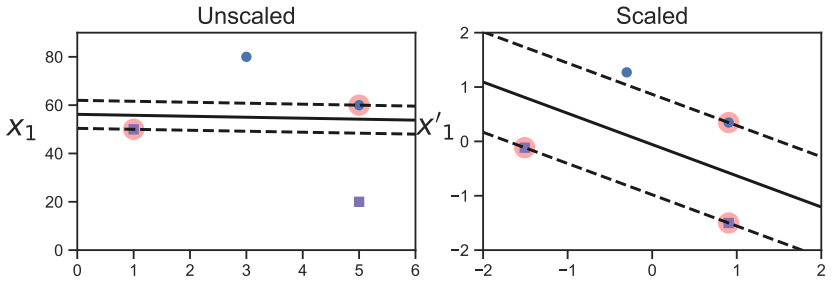


Figure 17.14: Illustration of the benefits of scaling the input features before computing a max margin classifier. Adapted from Figure 5.2 of [Gér19]. Generated by svm_classifier_feature_scaling.ipynb.

Hence we require $\tilde{y}_n f_n \geq 1$ for all n. Finally, note that maximizing $1/||\boldsymbol{w}||$ is equivalent to minimizing $||\boldsymbol{w}||^2$. Thus we get the new objective

$$\min_{\boldsymbol{w}, w_0} \frac{1}{2}||\boldsymbol{w}||^2 \quad \text{s.t.} \quad \tilde{y}_n(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_0) \geq 1, n = 1:N \tag{17.68}$$

(The factor of $\frac{1}{2}$ is added for convenience and doesn't affect the optimal parameters.) The constraint says that we want all points to be on the correct side of the decision boundary with a margin of at least 1.

Note that it is important to scale the input variables before using an SVM, otherwise the margin measures distance of a point to the boundary using all input dimensions equally. See Figure 17.14 for an illustration.

17.3.2 The dual problem

The objective in Equation (17.68) is a standard quadratic programming problem (Section 8.5.4), since we have a quadratic objective subject to linear constraints. This has $N + D + 1$ variables subject to N constraints, and is known as a **primal problem**.

In convex optimization, for every primal problem we can derive a **dual problem**. Let $\boldsymbol{\alpha} \in \mathbb{R}^N$ be the dual variables, corresponding to Lagrange multipliers that enforce the N inequality constraints. The generalized Lagrangian is given below (see Section 8.5.2 for relevant background information on constrained optimization):

$$\mathcal{L}(\boldsymbol{w}, w_0, \boldsymbol{\alpha}) = \frac{1}{2}\boldsymbol{w}^\mathsf{T}\boldsymbol{w} - \sum_{n=1}^{N} \alpha_n(\tilde{y}_n(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_0) - 1) \tag{17.69}$$

To optimize this, we must find a stationary point that satisfies

$$(\hat{\boldsymbol{w}}, \hat{w}_0, \hat{\boldsymbol{\alpha}}) = \min_{\boldsymbol{w}, w_0} \max_{\boldsymbol{\alpha}} \mathcal{L}(\boldsymbol{w}, w_0, \boldsymbol{\alpha}) \tag{17.70}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can do this by computing the partial derivatives wrt $\boldsymbol{w}$ and w_0 and setting to zero. We have

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}, w_0, \boldsymbol{\alpha}) = \boldsymbol{w} - \sum_{n=1}^{N} \alpha_n \tilde{y}_n \boldsymbol{x}_n \tag{17.71}$$

$$\frac{\partial}{\partial w_0} \mathcal{L}(\boldsymbol{w}, w_0, \boldsymbol{\alpha}) = -\sum_{n=1}^{N} \alpha_n \tilde{y}_n \tag{17.72}$$

and hence

$$\hat{\boldsymbol{w}} = \sum_{n=1}^{N} \hat{\alpha}_n \tilde{y}_n \boldsymbol{x}_n \tag{17.73}$$

$$0 = \sum_{n=1}^{N} \hat{\alpha}_n \tilde{y}_n \tag{17.74}$$

Plugging these into the Lagrangian yields the following

$$\mathcal{L}(\hat{\boldsymbol{w}}, \hat{w}_0, \boldsymbol{\alpha}) = \frac{1}{2} \hat{\boldsymbol{w}}^{\mathsf{T}} \hat{\boldsymbol{w}} - \sum_{n=1}^{N} \alpha_n \tilde{y}_n \hat{\boldsymbol{w}}^{\mathsf{T}} \boldsymbol{x}_n - \sum_{n=1}^{N} \alpha_n \tilde{y}_n w_0 + \sum_{n=1}^{N} \alpha_n \tag{17.75}$$

$$= \frac{1}{2} \hat{\boldsymbol{w}}^{\mathsf{T}} \hat{\boldsymbol{w}} - \hat{\boldsymbol{w}}^{\mathsf{T}} \hat{\boldsymbol{w}} - 0 + \sum_{n=1}^{N} \alpha_n \tag{17.76}$$

$$= -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \boldsymbol{x}_i^{\mathsf{T}} \boldsymbol{x}_j + \sum_{n=1}^{N} \alpha_n \tag{17.77}$$

This is called the **dual form** of the objective. We want to maximize this wrt $\boldsymbol{\alpha}$ subject to the constraints that $\sum_{n=1}^{N} \alpha_n \tilde{y}_n = 0$ and $0 \leq \alpha_n$ for $n = 1 : N$.

The above objective is a quadratic problem in N variables. Standard QP solvers take $O(N^3)$ time. However, specialized algorithms, which avoid the use of generic QP solvers, have been developed for this problem, such as the **sequential minimal optimization** or **SMO** algorithm [Pla98], which takes $O(N)$ to $O(N^2)$ time.

Since this is a convex objective, the solution must satisfy the KKT conditions (Section 8.5.2), which tell us that the following properties hold:

$$\alpha_n \geq 0 \tag{17.78}$$

$$\tilde{y}_n f(\boldsymbol{x}_n) - 1 \geq 0 \tag{17.79}$$

$$\alpha_n (\tilde{y}_n f(\boldsymbol{x}_n) - 1) = 0 \tag{17.80}$$

Hence either $\alpha_n = 0$ (in which case example n is ignored when computing $\hat{\boldsymbol{w}}$) or the constraint $\tilde{y}_n(\hat{\boldsymbol{w}}^{\mathsf{T}} \boldsymbol{x}_n + \hat{w}_0) = 1$ is active. This latter condition means that example n lies on the decision boundary; these points are known as the **support vectors**, as shown in Figure 17.13(b). We denote the set of support vectors by $\mathcal{S}$.

To perform prediction, we use

$$f(\boldsymbol{x}; \hat{\boldsymbol{w}}, \hat{w}_0) = \hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \boldsymbol{x}_n^\mathsf{T}\boldsymbol{x} + \hat{w}_0 \tag{17.81}$$

To solve for $\hat{w}_0$ we can use the fact that for any support vector, we have $\tilde{y}_n f(\boldsymbol{x}; \hat{\boldsymbol{w}}, \hat{w}_0) = 1$. Multiplying both sides by $\tilde{y}_n$, and exploiting the fact that $\tilde{y}_n^2 = 1$, we get $\hat{w}_0 = \tilde{y}_n - \hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x}_n$. In practice we get better results by averaging over all the support vectors to get

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} (\tilde{y}_n - \hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x}_n) = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} (\tilde{y}_n - \sum_{m \in \mathcal{S}} \alpha_m \tilde{y}_m \boldsymbol{x}_m^\mathsf{T}\boldsymbol{x}_n) \tag{17.82}$$

17.3.3 Soft margin classifiers

If the data is not linearly separable, there will be no feasible solution in which $\tilde{y}_n f_n \geq 1$ for all n. We therefore introduce **slack variables** $\xi_n \geq 0$ and replace the hard constraints that $\tilde{y}_n f_n \geq 0$ with the **soft margin constraints** that $\tilde{y}_n f_n \geq 1 - \xi_n$. The new objective becomes

$$\min_{\boldsymbol{w}, w_0, \boldsymbol{\xi}} \frac{1}{2}||\boldsymbol{w}||^2 + C \sum_{n=1}^N \xi_n \quad \text{s.t.} \quad \xi_n \geq 0, \quad \tilde{y}_n(\boldsymbol{x}_n^\mathsf{T}\boldsymbol{w} + w_0) \geq 1 - \xi_n \tag{17.83}$$

where $C \geq 0$ is a hyper parameter controlling how many points we allow to violate the margin constraint. (If $C = \infty$, we recover the unregularized, hard-margin classifier.)

The corresponding Lagrangian for the soft margin classifier becomes

$$\mathcal{L}(\boldsymbol{w}, w_0, \boldsymbol{\alpha}, \boldsymbol{\xi}, \boldsymbol{\mu}) = \frac{1}{2}\boldsymbol{w}^\mathsf{T}\boldsymbol{w} + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N \alpha_n(\tilde{y}_n(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_0) - 1 + \xi_n) - \sum_{n=1}^N \mu_n \xi_n \tag{17.84}$$

where $\alpha_n \geq 0$ and $\mu_n \geq 0$ are the Lagrange multipliers. Optimizing out $\boldsymbol{w}$, w_0 and $\boldsymbol{\xi}$ gives the dual form

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \boldsymbol{x}_i^\mathsf{T}\boldsymbol{x}_j \tag{17.85}$$

This is identical to the hard margin case; however, the constraints are different. In particular, the KKT conditions imply

$$0 \leq \alpha_n \leq C \tag{17.86}$$

$$\sum_{n=1}^N \alpha_n \tilde{y}_n = 0 \tag{17.87}$$

If $\alpha_n = 0$, the point is ignored. If $0 < \alpha_n < C$ then $\xi_n = 0$, so the point lies on the margin. If $\alpha_n = C$, the point can lie inside the margin, and can either be correctly classified if $\xi_n \leq 1$, or misclassified if $\xi_n > 1$. See Figure 17.13(b) for an illustration. Hence $\sum_n \xi_n$ is an upper bound on the number of misclassified points.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

As before, the bias term can be computed using

$$\hat{w}_0 = \frac{1}{|\mathcal{M}|} \sum_{n \in \mathcal{M}} (\tilde{y}_n - \sum_{m \in \mathcal{S}} \alpha_m \tilde{y}_m \boldsymbol{x}_m^\mathsf{T} \boldsymbol{x}_n) \tag{17.88}$$

where $\mathcal{M}$ is the set of points having $0 < \alpha_n < C$.

There is an alternative formulation of the soft margin SVM known as the **ν-SVM classifier** [Sch+00]. This involves maximizing

$$\mathcal{L}(\boldsymbol{\alpha}) = -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \boldsymbol{x}_i^\mathsf{T} \boldsymbol{x}_j \tag{17.89}$$

subject to the constraints that

$$0 \leq \alpha_n \leq 1/N \tag{17.90}$$

$$\sum_{n=1}^{N} \alpha_n \tilde{y}_n = 0 \tag{17.91}$$

$$\sum_{n=1}^{M} \alpha_n \geq \nu \tag{17.92}$$

This has the advantage that the parameter ν, which replaces C, can be interpreted as an upper bound on the fraction of **margin errors** (points for which $\xi_n > 0$), as well as a lower bound on the number of support vectors.

17.3.4 The kernel trick

So far we have converted the large margin binary classification problem into a dual problem in N unknowns ($\boldsymbol{\alpha}$) which (in general) takes $O(N^3)$ time to solve, which can be slow. However, the principal benefit of the dual problem is that we can replace all inner product operations $\boldsymbol{x}^\mathsf{T}\boldsymbol{x}'$ with a call to a positive definite (Mercer) kernel function, $\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}')$. This is called the **kernel trick**.

In particular, we can rewrite the prediction function in Equation (17.81) as follows:

$$f(\boldsymbol{x}) = \hat{\boldsymbol{w}}^\mathsf{T} \boldsymbol{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \boldsymbol{x}_n^\mathsf{T} \boldsymbol{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathcal{K}(\boldsymbol{x}_n, \boldsymbol{x}) + \hat{w}_0 \tag{17.93}$$

We also need to kernelize the bias term. This can be done by kernelizing Equation (17.82) as follows:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(\tilde{y}_i - (\sum_{j \in \mathcal{S}} \hat{\alpha}_j \tilde{y}_j \boldsymbol{x}_j)^\mathsf{T} \boldsymbol{x}_i \right) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(\tilde{y}_i - \sum_{j \in \mathcal{S}} \hat{\alpha}_j \tilde{y}_j \mathcal{K}(\boldsymbol{x}_j, \boldsymbol{x}_i) \right) \tag{17.94}$$

The kernel trick allows us to avoid having to deal with an explicit feature representation of our data, and allows us to easily apply classifiers to structured objects, such as strings and graphs.

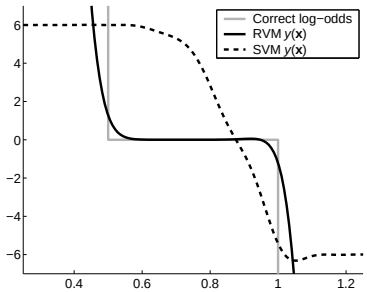


Figure 17.15: Log-odds vs x for 3 different methods. Adapted from Figure 10 of [Tip01]. Used with kind permission of Mike Tipping.

17.3.5 Converting SVM outputs into probabilities

An SVM classifier produces a hard-labeling, $\hat{y}(\boldsymbol{x}) = \text{sign}(f(\boldsymbol{x}))$. However, we often want a measure of confidence in our prediction. One heuristic approach is to interpret $f(\boldsymbol{x})$ as the log-odds ratio, $\log \frac{p(y=1|\boldsymbol{x})}{p(y=0|\boldsymbol{x})}$. We can then convert the output of an SVM to a probability using

$$p(y = 1|\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(af(\boldsymbol{x}) + b) \tag{17.95}$$

where a, b can be estimated by maximum likelihood on a separate validation set. (Using the training set to estimate a and b leads to severe overfitting.) This technique was first proposed in [Pla00], and is known as **Platt scaling**.

However, the resulting probabilities are not particularly well calibrated, since there is nothing in the SVM training procedure that justifies interpreting $f(\boldsymbol{x})$ as a log-odds ratio. To illustrate this, consider an example from [Tip01]. Suppose we have 1d data where $p(x|y = 0) = \text{Unif}(0, 1)$ and $p(x|y = 1) = \text{Unif}(0.5, 1.5)$. Since the class-conditional distributions overlap in the $[0.5, 1]$ range, the log-odds of class 1 over class 0 should be zero in this region, and infinite outside this region. We sampled 1000 points from the model, and then fit a probabilistic kernel classifier (an RVM, described in Section 17.4.1) and an SVM with a Gaussian kernel of width 0.1. Both models can perfectly capture the decision boundary, and achieve a generalization error of 25%, which is Bayes optimal in this problem. The probabilistic output from the RVM is a good approximation to the true log-odds, but this is not the case for the SVM, as shown in Figure 17.15.

17.3.6 Connection with logistic regression

We have seen that data points that are on the correct side of the decision boundary have $\xi_n = 0$; for the others, we have $\xi_n = 1 - \tilde{y}_n f(\boldsymbol{x}_n)$. Therefore we can rewrite the objective in Equation (17.83) as follows:

$$\mathcal{L}(\boldsymbol{w}) = \sum_{n=1}^{N} \ell_{\text{hinge}}(\tilde{y}_n, f(\boldsymbol{x}_n))) + \lambda |\boldsymbol{w}||^2 \tag{17.96}$$

where $\lambda = (2C)^{-1}$ and $\ell_{\text{hinge}}(y, \eta)$ is the **hinge loss** function defined by

$$\ell_{\text{hinge}}(\tilde{y}, \eta) = \max(0, 1 - \tilde{y}\eta) \tag{17.97}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

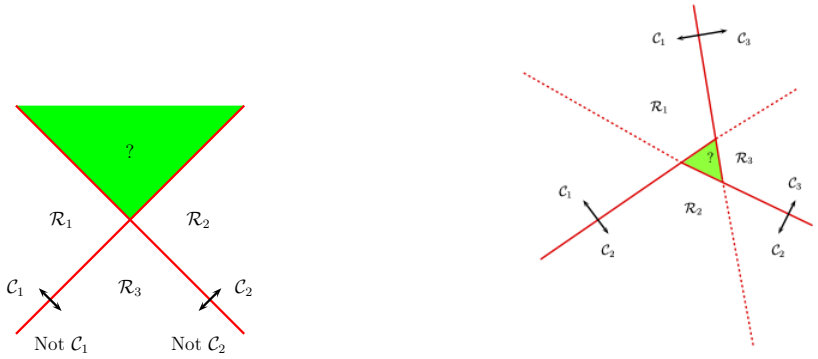


Figure 17.16: (a) The one-versus-rest approach. The green region is predicted to be both class 1 and class 2. (b) The one-versus-one approach. The label of the green region is ambiguous. Adapted from Figure 4.2 of [Bis06].

As we see from Figure 4.2, this is a convex, piecewise differentiable upper bound to the 0-1 loss, that has the shape of a partially open door hinge.

By contrast, (penalized) logistic regression optimizes

$$\mathcal{L}(\boldsymbol{w}) = \sum_{n=1}^{N} \ell_{ll}(\tilde{y}_n, f(\boldsymbol{x}_n))) + \lambda |\boldsymbol{w}||^2 \tag{17.98}$$

where the **log loss** is given by

$$\ell_{ll}(\tilde{y}, \eta) = -\log p(y|\eta) = \log(1 + e^{-\tilde{y}\eta}) \tag{17.99}$$

This is also plotted in Figure 4.2. We see that it is similar to the hinge loss, but with two important differences. First the hinge loss is piecewise linear, so we cannot use regular gradient methods to optimize it. (We can, however, compute the subgradient at $\tilde{y}\eta = 1$.) Second, the hinge loss has a region where it is strictly 0; this results in sparse estimates.

We see that both functions are convex upper bounds on the 0-1 loss, which is given by

$$\ell_{01}(\tilde{y}, \hat{y}) = \mathbb{I}(\tilde{y} \neq \hat{y}) = \mathbb{I}(\tilde{y}\,\hat{y} < 0) \tag{17.100}$$

These upper bounds are easier to optimize and can be viewed as surrogates for the 0-1 loss. See Section 4.3.2 for details.

17.3.7 Multi-class classification with SVMs

SVMs are inherently a binary classifier. One way to convert them to a multi-class classification model is to train C binary classifiers, where the data from class c is treated as positive, and the data from all the other classes is treated as negative. We then use the rule $\hat{y}(\boldsymbol{x}) = \arg\max_c f_c(\boldsymbol{x})$ to

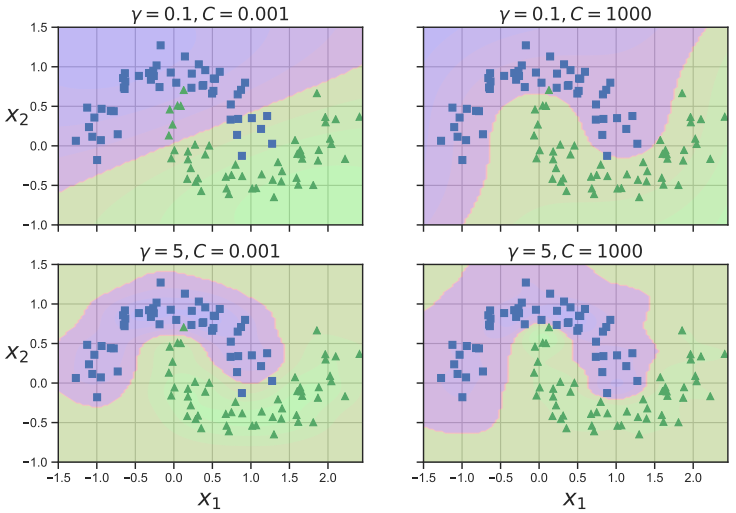


Figure 17.17: SVM classifier with RBF kernel with precision γ and regularizer C applied to two moons data. Adapted from Figure 5.9 of [Gér19]. Generated by svm_ classifier_ 2d.ipynb.

predict the final label, where $f_c(\boldsymbol{x}) = \log \frac{p(c=1|\boldsymbol{x})}{p(c=0|\boldsymbol{x})}$ is the score given by classifier c. This is known as the **one-versus-the-rest** approach (also called **one-vs-all**).

Unfortunately, this approach has several problems. First, it can result in regions of input space which are ambiguously labeled. For example, the green region at the top of Figure 17.16(a) is predicted to be both class 2 and class 1. A second problem is that the magnitude of the f_c's scores are not calibrated with each other, so it is hard to compare them. Finally, each binary subproblem is likely to suffer from the class imbalance problem (Section 10.3.8.2). For example, suppose we have 10 equally represented classes. When training f_1, we will have 10% positive examples and 90% negative examples, which can hurt performance.

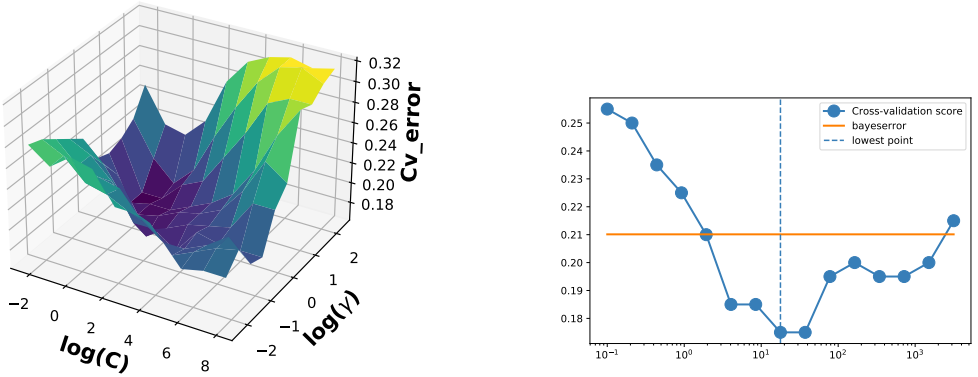
Another approach is to use the **one-versus-one** or OVO approach, also called **all pairs**, in which we train $C(C-1)/2$ classifiers to discriminate all pairs $f_{c,c'}$. We then classify a point into the class which has the highest number of votes. However, this can also result in ambiguities, as shown in Figure 17.16(b). Also, this requires fitting $O(C^2)$ models.

17.3.8 How to choose the regularizer C

SVMs require that you specify the kernel function and the parameter C. Typically C is chosen by cross-validation. Note, however, that C interacts quite strongly with the kernel parameters. For example, suppose we are using an RBF kernel with precision $\gamma = \frac{1}{2\sigma^2}$. If γ is large, corresponding to narrow kernels, we may need heavy regularization, and hence small C. If γ is small, a larger value of C should be used. So we see that γ and C are tightly coupled, as illustrated in Figure 17.17.

The authors of libsvm [HCL03] recommend using CV over a 2d grid with values $C \in \{2^{-5}, 2^{-3}, \ldots, 2^{15}\}$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 17.18: (a) A cross validation estimate of the 0-1 error for an SVM classifier with RBF kernel with different precisions $\gamma = 1/(2\sigma^2)$ and different regularizer $\lambda = 1/C$, applied to a synthetic data set drawn from a mixture of 2 Gaussians. (b) A slice through this surface for $\gamma = 5$ The red dotted line is the Bayes optimal error, computed using Bayes rule applied to the model used to generate the data. Adapted from Figure 12.6 of [HTF09]. Generated by svmCgammaDemo.ipynb.

and $\gamma \in \{2^{-15}, 2^{-13}, \ldots, 2^3\}$. See Figure 17.18 which shows the CV estimate of the 0-1 risk as a function of C and γ.

To choose C efficiently, one can develop a path following algorithm in the spirit of lars (Section 11.4.4). The basic idea is to start with C small, so that the margin is wide, and hence all points are inside of it and have $\alpha_i = 1$. By slowly increasing C, a small set of points will move from inside the margin to outside, and their α_i values will change from 1 to 0, as they cease to be support vectors. When C is maximal, the margin becomes empty, and no support vectors remain. See [Has+04] for the details.

17.3.9 Kernel ridge regression

Recall the equation for ridge regression from Equation (11.55):

$$\hat{\boldsymbol{w}}_{\text{map}} = (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda \mathbf{I}_D)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} = (\sum_n \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T} + \lambda \mathbf{I}_D)^{-1}(\sum_n \tilde{y}_n \boldsymbol{x}_n) \tag{17.101}$$

Using the matrix inversion lemma (Section 7.3.3), we can rewrite the ridge estimate as follows

$$\boldsymbol{w} = \mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{X}^\mathsf{T} + \lambda \mathbf{I}_N)^{-1}\boldsymbol{y} = \sum_n \boldsymbol{x}_n((\sum_n \boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n + \lambda \mathbf{I}_N)^{-1}\boldsymbol{y})_n \tag{17.102}$$

Let us define the following **dual variables**:

$$\boldsymbol{\alpha} \triangleq (\mathbf{X}\mathbf{X}^\mathsf{T} + \lambda \mathbf{I}_N)^{-1}\boldsymbol{y} = (\sum_n \boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n + \lambda \mathbf{I}_N)^{-1}\boldsymbol{y} \tag{17.103}$$

Then we can rewrite the **primal variables** as follows

$$\boldsymbol{w} = \mathbf{X}^\mathsf{T}\boldsymbol{\alpha} = \sum_{n=1}^{N} \alpha_n \boldsymbol{x}_n \tag{17.104}$$

This tells us that the solution vector is just a linear sum of the N training vectors. When we plug this in at test time to compute the predictive mean, we get

$$f(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} = \sum_{n=1}^{N} \alpha_n \boldsymbol{x}_n^\mathsf{T}\boldsymbol{x} \tag{17.105}$$

We can then use the kernel trick to rewrite this as

$$f(\boldsymbol{x}; \boldsymbol{w}) = \sum_{n=1}^{N} \alpha_n \mathcal{K}(\boldsymbol{x}_n, \boldsymbol{x}) \tag{17.106}$$

where

$$\boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I}_N)^{-1}\boldsymbol{y} \tag{17.107}$$

In other words,

$$f(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{k}^\mathsf{T}(\mathbf{K} + \lambda \mathbf{I}_N)^{-1}\boldsymbol{y} \tag{17.108}$$

where $\boldsymbol{k} = [\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}_1), \ldots, \mathcal{K}(\boldsymbol{x}, \boldsymbol{x}_N)]$. This is called **kernel ridge regression**.

The trouble with the above approach is that the solution vector $\boldsymbol{\alpha}$ is not sparse, so predictions at test time will take $O(N)$ time. We discuss a solution to this in Section 17.3.10.

17.3.10 SVMs for regression

Consider the following ℓ_2-regularized ERM problem:

$$J(\boldsymbol{w}, \lambda) = \lambda ||\boldsymbol{w}||^2 + \sum_{n=1}^{N} \ell(\tilde{y}_n, \hat{y}_n) \tag{17.109}$$

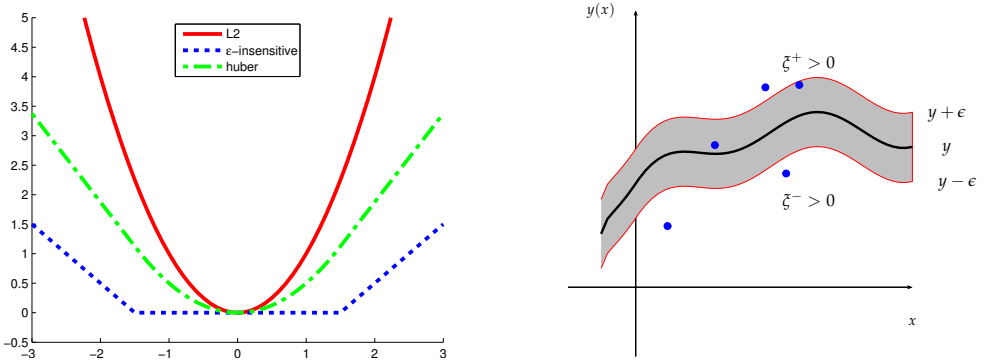
where $\hat{y}_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_0$. If we use the quadratic loss, $\ell(y, \hat{y}) = (y - \hat{y})^2$, where $y, \hat{y} \in \mathbb{R}$, we recover ridge regression (Section 11.3). If we then apply the kernel trick, we recover kernel ridge regression (Section 17.3.9).

The problem with kernel ridge regression is that the solution depends on all N training points, which makes it computationally intractable. However, by changing the loss function, we can make the optimal set of basis function coefficients, $\boldsymbol{\alpha}^*$, be sparse, as we show below.

In particular, consider the following variant of the Huber loss function (Section 5.1.5.3) called the **epsilon insensitive loss function**:

$$L_\epsilon(y, \hat{y}) \triangleq \begin{cases} 0 & \text{if } |y - \hat{y}| < \epsilon \\ |y - \hat{y}| - \epsilon & \text{otherwise} \end{cases} \tag{17.110}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 17.19: (a) Illustration of ℓ_2, Huber and ϵ-insensitive loss functions, where $\epsilon = 1.5$. Generated by huberLossPlot.ipynb. (b) Illustration of the ϵ-tube used in SVM regression. Points above the tube have $\xi_i^+ > 0$ and $\xi_i^- = 0$. Points below the tube have $\xi_i^+ = 0$ and $\xi_i^- > 0$. Points inside the tube have $\xi_i^+ = \xi_i^- = 0$. Adapted from Figure 7.7 of [Bis06].

This means that any point lying inside an **ϵ-tube** around the prediction is not penalized, as in Figure 17.19.

The corresponding objective function is usually written in the following form

$$J = \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{n=1}^{N} L_\epsilon(\tilde{y}_n, \hat{y}_n) \tag{17.111}$$

where $\hat{y}_n = f(\boldsymbol{x}_n) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_0$ and $C = 1/\lambda$ is a regularization constant. This objective is convex and unconstrained, but not differentiable, because of the absolute value function in the loss term. As in Section 11.4.9, where we discussed the lasso problem, there are several possible algorithms we could use. One popular approach is to formulate the problem as a constrained optimization problem. In particular, we introduce **slack variables** to represent the degree to which each point lies outside the tube:

$$\tilde{y}_n \leq f(\boldsymbol{x}_n) + \epsilon + \xi_n^+ \tag{17.112}$$

$$\tilde{y}_n \geq f(\boldsymbol{x}_n) - \epsilon - \xi_n^- \tag{17.113}$$

Given this, we can rewrite the objective as follows:

$$J = \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{n=1}^{N}(\xi_n^+ + \xi_n^-) \tag{17.114}$$

This is a quadratic function of $\boldsymbol{w}$, and must be minimized subject to the linear constraints in Equations 17.112-17.113, as well as the positivity constraints $\xi_n^+ \geq 0$ and $\xi_n^- \geq 0$. This is a standard quadratic program in $2N + D + 1$ variables.

By forming the Lagrangian and optimizing, as we did above, one can show that the optimal solution has the following form

$$\hat{\boldsymbol{w}} = \sum_n \alpha_n \boldsymbol{x}_n \tag{17.115}$$

where $\alpha_n \geq 0$ are the dual variables. (See e.g., [SS02] for details.) Fortunately, the $\boldsymbol{\alpha}$ vector is sparse, meaning that many of its entries are equal to 0. This is because the loss doesn't care about errors which are smaller than ϵ. The degree of sparsity is controlled by C and ϵ.

The $\boldsymbol{x}_n$ for which $\alpha_n > 0$ are called the **support vectors**; these are points for which the errors lie on or outside the ϵ tube. These are the only training examples we need to keep for prediction at test time, since

$$f(\boldsymbol{x}) = \hat{w}_0 + \hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x} = \hat{w}_0 + \sum_{n:\alpha_n>0} \alpha_n \boldsymbol{x}_n^\mathsf{T}\boldsymbol{x} \tag{17.116}$$

Finally, we can use the kernel trick to get

$$f(\boldsymbol{x}) = \hat{w}_0 + \sum_{n:\alpha_n>0} \alpha_n \mathcal{K}(\boldsymbol{x}_n, \boldsymbol{x}) \tag{17.117}$$

This overall technique is called **support vector machine regression** or **SVM regression** for short, and was first proposed in [VGS97].

In Figure 17.20, we give an example where we use an RBF kernel with $\gamma = 1$. When C is small, the model is heavily regularized; when C is large, the model is less regularized and can fit the data better. We also see that when ϵ is small, the tube is smaller, so there are more support vectors.

17.4 Sparse vector machines

GPs are very flexible models, but incur an $O(N)$ time cost at prediction time, which can be prohibitive. SVMs solve that problem by estimating a sparse weight vector. However, SVMs do not give calibrated probabilistic outputs.

We can get the best of both worlds by using parametric models, where the feature vector is defined using basis functions centered on each of the training points, as follows:

$$\boldsymbol{\phi}(\boldsymbol{x}) = [\mathcal{K}(\boldsymbol{x}, \boldsymbol{x}_1), \ldots, \mathcal{K}(\boldsymbol{x}, \boldsymbol{x}_N)] \tag{17.118}$$

where $\mathcal{K}$ is any similarity kernel, not necessarily a Mercer kernel. Equation (17.118) maps $\boldsymbol{x} \in \mathcal{X}$ into $\boldsymbol{\phi}(\boldsymbol{x}) \in \mathbb{R}^N$. We can plug this new feature vector into any discriminative model, such as logistic regression. Since we have $D = N$ parameters, we need to use some kind of regularization, to prevent overfitting. If we fit such a model using ℓ_2 regularization (which we will call **L2VM**, for ℓ_2-vector machine), the result often has good predictive performance, but the weight vector $\boldsymbol{w}$ will be dense, and will depend on all N training points. A natural solution is to impose a sparsity-promoting prior on $\boldsymbol{w}$, so that not all the exemplars need to be kept. We call such methods **sparse vector machines**.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

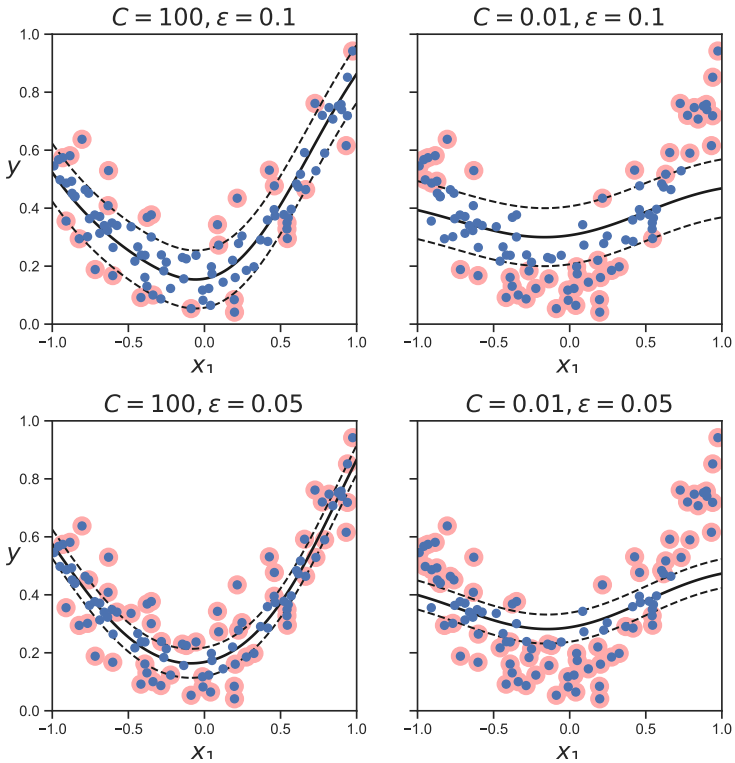


Figure 17.20: Illustration of support vector regression. Adapted from Figure 5.11 of [Gér19]. Generated by svm_regression_1d.ipynb.

17.4.1 Relevance vector machines (RVMs)

The simplest way to ensure $\boldsymbol{w}$ is sparse is to use ℓ_1 regularization, as in Section 11.4. We call this **L1VM** or **Laplace vector machine**, since this approach is equivalent to using MAP estimation with a Laplace prior for $\boldsymbol{w}$.

However, sometimes ℓ_1 regularization does not result in a sufficient level of sparsity for a given level of accuracy. An alternative approach is based on the use of **ARD** or **automatic relevancy determination**, which uses type II maximum likelihood (aka empirical Bayes) to estimate a sparse weight vector [Mac95; Nea96]. If we apply this technique to a feature vector defined in terms of kernels, as in Equation (17.118), we get a method called the **relevance vector machine** or **RVM** [Tip01; TF03].

17.4.2 Comparison of sparse and dense kernel methods

In Figure 17.21, we compare L2VM, L1VM, RVM and an SVM using an RBF kernel on a binary classification problem in 2d. We use cross validation to pick $C = 1/\lambda$ for the SVM (see Section 17.3.8),

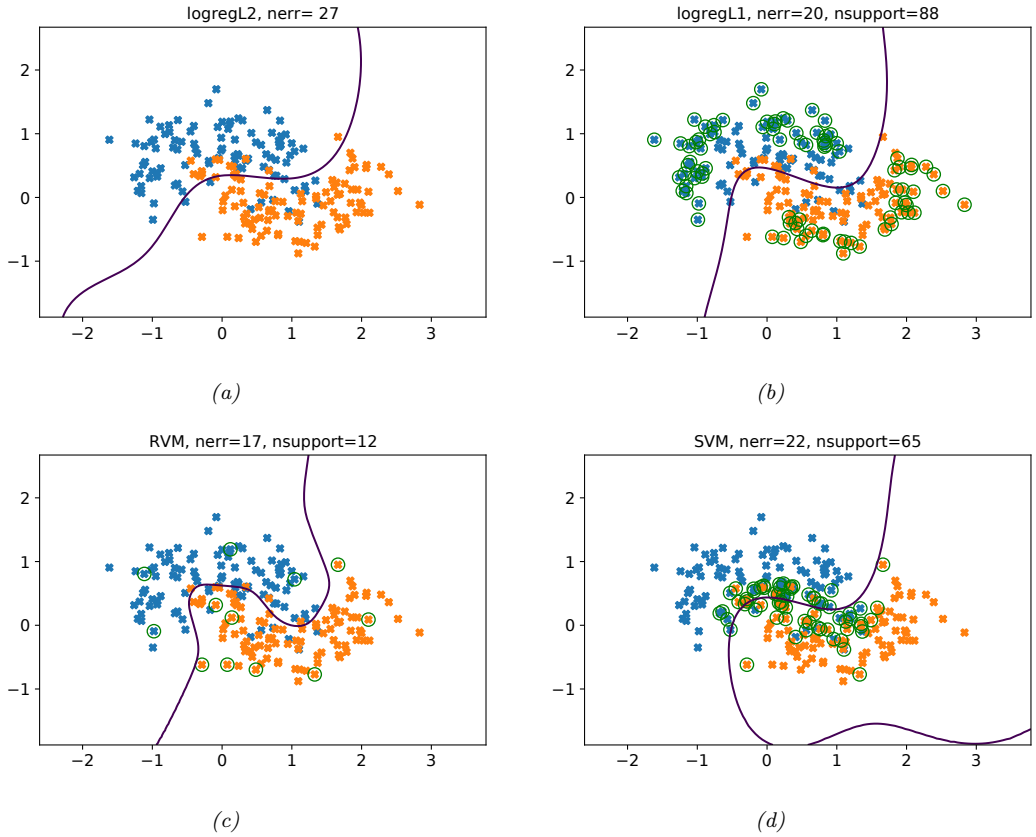


Figure 17.21: Example of non-linear binary classification using an RBF kernel with bandwidth $\sigma = 0.3$. (a) L2VM. (b) L1VM. (c) RVM. (d) SVM. Green circles denote the support vectors. Generated by kernelBinaryClassifDemo.ipynb.

and then use the same value of the regularizer for L2VM and L1VM. We see that all the methods give similar predictive performance. However, we see that the RVM is the sparsest model, so it will be the fastest at run time.

In Figure 17.22, we compare L2VM, L1VM, RVM and an SVM using an RBF kernel on a 1d regression problem. Again, we see that predictions are quite similar, but RVM is the sparsest, then L1VM, then SVM. This is further illustrated in Figure 17.23.

Beyond these small empirical examples, we provide a more general summary of the different methods in Table 17.1. The columns of this table have the following meaning:

- Optimize $\boldsymbol{w}$: a key question is whether the objective $\mathcal{L}(\boldsymbol{w}) = -\log p(\mathcal{D}|\boldsymbol{w}) - \log p(\boldsymbol{w})$ is convex or not. L2VM, L1VM and SVMs have convex objectives. RVMs do not. GPs are Bayesian methods that integrate out the weights $\boldsymbol{w}$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

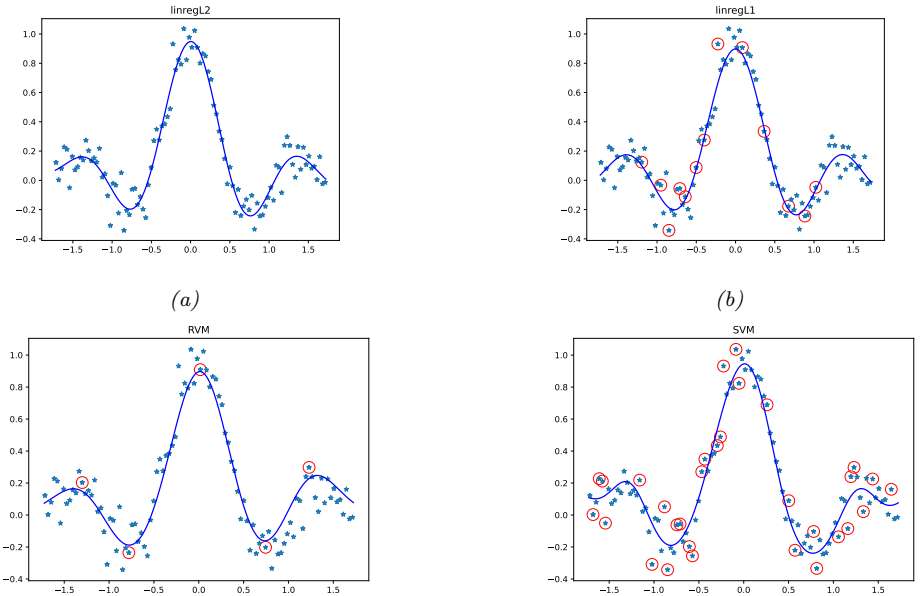


Figure 17.22: Model fits for kernel based regression on the noisy sinc function using an RBF kernel with bandwidth $\sigma = 0.3$. (a) L2VM with $\lambda = 0.5$. (b) L1VM with $\lambda = 0.5$. (c) RVM. (d) SVM regression with $C = 1/\lambda$. chosen by cross validation. Red circles denote the retained training exemplars. Generated by rvm_regression_1d.ipynb.

- Optimize kernel: all the methods require that we "tune" the kernel parameters, such as the bandwidth of the RBF kernel, as well as the level of regularization. For methods based on Gaussian priors, including L2VM, RVMs and GPs, we can use efficient gradient based optimizers to maximize the marginal likelihood. For SVMs and L1VMs, we must use cross validation, which is slower (see Section 17.3.8).

- Sparse: L1VM, RVMs and SVMs are sparse kernel methods, in that they only use a subset of the training examples. GPs and L2VM are not sparse: they use all the training examples. The principle advantage of sparsity is that prediction at test time is usually faster. However, this usually results in overconfidence in the predictions.

- Probabilistic: All the methods except for SVMs produce probabilistic output of the form $p(y|\boldsymbol{x})$. SVMs produce a "confidence" value that can be converted to a probability, but such probabilities are usually very poorly calibrated (see Section 17.3.5).

- Multiclass: All the methods except for SVMs naturally work in the multiclass setting, by using a categorical distribution instead of a Bernoulli. The SVM can be made into a multiclass classifier, but there are various difficulties with this approach, as discussed in Section 17.3.7.

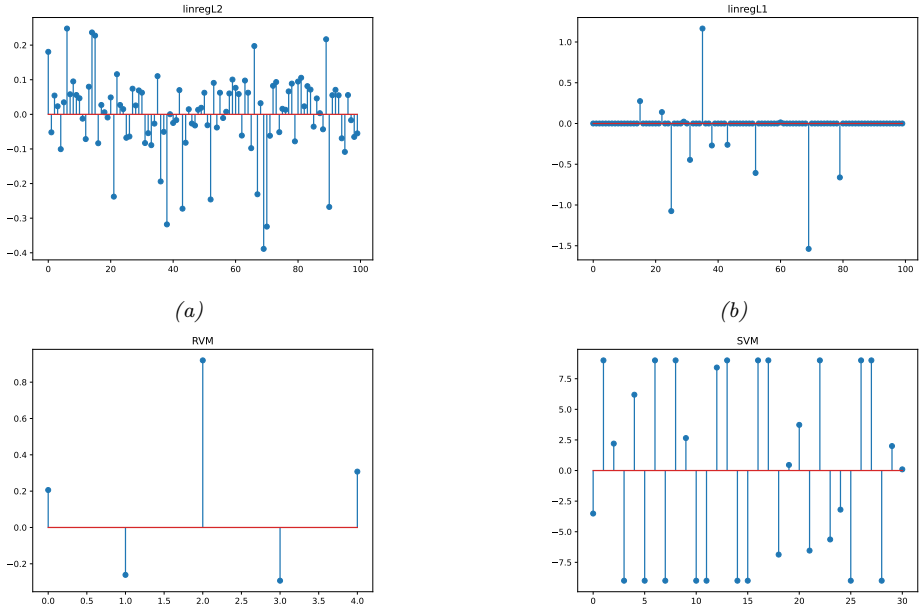


Figure 17.23: Estimated coefficients for the models in Figure 17.22. Generated by rvm_regression_1d.ipynb.

Method	Opt. $\boldsymbol{w}$	Opt. kernel	Sparse	Prob.	Multiclass	Non-Mercer	Section
SVM	Convex	CV	Yes	No	Indirectly	No	17.3
L2VM	Convex	EB	No	Yes	Yes	Yes	17.4.1
L1VM	Convex	CV	Yes	Yes	Yes	Yes	17.4.1
RVM	Not convex	EB	Yes	Yes	Yes	Yes	17.4.1
GP	N/A	EB	No	Yes	Yes	No	17.2.7

Table 17.1: Comparison of various kernel based classifiers. EB = empirical Bayes, CV = cross validation. See text for details.

- Mercer kernel: SVMs and GPs require that the kernel is positive definite; the other techniques do not, since the kernel function in Equation (17.118) can be an arbitrary function of two inputs.

17.5 Exercises

Exercise 17.1 [Fitting an SVM classifier by hand †]

(Source: Jaakkola.) Consider a dataset with 2 points in 1d: $x_1 = 0$ with label $y_1 = -1$ and $x_2 = \sqrt{2}$ with label $y_2 = 1$. Consider mapping each point to 3d using the feature vector $\boldsymbol{\phi}(x) = [1, \sqrt{2}x, x^2]^T$. (This is

equivalent to using a second order polynomial kernel.) The max margin classifier has the form

$$\min ||\boldsymbol{w}||^2 \quad \text{s.t.} \tag{17.119}$$

$$y_1(\boldsymbol{w}^T\phi(\boldsymbol{x}_1) + w_0) \geq 1 \tag{17.120}$$

$$y_2(\boldsymbol{w}^T\phi(\boldsymbol{x}_2) + w_0) \geq 1 \tag{17.121}$$

a. Write down a vector that is parallel to the optimal vector $\boldsymbol{w}$. Hint: recall from Figure 17.12(a) that $\boldsymbol{w}$ is perpendicular to the decision boundary between the two points in the 3d feature space.

b. What is the value of the margin that is achieved by this $\boldsymbol{w}$? Hint: recall that the margin is the distance from each support vector to the decision boundary. Hint 2: think about the geometry of 2 points in space, with a line separating one from the other.

c. Solve for $\boldsymbol{w}$, using the fact that the margin is equal to $1/||\boldsymbol{w}||$.

d. Solve for w_0 using your value for $\boldsymbol{w}$ and Equations 17.119 to 17.121. Hint: the points will be on the decision boundary, so the inequalities will be tight.

e. Write down the form of the discriminant function $f(x) = w_0 + \boldsymbol{w}^T\boldsymbol{\phi}(x)$ as an explicit function of x.

18 Trees, Forests, Bagging, and Boosting

18.1 Classification and regression trees (CART)

Classification and regression trees or **CART** models [BFO84], also called **decision trees** [Qui86; Qui93], are defined by recursively partitioning the input space, and defining a local model in each resulting region of input space. The overall model can be represented by a tree, with one leaf per region, as we explain below.

18.1.1 Model definition

We start by considering regression trees, where all inputs are real-valued. The tree consists of a set of nested decision rules. At each node i, the feature dimension d_i of the input vector $\boldsymbol{x}$ is compared to a threshold value t_i, and the input is then passed down to the left or right branch, depending on whether it is above or below threshold. At the leaves of the tree, the model specifies the predicted output for any input that falls into that part of the input space.

For example, consider the regression tree in Figure 18.1(a). The first node asks if x_1 is less than some threshold t_1. If yes, we then ask if x_2 is less than some other threshold t_2. If yes, we enter the bottom left leaf node. This corresponds to the region of space defined by

$$R_1 = \{\boldsymbol{x} : x_1 \leq t_1, x_2 \leq t_2\} \tag{18.1}$$

We can associate this region with the predicted output, say $y = 2$. In a similar way, we can partition the entire input space into 5 regions using **axis parallel splits**, as shown in Figure 18.1(b).[1]

Formally, a regression tree can be defined by

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \sum_{j=1}^{J} w_j \mathbb{I}(\boldsymbol{x} \in R_j) \tag{18.2}$$

where R_j is the region specified by the j'th leaf node, w_j is the predicted output for that node,

$$w_j = \frac{\sum_{n=1}^{N} y_n \mathbb{I}(\boldsymbol{x}_n \in R_j)}{\sum_{n=1}^{N} \mathbb{I}(\boldsymbol{x}_n \in R_j)} \tag{18.3}$$

1. By using enough splits (i.e., deep enough trees), we can make a piecewise linear approximation to decision boundaries with more complex shapes, but it may require a lot of data to fit such a model.

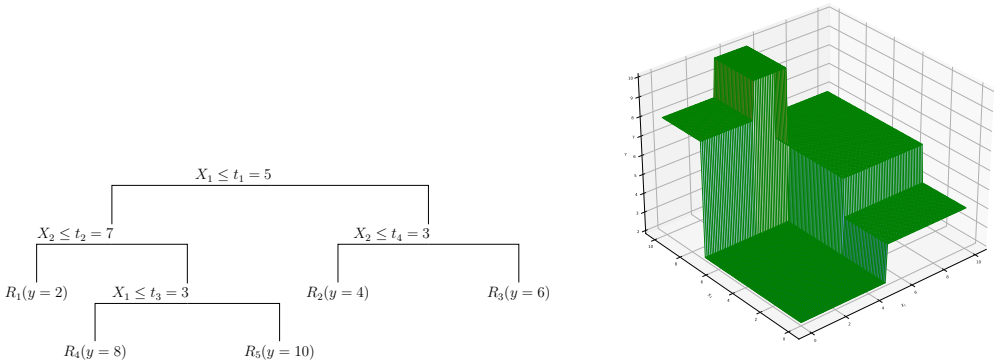


Figure 18.1: (a) A regression tree on two inputs. (b) Corresponding piecewise constant surface, where the regions have heights 2, 4, 6, 8 and 10. Adapted from Figure 9.2 of [HTF09]. Generated by regtreeSurfaceDemo.ipynb.

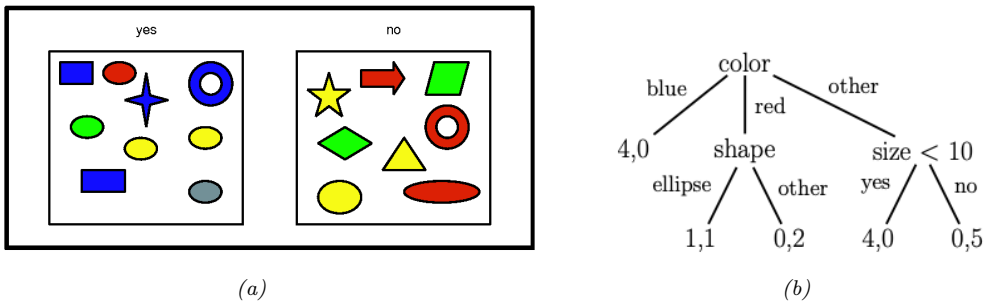


Figure 18.2: (a) A set of shapes with corresponding binary labels. The features are: color (values "blue", "red", "other"), shape (values "ellipse", "other"), and size (real-valued). (b) A hypothetical classification tree fitted to this data. A leaf labeled as (n_1, n_0) means that there are n_1 positive examples that fall into this partition, and n_0 negative examples.

and $\boldsymbol{\theta} = \{(R_j, w_j) : j = 1 : J\}$, where J is the number of nodes. The regions themselves are defined by the feature dimensions that are used in each split, and the corresponding thresholds, on the path from the root to the leaf. For example, in Figure 18.1(a), we have $R_1 = [(x_1 \leq t_1), (x_2 \leq t_2)]$, $R_4 = [(x_1 \leq t_1), (x_2 > t_2), (x_3 \leq t_3)]$, etc. (For categorical inputs, we can define the splits based on comparing feature x_i to each of the possible values for that feature, rather than comparing to a numeric threshold.) We discuss how to learn these regions in Section 18.1.2.

For classification problems, the leaves contain a distribution over the class labels, rather than just the mean response. See Figure 18.2 for an example of a classification tree.

18.1.2 Model fitting

To fit the model, we need to minimize the following loss:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{n=1}^{N} \ell(y_n, f(\boldsymbol{x}_n; \boldsymbol{\theta})) = \sum_{j=1}^{J} \sum_{\boldsymbol{x}_n \in R_j} \ell(y_n, w_j) \tag{18.4}$$

Unfortunately, this is not differentiable, because of the need to learn the discrete tree structure. Indeed, finding the optimal partitioning of the data is NP-complete [HR76]. The standard practice is to use a greedy procedure, in which we iteratively grow the tree one node at a time. This approach is used by CART [BFO84], **C4.5** [Qui93], and **ID3** [Qui86], which are three popular implementations of the method.

The idea is as follows. Suppose we are at node i; let $\mathcal{D}_i = \{(\boldsymbol{x}_n, y_n) \in N_i\}$ be the set of examples that reach this node. We will consider how to split this node into a left branch and right branch so as to minimize the error in each child subtree.

If the j'th feature is a real-valued scalar, we can partition the data at node i by comparing to a threshold t. The set of possible thresholds $\mathcal{T}_j$ for feature j can be obtained by sorting the unique values of $\{x_{nj}\}$. For example, if feature 1 has the values $\{4.5, -12, 72, -12\}$, then we set $\mathcal{T}_1 = \{-12, 4.5, 72\}$. For each possible threshold, we define the left and right splits, $\mathcal{D}_i^L(j,t) = \{(\boldsymbol{x}_n, y_n) \in N_i : x_{n,j} \leq t\}$ and $\mathcal{D}_i^R(j,t) = \{(\boldsymbol{x}_n, y_n) \in N_i : x_{n,j} > t\}$.

If the j'th feature is categorical, with K_j possible values, then we check if the feature is equal to each of those values or not. This defines a set of K_j possible binary splits: $\mathcal{D}_i^L(j,t) = \{(\boldsymbol{x}_n, y_n) \in N_i : x_{n,j} = t\}$ and $\mathcal{D}_i^R(j,t) = \{(\boldsymbol{x}_n, y_n) \in N_i : x_{n,j} \neq t\}$.) (Alternatively, we could allow for a multi-way split, as in Figure 18.2(b). However, this may cause **data fragmentation**, in which too little data might "fall" into each subtree, resulting in overfitting. Therefore it is more common to use binary splits.)

Once we have computed $\mathcal{D}_i^L(j,t)$ and $\mathcal{D}_i^R(j,t)$ for each j and t at node i, we choose the best feature j_i to split on, and the best value for that feature, t_i, as follows:

$$(j_i, t_i) = \arg \min_{j \in \{1,\ldots,D\}} \min_{t \in \mathcal{T}_j} \frac{|\mathcal{D}_i^L(j,t)|}{|\mathcal{D}_i|} c(\mathcal{D}_i^L(j,t)) + \frac{|\mathcal{D}_i^R(j,t)|}{|\mathcal{D}_i|} c(\mathcal{D}_i^R(j,t)) \tag{18.5}$$

We now discuss the cost function $c(\mathcal{D}_i)$ which is used to evaluate the cost of node i. For regression, we can use the mean squared error

$$\text{cost}(\mathcal{D}_i) = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}_i} (y_n - \overline{y})^2 \tag{18.6}$$

where $\overline{y} = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}_i} y_n$ is the mean of the response variable for examples reaching node i.

For classification, we first compute the empirical distribution over class labels for this node:

$$\hat{\pi}_{ic} = \frac{1}{|\mathcal{D}_i|} \sum_{n \in \mathcal{D}_i} \mathbb{I}(y_n = c) \tag{18.7}$$

Given this, we can then compute the **Gini index**

$$G_i = \sum_{c=1}^{C} \hat{\pi}_{ic}(1 - \hat{\pi}_{ic}) = \sum_c \hat{\pi}_{ic} - \sum_c \hat{\pi}_{ic}^2 = 1 - \sum_c \hat{\pi}_{ic}^2 \tag{18.8}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is the expected error rate. To see this, note that $\hat{\pi}_{ic}$ is the probability a random entry in the leaf belongs to class c, and $1 - \hat{\pi}_{ic}$ is the probability it would be misclassified.

Alternatively we can define cost as the entropy or **deviance** of the node:

$$H_i = \mathbb{H}(\hat{\boldsymbol{\pi}}_i) = -\sum_{c=1}^{C} \hat{\pi}_{ic} \log \hat{\pi}_{ic} \tag{18.9}$$

A node that is **pure** (i.e., only has examples of one class) will have 0 entropy.

Given one of the above cost functions, we can use Equation (18.5) to pick the best feature, and best threshold at each node. We then partition the data, and call the fitting algorithm recursively on each subset of the data.

In [SL99], they propose to soften the hard split decision at each node by using a binary logistic regression model. This is called a **differentiable decision tree** or **soft decision tree**. For a fixed tree structure, this allows them to fit the model by gradient-based optimization methods.

18.1.3 Regularization

If we let the tree become deep enough, it can achieve 0 error on the training set (assuming no label noise), by partioning the input space into sufficiently small regions where the output is constant. However, this will typically result in overfitting. To prevent this, there are two main approaches. The first is to stop the tree growing process according to some heuristic, such as having too few examples at a node, or reaching a maximum depth. The second approach is to grow the tree to its maximum depth, where no more splits are possible, and then to **prune** it back, by merging split subtrees back into their parent (see e.g., [BA97b]). This can partially overcome the greedy nature of top-down tree growing. (For example, consider applying the top-down approach to the xor data in Figure 13.1: the algorithm would never make any splits, since each feature on its own has no predictive power.) However, forward growing and backward pruning is slower than the greedy top-down approach.

18.1.4 Handling missing input features

In general, it is hard for discriminative models, such as neural networks, to handle missing input features, as we discussed in Section 1.5.5. However, for trees, there are some simple heuristics that can work well.

The standard heuristic for handling missing inputs in decision trees is to look for a series of "backup" variables, which can induce a similar partition to the chosen variable at any given split; these can be used in case the chosen variable is unobserved at test time. These are called **surrogate splits**. This method finds highly correlated features, and can be thought of as learning a local joint model of the input. This has the advantage over a generative model of not modeling the entire joint distribution of inputs, but it has the disadvantage of being entirely ad hoc. A simpler approach, applicable to categorical variables, is to code "missing" as a new value, and then to treat the data as fully observed.

18.1.5 Pros and cons

Tree models are popular for several reasons:

- They are easy to interpret.

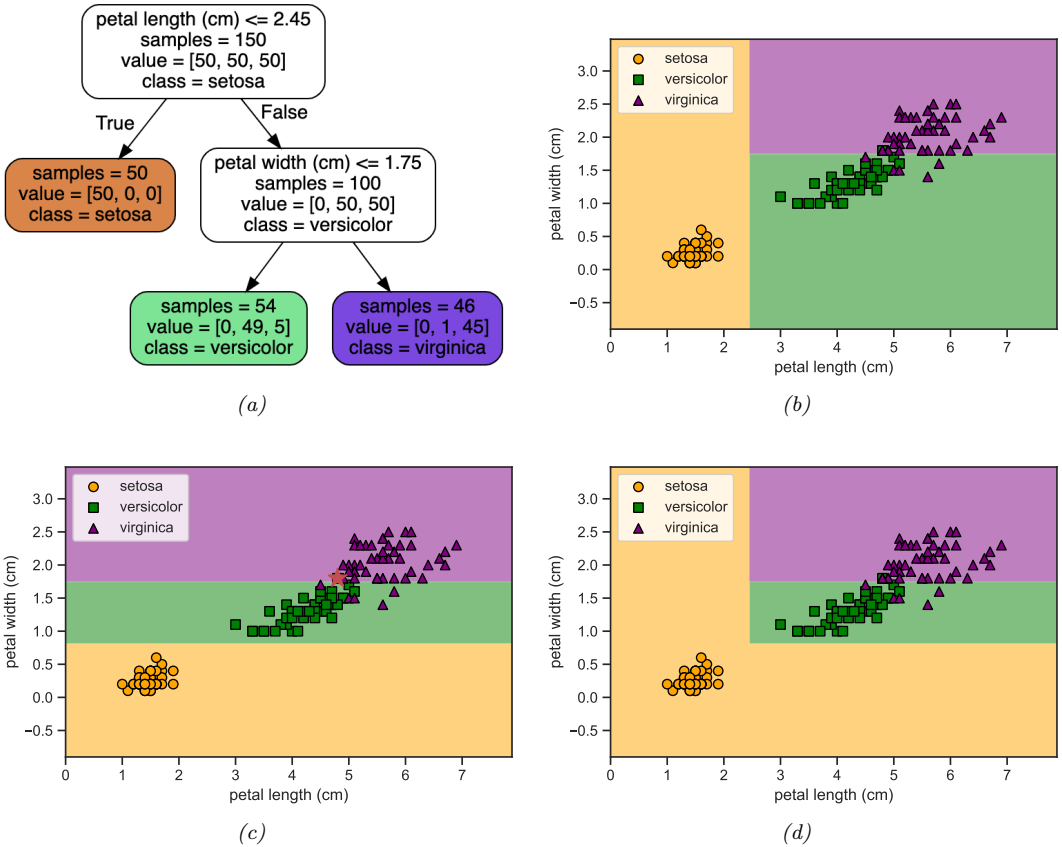


Figure 18.3: (a) A decision tree of depth 2 fit to the iris data, using just the petal length and petal width features. Leaf nodes are color coded according to the majority class. The number of training samples that pass from the root to each node is shown inside each box, as well as how many of these values fall into each class. This can be normalized to get a distribution over class labels for each node. (b) Decision surface induced by (a). (c) Fit to data where we omit a single data point (shown by red star). (d) Ensemble of the two models in (b) and (c). Generated by dtree_sensitivity.ipynb.

- They can easily handle mixed discrete and continuous inputs.
- They are insensitive to monotone transformations of the inputs (because the split points are based on ranking the data points), so there is no need to standardize the data.
- They perform automatic variable selection.
- They are relatively robust to outliers.
- They are fast to fit, and scale well to large data sets.
- They can handle missing input features.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

However, tree models also have some disadvantages. The primary one is that they do not predict very accurately compared to other kinds of model. This is in part due to the greedy nature of the tree construction algorithm.

A related problem is that trees are **unstable**: small changes to the input data can have large effects on the structure of the tree, due to the hierarchical nature of the tree-growing process, causing errors at the top to affect the rest of the tree. For example, consider the tree in Figure 18.3b. Omitting even a single data point from the training set can result in a dramatically different decision surface, as shown in Figure 18.3c, due to the use of axis parallel splits. (Omitting features can also cause instability.) In Section 18.3 and Section 18.4, we will turn this instability into a virtue.

18.2 Ensemble learning

In Section 18.1, we saw that decision trees can be quite unstable, in the sense that their predictions might vary a lot if the training data is perturbed. In other words, decision trees are a high variance estimator. A simple way to reduce variance is to average multiple models. This is called **ensemble learning**. The result model has the form

$$f(y|\boldsymbol{x}) = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} f_m(y|\boldsymbol{x}) \tag{18.10}$$

where f_m is the m'th base model. The ensemble will have similar bias to the base models, but lower variance, generally resulting in improved overall performance (see Section 4.7.6.3 for details on the bias-variance tradeoff).

Averaging is a sensible way to combine predictions from regression models. For classifiers, it can sometimes be better to take a majority vote of the outputs. (This is sometimes called a **committee method**.) To see why this can help, suppose each base model is a binary classifier with an accuracy of θ, and suppose class 1 is the correct class. Let $Y_m \in \{0, 1\}$ be the prediction for the m'th model, and let $S = \sum_{m=1}^{M} Y_m$ be the number of votes for class 1. We define the final predictor to be the majority vote, i.e., class 1 if $S > M/2$ and class 0 otherwise. The probability that the ensemble will pick class 1 is

$$p = \Pr(S > M/2) = 1 - B(M/2, M, \theta) \tag{18.11}$$

where $B(x, M, \theta)$ is the cdf of the binomial distribution with parameters M and θ evaluated at x. For $\theta = 0.51$ and $M = 1000$, we get $p = 0.73$ and with $M = 10,000$ we get $p = 0.97$.

The performance of the voting approach is dramatically improved, because we assumed each predictor made independent errors. In practice, their mistakes may be correlated, but as long as we ensemble sufficiently diverse models, we can still come out ahead.

18.2.1 Stacking

An alternative to using an unweighted average or majority vote is to learn how to combine the base models, by using

$$f(y|\boldsymbol{x}) = \sum_{m \in \mathcal{M}} w_m f_m(y|\boldsymbol{x}) \tag{18.12}$$

This is called **stacking**, which stands for "stacked generalization" [Wol92]. Note that the combination weights used by stacking need to be trained on a separate dataset, otherwise they would put all their mass on the best performing base model.

18.2.2 Ensembling is not Bayes model averaging

It is worth noting that an ensemble of models is not the same as using Bayes model averaging over models (Section 4.6), as pointed out in [Min00]. An ensemble considers a larger hypothesis class of the form

$$p(y|\boldsymbol{x}, \boldsymbol{w}, \boldsymbol{\theta}) = \sum_{m \in \mathcal{M}} w_m p(y|\boldsymbol{x}, \boldsymbol{\theta}_m) \tag{18.13}$$

whereas BMA uses

$$p(y|\boldsymbol{x}, \mathcal{D}) = \sum_{m \in \mathcal{M}} p(m|\mathcal{D}) p(y|\boldsymbol{x}, m, \mathcal{D}) \tag{18.14}$$

The key difference is that in the case of BMA, the weights $p(m|\mathcal{D})$ sum to one, and in the limit of infinite data, only a single model will be chosen (namely the MAP model). By contrast, the ensemble weights w_m are arbitrary, and don't collapse in this way to a single model.

18.3 Bagging

In this section, we discuss **bagging** [Bre96], which stands for "bootstrap aggregating". This is a simple form of ensemble learning in which we fit M different base models to different randomly sampled versions of the data; this encourages the different models to make diverse predictions. The datasets are sampled with replacement (a technique known as bootstrap sampling, Section 4.7.3), so a given example may appear multiple times, until we have a total of N examples per model (where N is the number of original data points).

The disadvantage of bootstrap is that each base model only sees, on average, 63% of the unique input examples. To see why, note that the chance that a single item will not be selected from a set of size N in any of N draws is $(1 - 1/N)^N$. In the limit of large N, this becomes $e^{-1} \approx 0.37$, which means only $1 - 0.37 = 0.63$ of the data points will be selected.

The 37% of the training instances that are not used by a given base model are called **out-of-bag instances** (oob). We can use the predicted performance of the base model on these oob instances as an estimate of test set performance. This provides a useful alternative to cross validation.

The main advantage of bootstrap is that it prevents the ensemble from relying too much on any individual training example, which enhances robustness and generalization [Gra04]. For example, comparing Figure 18.3b and Figure 18.3c, we see that omitting a single example from the training set can have a large impact on the decision tree that we learn (even though the tree growing algorithm is otherwise deterministic). By averaging the predictions from both of these models, we get the more reasonable prediction model in Figure 18.3d. This advantage generally increases with the size of the ensemble, as shown in Figure 18.4. (Of course, larger ensembles take more memory and more time.)

Bagging does not always improve performance. In particular, it relies on the base models being unstable estimators, so that omitting some of the data significantly changes the resulting model fit.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

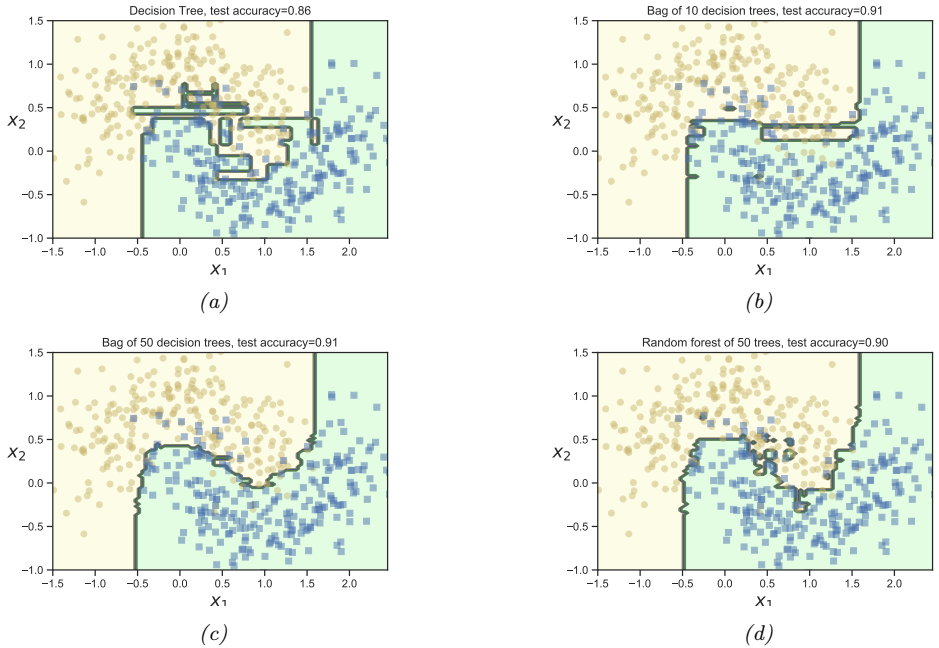


Figure 18.4: (a) A single decision tree. (b-c) Bagging ensemble of 10 and 50 trees. (d) Random forest of 50 trees. Adapted from Figure 7.5 of [Gér19]. Generated by bagging_trees.ipynb and rf_demo_2d.ipynb.

This is the case for decision trees, but not for other models, such as nearest neighbor classifiers. For neural networks, the story is more mixed. They can be unstable wrt their training set. On the other hand, deep networks will underperform if they only see 63% of the data, so bagged DNNs do not usually work well [NTL20].

18.4 Random forests

Bagging relies on the assumption that re-running the same learning algorithm on different subsets of the data will result in sufficiently diverse base models. The technique known as **random forests** [Bre01] tries to decorrelate the base learners even further by learning trees based on a randomly chosen subset of input variables (at each node of the tree), as well as a randomly chosen subset of data cases. It does this by modifying Equation (18.5) so the the feature split dimension j is optimized over a random subset of the features, $S_i \subset \{1, \ldots, D\}$.

For example, consider the email spam dataset [HTF09, p301]. This dataset contains 4601 email messages, each of which is classified as spam (1) or non-spam (0). The data was open sourced by George Forman from Hewlett-Packard (HP) Labs.

There are 57 quantitative (real-valued) features, as follows:

- 48 features corresponding to the percentage of words in the email that match a given word, such as "remove" or "labs".

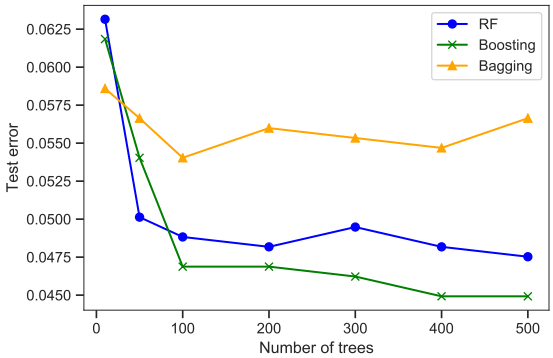


Figure 18.5: Preditive accuracy vs size of tree ensemble for bagging, random forests and gradient boosting with log loss. Adapted from Figure 15.1 of [HTF09]. Generated by spam_tree_ensemble_compare.ipynb.

- 6 features corresponding to the percentage of characters in the email that match a given character, namely `; . [ ! $ #`
- 3 features corresponding to the average length, max length, and sum of lengths of uninterrupted sequences of capital letters. (These features are called CAPAVE, CAPMAX and CAPTOT.)

Figure 18.5 shows that random forests work much better than bagged decision trees, because many input features are irrelevant. (We also see that a method called "boosting", discussed in Section 18.5, works even better; however, this requires sequentially fitting trees, whereas random forests can be fit in parallel.)

18.5 Boosting

Ensembles of trees, whether fit by bagging or the random forest algorithm, corresponding to a model of the form

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \sum_{m=1}^{M} \beta_m F_m(\boldsymbol{x};\boldsymbol{\theta}_m) \tag{18.15}$$

where F_m is the m'th tree, and β_m is the corresponding weight, often set to $\beta_m = 1/M$. We can generalize this by allowing the F_m functions to be general function approximators, such as neural networks, not just trees. The result is called an **additive model** [HTF09]. We can think of this as a linear model with **adaptive basis functions**. The goal, as usual, is to minimize the empirical loss (with an optional regularizer):

$$\mathcal{L}(f) = \sum_{i=1}^{N} \ell(y_i, f(\boldsymbol{x}_i)) \tag{18.16}$$

Boosting [Sch90; FS96] is an algorithm for sequentially fitting additive models where each F_m is a binary classifier that returns $F_m \in \{-1, +1\}$. In particular, we first fit F_1 on the original data,

and then we weight the data samples by the errors made by F_1, so misclassified examples get more weight. Next we fit F_2 to this weighted data set. We keep repeating this process until we have fit the desired number M of components. (M is a hyper-parameter that controls the complexity of the overall model, and can be chosen by monitoring performance on a validation set, and using early stopping.)

It can be shown that, as long as each F_m has an accuracy that is better than chance (even on the weighted dataset), then the final ensemble of classifiers will have higher accuracy than any given component. That is, if F_m is a **weak learner** (so its accuracy is only slightly better than 50%), then we can boost its performance using the above procedure so that the final f becomes a **strong learner**. (See e.g., [SF12] for more details on the learning theory approach to boosting.)

Note that boosting reduces the bias of the strong learner, by fitting trees that depend on each other, whereas bagging and RF reduce the variance by fitting independent trees. In many cases, boosting can work better. See Figure 18.5 for an example.

The original boosting algorithm focused on binary classification with a particular loss function that we will explain in Section 18.5.3, and was derived from the PAC learning theory framework (see Section 5.4.4). In the rest of this section, we focus on a more statistical version of boosting, due to [FHT00; Fri01], which works with arbitrary loss functions, making the method suitable for regression, multi-class classification, ranking, etc. Our presentation is based on [HTF09, ch10] and [BH07], which should be consulted for further details.

18.5.1 Forward stagewise additive modeling

In this section, we discuss **forward stagewise additive modeling**, in which we sequentially optimize the objective in Equation (18.16) for general (differentiable) loss functions, where f is an additive model as in Equation 18.15. That is, at iteration m, we compute

$$(\beta_m, \boldsymbol{\theta}_m) = \underset{\beta, \boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^{N} \ell(y_i, f_{m-1}(\boldsymbol{x}_i) + \beta F(\boldsymbol{x}_i; \boldsymbol{\theta})) \tag{18.17}$$

We then set

$$f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \beta_m F(\boldsymbol{x}; \boldsymbol{\theta}_m) = f_{m-1}(\boldsymbol{x}) + \beta_m F_m(\boldsymbol{x}) \tag{18.18}$$

(Note that we do not adjust the parameters of previously added models.) The details on how to perform this optimization step depend on the loss function that we choose, and (in some cases) on the form of the weak learner F, as we discuss below.

18.5.2 Quadratic loss and least squares boosting

Suppose we use squared error loss, $\ell(y, \hat{y}) = (y - \hat{y})^2$. In this case, the i'th term in the objective at step m becomes

$$\ell(y_i, f_{m-1}(\boldsymbol{x}_i) + \beta F(\boldsymbol{x}_i; \boldsymbol{\theta})) = (y_i - f_{m-1}(\boldsymbol{x}_i) - \beta F(\boldsymbol{x}_i; \boldsymbol{\theta}))^2 = (r_{im} - \beta F(\boldsymbol{x}_i; \boldsymbol{\theta}))^2 \tag{18.19}$$

where $r_{im} = y_i - f_{m-1}(\boldsymbol{x}_i)$ is the residual of the current model on the i'th observation. We can minimize the above objective by simply setting $\beta = 1$, and fitting F to the residual errors. This is called **least squares boosting** [BY03].

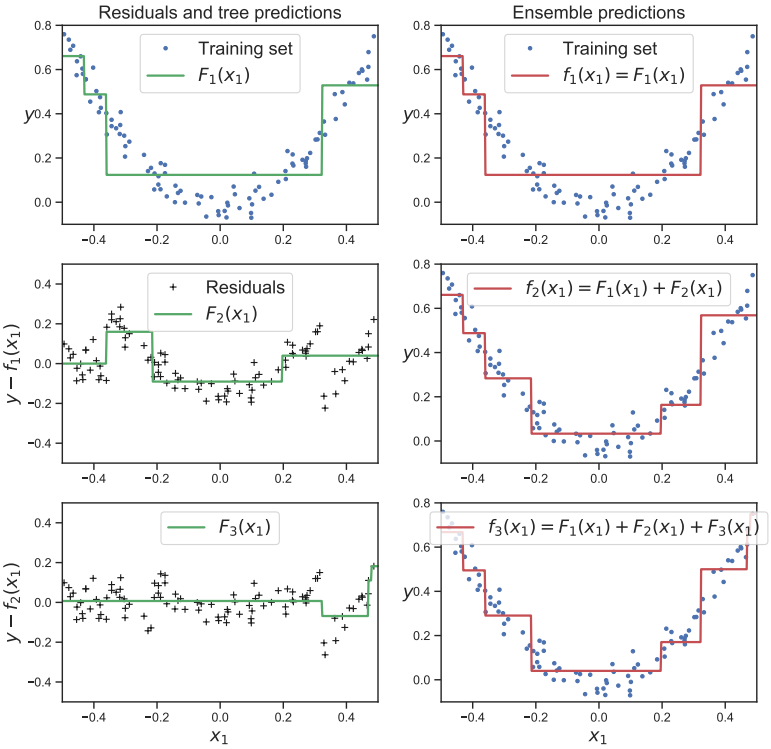


Figure 18.6: Illustration of boosting using a regression tree of depth 2 applied to a 1d dataset. Adapted from Figure 7.9 of [Gér19]. Generated by boosted_regr_trees.ipynb.

We give an example of this process in Figure 18.6, where we use a regression tree of depth 2 as the weak learner. On the left, we show the result of fitting the weak learner to the residuals, and on the right, we show the current strong learner. We see how each new weak learner that is added to the ensemble corrects the errors made by earlier versions of the model.

18.5.3 Exponential loss and AdaBoost

Suppose we are interested in binary classification, i.e., predicting $\tilde{y}_i \in \{-1, +1\}$. Let us assume the weak learner computes

$$p(y = 1|\boldsymbol{x}) = \frac{e^{F(\boldsymbol{x})}}{e^{-F(\boldsymbol{x})} + e^{F(\boldsymbol{x})}} = \frac{1}{1 + e^{-2F(\boldsymbol{x})}} \tag{18.20}$$

so $F(\boldsymbol{x})$ returns half the log odds. We know from Equation (10.13) that the negative log likelihood is given by

$$\ell(\tilde{y}, F(\boldsymbol{x})) = \log(1 + e^{-2\tilde{y}F(\boldsymbol{x})}) \tag{18.21}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

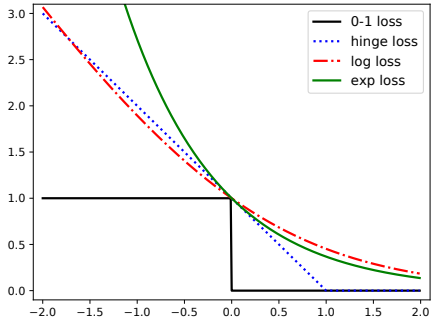


Figure 18.7: Illustration of various loss functions for binary classification. The horizontal axis is the margin $m(\boldsymbol{x}) = \tilde{y}F(\boldsymbol{x})$, the vertical axis is the loss. The log loss uses log base 2. Generated by hinge_loss_plot.ipynb.

We can minimize this by ensuring that the **margin** $m(\boldsymbol{x}) = \tilde{y}F(\boldsymbol{x})$ is as large as possible. We see from Figure 18.7 that the log loss is a smooth upper bound on the 0-1 loss. We also see that it penalizes negative margins more heavily than positive ones, as desired (since positive margins are already correctly classified).

However, we can also use other loss functions. In this section, we consider the **exponential loss**

$$\ell(\tilde{y}, F(\boldsymbol{x})) = \exp(-\tilde{y}F(\boldsymbol{x})) \tag{18.22}$$

We see from Figure 18.7 that this is also a smooth upper bound on the 0-1 loss. In the population setting (with infinite sample size), the optimal solution to the exponential loss is the same as for log loss. To see this, we can just set the derivative of the expected loss (for each $\boldsymbol{x}$) to zero:

$$\frac{\partial}{\partial F(\boldsymbol{x})}\mathbb{E}\left[e^{-\tilde{y}f(\boldsymbol{x})}|\boldsymbol{x}\right] = \frac{\partial}{\partial F(\boldsymbol{x})}[p(\tilde{y}=1|\boldsymbol{x})e^{-F(\boldsymbol{x})} + p(\tilde{y}=-1|\boldsymbol{x})e^{F(\boldsymbol{x})}] \tag{18.23}$$

$$= -p(\tilde{y}=1|\boldsymbol{x})e^{-F(\boldsymbol{x})} + p(\tilde{y}=-1|\boldsymbol{x})e^{F(\boldsymbol{x})} \tag{18.24}$$

$$= 0 \Rightarrow \frac{p(\tilde{y}=1|\boldsymbol{x})}{p(\tilde{y}=-1|\boldsymbol{x})} = e^{2F(\boldsymbol{x})} \tag{18.25}$$

However, it turns out that the exponential loss is easier to optimize in the boosting setting, as we show below. (We consider the log loss case in Section 18.5.4.)

We now discuss how to solve for the m'th weak learner, F_m, when we use exponential loss. We will assume that the base classifier F_m returns a binary class label; the resulting algorithm is called **discrete AdaBoost** [FHT00]. If F_m returns a probability instead, a modified algorithm, known as **real AdaBoost**, can be used [FHT00].

At step m we have to minimize

$$L_m(F) = \sum_{i=1}^{N} \exp[-\tilde{y}_i(f_{m-1}(\boldsymbol{x}_i) + \beta F(\boldsymbol{x}_i))] = \sum_{i=1}^{N} \omega_{i,m} \exp(-\beta \tilde{y}_i F(\boldsymbol{x}_i)) \tag{18.26}$$

where $\omega_{i,m} \triangleq \exp(-\tilde{y}_i f_{m-1}(\boldsymbol{x}_i))$ is a weight applied to datacase i, and $\tilde{y}_i \in \{-1, +1\}$. We can rewrite this objective as follows:

$$L_m = e^{-\beta} \sum_{\tilde{y}_i = F(\boldsymbol{x}_i)} \omega_{i,m} + e^{\beta} \sum_{\tilde{y}_i \neq F(\boldsymbol{x}_i)} \omega_{i,m} \tag{18.27}$$

$$= (e^{\beta} - e^{-\beta}) \sum_{i=1}^{N} \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\boldsymbol{x}_i)) + e^{-\beta} \sum_{i=1}^{N} \omega_{i,m} \tag{18.28}$$

Consequently the optimal function to add is

$$F_m = \operatorname*{argmin}_F \sum_{i=1}^{N} \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\boldsymbol{x}_i)) \tag{18.29}$$

This can be found by applying the weak learner to a weighted version of the dataset, with weights $\omega_{i,m}$.

All that remains is to solve for the size of the update, β. Subsituting F_m into L_m and solving for β we find

$$\beta_m = \frac{1}{2} \log \frac{1 - \mathrm{err}_m}{\mathrm{err}_m} \tag{18.30}$$

where

$$\mathrm{err}_m = \frac{\sum_{i=1}^{N} \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_i))}{\sum_{i=1}^{N} \omega_{i,m}} \tag{18.31}$$

Therefore overall update becomes

$$f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \beta_m F_m(\boldsymbol{x}) \tag{18.32}$$

After updating the strong learner, we need to recompute the weights for the next iteration, as follows:

$$\omega_{i,m+1} = e^{-\tilde{y}_i f_m(\boldsymbol{x}_i)} = e^{-\tilde{y}_i f_{m-1}(\boldsymbol{x}_i) - \tilde{y}_i \beta_m F_m(\boldsymbol{x}_i)} = \omega_{i,m} e^{-\tilde{y}_i \beta_m F_m(\boldsymbol{x}_i)} \tag{18.33}$$

If $\tilde{y}_i = F_m(\boldsymbol{x}_i)$, then $\tilde{y}_i F_m(\boldsymbol{x}_i) = 1$, and if $\tilde{y}_i \neq F_m(\boldsymbol{x}_i)$, then $\tilde{y}_i F_m(\boldsymbol{x}_i) = -1$. Hence $-\tilde{y}_i F_m(\boldsymbol{x}_i) = 2\mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_o)) - 1$, so the update becomes

$$\omega_{i,m+1} = \omega_{i,m} e^{\beta_m (2\mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_i)) - 1)} = \omega_{i,m} e^{2\beta_m \mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_i))} e^{-\beta_m} \tag{18.34}$$

Since the $e^{-\beta_m}$ is constant across all examples, it can be dropped. If we then define $\alpha_m = 2\beta_m$, the update becomes

$$\omega_{i,m+1} = \begin{cases} \omega_{i,m} e^{\alpha_m} & \text{if } \tilde{y}_i \neq F_m(\boldsymbol{x}_i) \\ \omega_{i,m} & \text{otherwise} \end{cases} \tag{18.35}$$

Thus we see that we exponentially increase weights of misclassified examples. The resulting algorithm shown in Algorithm 18.1, and is known as **Adaboost.M1** [FS96].

A multiclass generalization of exponential loss, and an adaboost-like algorithm to minimize it, known as **SAMME** (stagewise additive modeling using a multiclass exponential loss function), is described in [Has+09]. This is implemented in scikit learn (the AdaBoostClassifier class).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 18.1: Adaboost.M1, for binary classification with exponential loss

$\omega_i = 1/N$
for $m = 1 : M$ **do**
  Fit a classifier $F_m(\boldsymbol{x})$ to the training set using weights $\boldsymbol{w}$
  Compute $\text{err}_m = \frac{\sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_i))}{\sum_{i=1}^N \omega_{i,m}}$
  Compute $\alpha_m = \log[(1 - \text{err}_m)/\text{err}_m]$
  Set $\omega_i \leftarrow \omega_i \exp[\alpha_m \mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_i))]$
Return $f(\boldsymbol{x}) = \text{sgn}\left[\sum_{m=1}^M \alpha_m F_m(\boldsymbol{x})\right]$

18.5.4 LogitBoost

The trouble with exponential loss is that it puts a lot of weight on misclassified examples, as is apparent from the exponential blowup on the left hand side of Figure 18.7. This makes the method very sensitive to outliers (mislabeled examples). In addition, $e^{-\tilde{y}f}$ is not the logarithm of any pmf for binary variables $\tilde{y} \in \{-1, +1\}$; consequently we cannot recover probability estimates from $f(\boldsymbol{x})$.

A natural alternative is to use log loss, as we discussed in Section 18.5.3. This only punishes mistakes linearly, as is clear from Figure 18.7. Furthermore, it means that we will be able to extract probabilities from the final learned function, using

$$p(y = 1|\boldsymbol{x}) = \frac{e^{f(\boldsymbol{x})}}{e^{-f(\boldsymbol{x})} + e^{f(\boldsymbol{x})}} = \frac{1}{1 + e^{-2f(\boldsymbol{x})}} \tag{18.36}$$

The goal is to minimze the expected log-loss, given by

$$L_m(F) = \sum_{i=1}^N \log\left[1 + \exp\left(-2\tilde{y}_i(f_{m-1}(\boldsymbol{x}) + F(\boldsymbol{x}_i))\right)\right] \tag{18.37}$$

By performing a Newton update on this objective (similar to IRLS), one can derive the algorithm shown in Algorithm 18.2. This is known as **logitBoost** [FHT00]. The key subroutine is the ability of the weak learner F to solve a weighted least squares problem. This method can be generalized to the multi-class setting, as explained in [FHT00].

18.5.5 Gradient boosting

Rather than deriving new versions of boosting for every different loss function, it is possible to derive a generic version, known as **gradient boosting** [Fri01; Mas+00]. To explain this, imagine solving $\hat{\boldsymbol{f}} = \text{argmin}_{\boldsymbol{f}} \mathcal{L}(\boldsymbol{f})$ by performing gradient descent in the space of functions. Since functions are infinite dimensional objects, we will represent them by their values on the training set, $\boldsymbol{f} = (f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_N))$. At step m, let $\boldsymbol{g}_m$ be the gradient of $\mathcal{L}(\boldsymbol{f})$ evaluated at $\boldsymbol{f} = \boldsymbol{f}_{m-1}$:

$$g_{im} = \left[\frac{\partial \ell(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\right]_{f=f_{m-1}} \tag{18.38}$$

Algorithm 18.2: LogitBoost, for binary classification with log-loss

$\omega_i = 1/N$, $\pi_i = 1/2$
for $m = 1 : M$ **do**
  Compute the working response $z_i = \frac{y_i^* - \pi_i}{\pi_i(1-\pi_i)}$
  Compute the weights $\omega_i = \pi_i(1 - \pi_i)$
  $F_m = \text{argmin}_F \sum_{i=1}^N \omega_i (z_i - F(\boldsymbol{x}_i))^2$
  Update $f(\boldsymbol{x}) \leftarrow f(\boldsymbol{x}) + \frac{1}{2} F_m(\boldsymbol{x})$
  Compute $\pi_i = 1/(1 + \exp(-2f(\boldsymbol{x}_i)))$;
Return $f(\boldsymbol{x}) = \text{sgn}\left[\sum_{m=1}^M F_m(\boldsymbol{x})\right]$

Name	Loss	$-\partial \ell(y_i, f(\boldsymbol{x}_i))/\partial f(\boldsymbol{x}_i)$
Squared error	$\frac{1}{2}(y_i - f(\boldsymbol{x}_i))^2$	$y_i - f(\boldsymbol{x}_i)$
Absolute error	$\lvert y_i - f(\boldsymbol{x}_i)\rvert$	$\text{sgn}(y_i - f(\boldsymbol{x}_i))$
Exponential loss	$\exp(-\tilde{y}_i f(\boldsymbol{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\boldsymbol{x}_i))$
Binary Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$
Multiclass logloss	$-\sum_c y_{ic} \log \pi_{ic}$	$y_{ic} - \pi_{ic}$

Table 18.1: Some commonly used loss functions and their gradients. For binary classification problems, we assume $\tilde{y}_i \in \{-1, +1\}$, and $\pi_i = \sigma(2f(\boldsymbol{x}_i))$. For regression problems, we assume $y_i \in \mathbb{R}$. Adapted from [HTF09, p360] and [BH07, p483].

Gradients of some common loss functions are given in Table 18.1. We then make the update

$$\boldsymbol{f}_m = \boldsymbol{f}_{m-1} - \beta_m \boldsymbol{g}_m \tag{18.39}$$

where β_m is the step length, chosen by

$$\beta_m = \underset{\beta}{\text{argmin}}\, \mathcal{L}(\boldsymbol{f}_{m-1} - \beta \boldsymbol{g}_m) \tag{18.40}$$

In its current form, this is not much use, since it only optimizes f at a fixed set of N points, so we do not learn a function that can generalize. However, we can modify the algorithm by fitting a weak learner to approximate the negative gradient signal. That is, we use this update

$$F_m = \underset{F}{\text{argmin}} \sum_{i=1}^N (-g_{im} - F(\boldsymbol{x}_i))^2 \tag{18.41}$$

The overall algorithm is summarized in Algorithm 18.3. We have omitted the line search step for β_m, which is not strictly necessary, as argued in [BH07]. However, we have introduced a learning rate or **shrinkage factor** $0 < \nu \leq 1$, to control the size of the updates, for regularization purposes.

If we apply this algorithm using squared loss, we recover L2Boosting, since $-g_{im} = y_i - f_{m-1}(\boldsymbol{x}_i)$ is just the residual error. We can also apply this algorithm to other loss functions, such as absolute loss or Huber loss (Section 5.1.5.3), which is useful for robust regression problems.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 18.3: Gradient boosting

Initialize $f_0(\boldsymbol{x}) = \text{argmin}_F \sum_{i=1}^N L(y_i, F(\boldsymbol{x}_i))$
for $m = 1 : M$ **do**
  Compute the gradient residual using $r_{im} = -\left[\frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\right]_{f(\boldsymbol{x}_i)=f_{m-1}(\boldsymbol{x}_i)}$
  Use the weak learner to compute $F_m = \text{argmin}_F \sum_{i=1}^N (r_{im} - F(\boldsymbol{x}_i))^2$
  Update $f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \nu F_m(\boldsymbol{x})$
Return $f(\boldsymbol{x}) = f_M(\boldsymbol{x})$

For classification, we can use log-loss. In this case, we get an algorithm known as **BinomialBoost** [BH07]. The advantage of this over LogitBoost is that it does not need to be able to do weighted fitting: it just applies any black-box regression model to the gradient vector. To apply this to multi-class classification, we can fit C separate regression trees, using the pseudo residual of the form

$$-g_{icm} = \frac{\partial \ell(y_i, f_{1m}(\boldsymbol{x}_i), \ldots, f_{Cm}(\boldsymbol{x}_i))}{\partial f_{cm}(\boldsymbol{x}_i)} = \mathbb{I}(y_i = c) - \pi_{ic} \tag{18.42}$$

Although the trees are fit separately, their predictions are combined via a softmax transform

$$p(y = c|\boldsymbol{x}) = \frac{e^{f_c(\boldsymbol{x})}}{\sum_{c'=1}^C e^{f_{c'}(\boldsymbol{x})}} \tag{18.43}$$

When we have large datasets, we can use a stochastic variant in which we subsample (without replacement) a random fraction of the data to pass to the regression tree at each iteration. This is called **stochastic gradient boosting** [Fri99]. Not only is it faster, but it can also generalize better, because subsampling the data is a form of regularization.

18.5.5.1 Gradient tree boosting

In practice, gradient boosting nearly always assumes that the weak learner is a regression tree, which is a model of the form

$$F_m(\boldsymbol{x}) = \sum_{j=1}^{J_m} w_{jm} \mathbb{I}(\boldsymbol{x} \in R_{jm}) \tag{18.44}$$

where w_{jm} is the predicted output for region R_{jm}. (In general, w_{jm} could be a vector.) This combination is called **gradient boosted regression trees**, or **gradient tree boosting**. (A related version is known as **MART**, which stands for "multivariate additive regression trees" [FM03].)

To use this in gradient boosting, we first find good regions R_{jm} for tree m using standard regression tree learning (see Section 18.1) on the residuals; we then (re)solve for the weights of each leaf by solving

$$\hat{w}_{jm} = \underset{w}{\text{argmin}} \sum_{\boldsymbol{x}_i \in R_{jm}} \ell(y_i, f_{m-1}(\boldsymbol{x}_i) + w) \tag{18.45}$$

For squared error (as used by gradient boosting), the optimal weight $\hat{w}_{jm}$ is the just the mean of the residuals in that leaf.

18.5.5.2 XGBoost

XGBoost (https://github.com/dmlc/xgboost), which stands for "extreme gradient boosting", is a very efficient and widely used implementation of gradient boosted trees, that adds a few more improvements beyond the description in Section 18.5.5.1. The details can be found in [CG16], but in brief, the extensions are as follows: it adds a regularizer on the tree complexity, it uses a second order approximation of the loss (from [FHT00]) instead of just a linear approximation, it samples features at internal nodes (as in random forests), and it uses various computer science methods (such as handling out-of-core computation for large datasets) to ensure scalability.[2]

In more detail, XGBoost optimizes the following regularized objective

$$\mathcal{L}(f) = \sum_{i=1}^{N} \ell(y_i, f(\boldsymbol{x}_i)) + \Omega(f) \tag{18.46}$$

where

$$\Omega(f) = \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2 \tag{18.47}$$

is the regularizer, where J is the number of leaves, and $\gamma \geq 0$ and $\lambda \geq 0$ are regularization coefficients. At the m'th step, the loss is given by

$$\mathcal{L}_m(F_m) = \sum_{i=1}^{N} \ell(y_i, f_{m-1}(\boldsymbol{x}_i) + F_m(\boldsymbol{x}_i)) + \Omega(F_m) + \text{const} \tag{18.48}$$

We can compute a second order Taylor expansion of this as follows:

$$\mathcal{L}_m(F_m) \approx \sum_{i=1}^{N} \left[\ell(y_i, f_{m-1}(\boldsymbol{x}_i)) + g_{im} F_m(\boldsymbol{x}_i) + \frac{1}{2} h_{im} F_m^2(\boldsymbol{x}_i) \right] + \Omega(F_m) + \text{const} \tag{18.49}$$

where h_{im} is the Hessian

$$h_{im} = \left[\frac{\partial^2 \ell(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)^2} \right]_{f=f_{m-1}} \tag{18.50}$$

In the case of regression trees, we have $F(\boldsymbol{x}) = w_{q(\boldsymbol{x})}$, where $q : \mathbb{R}^D \to \{1, \ldots, J\}$ specifies which leaf node $\boldsymbol{x}$ belongs to, and $\boldsymbol{w} \in \mathbb{R}^J$ are the leaf weights. Hence we can rewrite Equation (18.49) as

2. Some other popular gradient boosted trees packages are **CatBoost** (https://catboost.ai/) and **LightGBM** (https://github.com/Microsoft/LightGBM).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

follows, dropping terms that are independent of F_m:

$$\mathcal{L}_m(q, \boldsymbol{w}) \approx \sum_{i=1}^{N} \left[g_{im} F_m(\boldsymbol{x}_i) + \frac{1}{2} h_{im} F_m^2(\boldsymbol{x}_i) \right] + \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2 \tag{18.51}$$

$$= \sum_{j=1}^{J} \left[(\sum_{i \in I_j} g_{im}) w_j + \frac{1}{2} (\sum_{i \in I_j} h_i + \lambda) w_j^2 \right] + \gamma J \tag{18.52}$$

where $I_j = \{i : q(\boldsymbol{x}_i) = j\}$ is the set of indices of data points assigned to the j'th leaf.

Let us define $G_{jm} = \sum_{i \in I_j} g_{im}$ and $H_{jm} = \sum_{i \in I_j} h_{im}$. Then the above simplifies to

$$\mathcal{L}_m(q, \boldsymbol{w}) = \sum_{j=1}^{J} \left[G_{jm} w_j + \frac{1}{2} (H_{jm} + \lambda) w_j^2 \right] + \gamma J \tag{18.53}$$

This is a quadratic in each w_j, so the optimal weights are given by

$$w_j^* = -\frac{G_{jm}}{H_{jm} + \lambda} \tag{18.54}$$

The loss for evaluating different tree structures q then becomes

$$\mathcal{L}_m(q, \boldsymbol{w}^*) = -\frac{1}{2} \sum_{j=1}^{J} \frac{G_{jm}^2}{H_{jm} + \lambda} + \gamma J \tag{18.55}$$

We can greedily optimize this using a recursive node splitting procedure, as in Section 18.1. Specifically, for a given leaf j, we consider splitting it into a left and right half, $I = I_L \cup I_R$. We can compute the gain (reduction in loss) of such a split as follows:

$$\text{gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{(H_L + H_R) + \lambda} \right] - \gamma \tag{18.56}$$

where $G_L = \sum_{i \in I_L} g_{im}$, $G_R = \sum_{i \in I_R} g_{im}$, $H_L = \sum_{i \in I_L} h_{im}$, and $H_R = \sum_{i \in I_R} h_{im}$. Thus we see that it is not worth splitting a node if the gain is negative (i.e., the first term is less than γ).

A fast approximation for evaluating this objective, that does not require sorting the features (for choosing the optimal threshold to split on), is described in [CG16].

18.6 Interpreting tree ensembles

Trees are popular because they are interpretable. Unfortunately, ensembles of trees (whether in the form of bagging, random forests, or boosting) lose that property. Fortunately, there are some simple methods we can use to interpret what function has been learned.

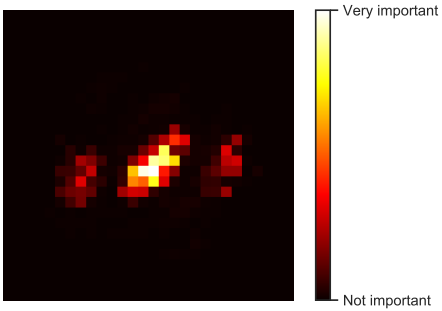


Figure 18.8: Feature importance of a random forest classifier trained to distinguish MNIST digits from classes 0 and 8. Adapted from Figure 7.6 of [Gér19]. Generated by rf_feature_importance_mnist.ipynb.

18.6.1 Feature importance

For a single decision tree T , [BFO84] proposed the following measure for **feature importance** of feature k:

$$R_k(T) = \sum_{j=1}^{J-1} G_j \mathbb{I}\left(v_j = k\right) \tag{18.57}$$

where the sum is over all non-leaf (internal) nodes, G_j is the gain in accuracy (reduction in cost) at node j, and $v_j = k$ if node j uses feature k. We can get a more reliable estimate by averaging over all trees in the ensemble:

$$R_k = \frac{1}{M} \sum_{m=1}^{M} R_k(T_m) \tag{18.58}$$

After computing these scores, we can normalize them so the largest value is 100%. We give some examples below.

Figure 18.8 gives an example of estimating feature importance for a classifier trained to distinguish MNIST digits from classes 0 and 8. We see that it focuses on the parts of the image that differ between these classes.

In Figure 18.9, we plot the relative importance of each of the features for the spam dataset (Section 18.4). Not surprisingly, we find that the most important features are the words "george" (the name of the recipient) and "hp" (the company he worked for), as well as the characters ! and $. (Note it can be the presence or absence of these features that is informative.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

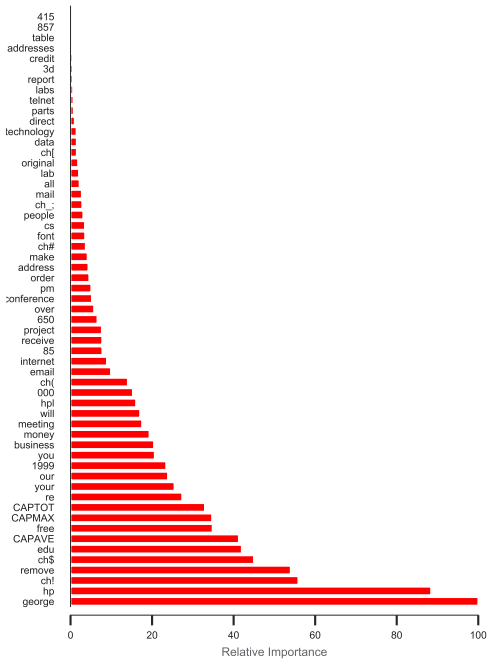
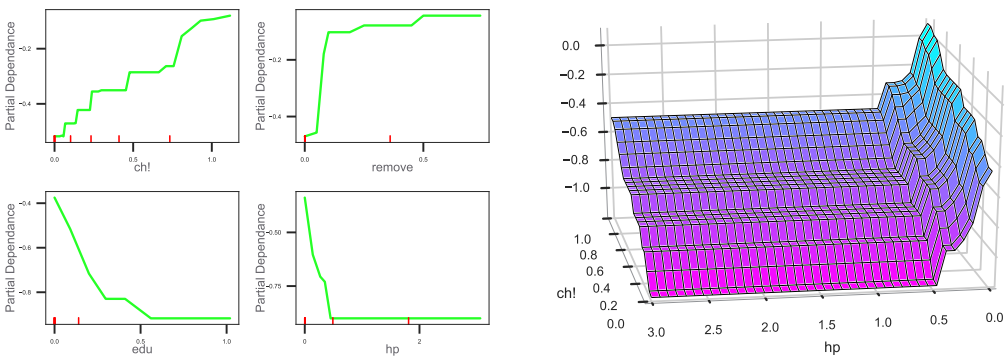


Figure 18.9: Feature importance of a gradient boosted classifier trained to distinguish spam from non-spam email. The dataset has X training examples with Y features, corresponding to token frequency. Adapted from Figure 10.6 of [HTF09]. Generated by spam_tree_ensemble_interpret.ipynb.



(a) *(b)*

Figure 18.10: (a) Partial dependence of log-odds of the spam class for 4 important predictors. The red ticks at the base of the plot are deciles of the empirical distribution for this feature. (b) Joint partial dependence of log-odds on the features hp and !. Adapted from Figure 10.6–10.8 of [HTF09]. Generated by spam_tree_ensemble_interpret.ipynb.

18.6.2 Partial dependency plots

After we have identified the most relevant input features, we can try to assess the impact they have on the output. A **partial dependency plot** for feature k is a plot of

$$\overline{f}_k(x_k) = \frac{1}{N}\sum_{n=1}^{N} f(\boldsymbol{x}_{n,-k}, x_k) \tag{18.59}$$

vs x_k. Thus we marginalize out all features except k. In the case of a binary classifier, we can convert this to log odds, $\log p(y=1|x_k)/p(y=0|x_k)$, before plotting. We illustrate this for our spam example in Figure 18.10a for 4 different features. We see that as the frequency of ! and "remove" increases, so does the probability of spam. Conversely, as the frequency of "edu" or "hp" increases, the probability of spam decreases.

We can also try to capture interaction effects between features j and k by computing

$$\overline{f}_{jk}(x_j, x_k) = \frac{1}{N}\sum_{n=1}^{N} f(\boldsymbol{x}_{n,-jk}, x_j, x_k) \tag{18.60}$$

We illustrate this for our spam example in Figure 18.10b for hp and !. We see that higher frequency of ! makes it more likely to be spam, but much more so if the word "hp" is missing.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Part V

Beyond Supervised Learning

19 Learning with Fewer Labeled Examples

Many ML models, especially neural networks, often have many more parameters than we have labeled training examples. For example, a ResNet CNN (Section 14.3.4) with 50 layers has 23 million parameters. Transformer models (Section 15.5) can be even bigger. Of course these parameters are highly correlated, so they are not independent "degrees of freedom". Nevertheless, such big models are slow to train and, more importantly, they may easily overfit. This is particularly a problem when you do not have a large labeled training set. In this chapter, we discuss some ways to tackle this issue, beyond the generic regularization techniques we discussed in Section 13.5 such as early stopping, weight decay and dropout.

19.1 Data augmentation

Suppose we just have a single small labeled dataset. In some cases, we may be able to create artificially modified versions of the input vectors, which capture the kinds of variations we expect to see at test time, while keeping the original labels unchanged. This is called **data augmentation**.[1] We give some examples below, and then discuss why this approach works.

19.1.1 Examples

For image classification tasks, standard data augmentation methods include random crops, zooms, and mirror image flips, as illustrated in Figure 19.1. [GVZ16] gives a more sophisticated example, where they render text characters onto an image in a realistic way, thereby creating a very large dataset of text "in the wild". They used this to train a state of the art visual text localization and reading system. Other examples of data augmentation include artifically adding background noise to clean speech signals, and artificially replacing characters or words at random in text documents.

If we afford to train and test the model many times using different versions of the data, we can learn which augmentations work best, using blackbox optimization methods such as RL (see e.g., [Cub+19]) or Bayesian optimization (see e.g., [Lim+19]); this is called **AutoAugment**. We can also learn to combine multiple augmentations together; this is called **AutoAugment** [Cub+19].

For some examples of augmentation in NLP, see e.g., [Fen+21].

1. The term "data augmentation" is also used in statistics to mean the addition of auxiliary latent variables to a model in order to speed up convergence of posterior inference algorithms [DM01].



Figure 19.1: Illustration of random crops and zooms of the image on the left. Generated by image_augmentation_jax.ipynb.

19.1.2 Theoretical justification

Data augmentation often significantly improves performance (predictive accuracy, robustness, etc). At first this might seem like we are getting something for nothing, since we have not provided additional data. However, the data augmentation mechanism can be viewed as a way to algorithmically inject prior knowledge.

To see this, recall that in standard ERM training, we minimize the empirical risk

$$R(f) = \int \ell(f(\boldsymbol{x}), \boldsymbol{y}) p^*(\boldsymbol{x}, \boldsymbol{y}) d\boldsymbol{x} d\boldsymbol{y} \tag{19.1}$$

where we approximate $p^*(\boldsymbol{x}, \boldsymbol{y})$ by the empirical distribution

$$p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{N} \sum_{n=1}^{N} \delta(\boldsymbol{x} - \boldsymbol{x}_n) \delta(\boldsymbol{y} - \boldsymbol{y}_n) \tag{19.2}$$

We can think of data augmentation as replacing the empirical distribution with the following algorithmically smoothed distribution

$$p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y}|A) = \frac{1}{N} \sum_{n=1}^{N} p(\boldsymbol{x}|\boldsymbol{x}_n, A) \delta(\boldsymbol{y} - \boldsymbol{y}_n) \tag{19.3}$$

where A is the data augmentation algorithm, which generates a sample $\boldsymbol{x}$ from a training point $\boldsymbol{x}_n$, such that the label ("semantics") is not changed. (A very simple example would be a Gaussian kernel, $p(\boldsymbol{x}|\boldsymbol{x}_n, A) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{x}_n, \sigma^2 \mathbf{I})$.) This has been called **vicinal risk minimization** [Cha+01], since we are minimizing the risk in the vicinity of each training point $\boldsymbol{x}$. For more details on this perspective, see [Zha+17b; CDL19; Dao+19].

19.2 Transfer learning

This section is coauthored with Colin Raffel.

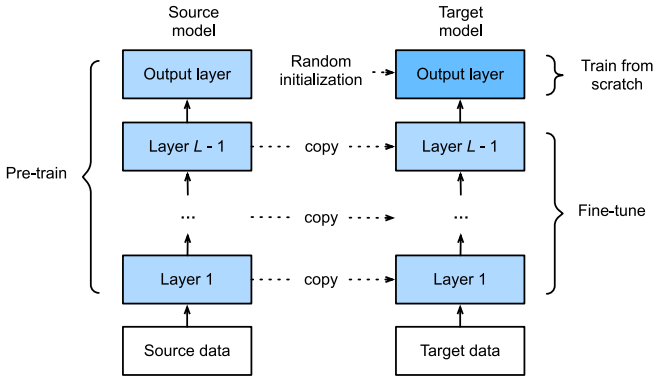


Figure 19.2: Illustration of fine-tuning a model on a new dataset. The final output layer is trained from scratch, since it might correspond to a different label set. The other layers are initialized at their previous parameters, and then optionally updated using a small learning rate. From Figure 13.2.1 of [Zha+20]. Used with kind permission of Aston Zhang.

Many data-poor tasks have some high-level structural similarity to other data-rich tasks. For example, consider the task of **fine-grained visual classification** of endangered bird species. Given that endangered birds are by definition rare, it is unlikely that a large quantity of diverse labeled images of these birds exist. However, birds bear many structural similarities across species - for example, most birds have wings, feathers, beaks, claws, etc. We therefore might expect that first training a model on a large dataset of non-endangered bird species and then continuing to train it on a small dataset of endangered species could produce better performance than training on the small dataset alone.

This is called **transfer learning**, since we are transferring information from one dataset to another, via a shared set of parameters. More precisely, we first perform a **pre-training phase**, in which we train a model with parameters $\boldsymbol{\theta}$ on a large **source dataset** $\mathcal{D}_p$; this may be labeled or unlabeled. We then perform a second **fine-tuning phase** on the small labeled **target dataset** $\mathcal{D}_q$ of interest. We discuss these two phases in more detail below, but for more information, see e.g., [Tan+18; Zhu+21] for recent surveys.

19.2.1 Fine-tuning

Suppose, for now, that we already have a pretrained classifier, $p(y|\boldsymbol{x}, \boldsymbol{\theta}_p)$, such as a CNN, that works well for inputs $\boldsymbol{x} \in \mathcal{X}_p$ (e.g. natural images) and outputs $y \in \mathcal{Y}_p$ (e.g., ImageNet labels), where the data comes from a distribution $p(\boldsymbol{x}, y)$ similar to the one used in training. Now we want to create a new model $q(y|\boldsymbol{x}, \boldsymbol{\theta}_q)$ that works well for inputs $\boldsymbol{x} \in \mathcal{X}_q$ (e.g. bird images) and outputs $y \in \mathcal{Y}_q$ (e.g., fine-grained bird labels), where the data comes from a distribution $q(\boldsymbol{x}, y)$ which may be different from p.

We will assume that the set of possible inputs is the same, so $\mathcal{X}_q \approx \mathcal{X}_p$ (e.g., both are RGB images), or that we can easily transform inputs from domain p to domain q (e.g., we can convert an RGB image to grayscale by dropping the chrominance channels and just keeping luminance). (If this is not

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

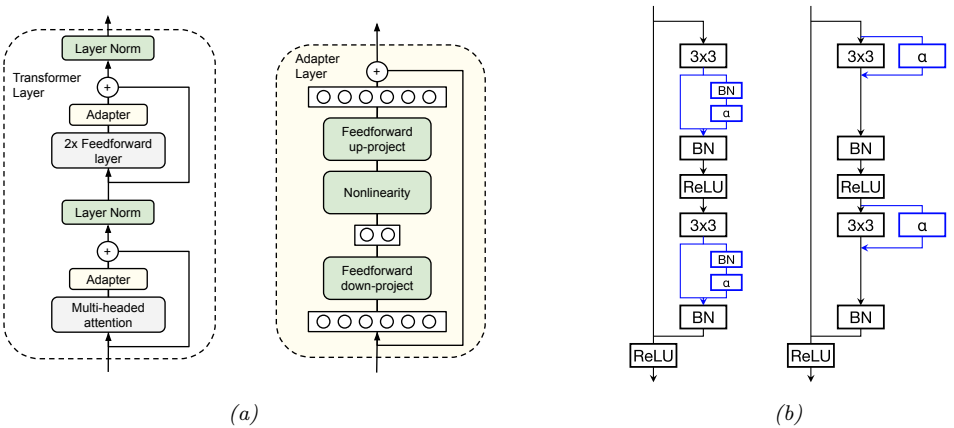


Figure 19.3: (a) Adding adapter layers to a transformer. From Figure 2 of [Hou+19]. Used with kind permission of Neil Houlsby. (b) Adding adapter layers to a resnet. From Figure 2 of [RBV18]. Used with kind permission of Sylvestre-Alvise Rebuffi.

the case, then we may need to use a method called domain adaptation, that modifies models to map between modalities, as discussed in Section 19.2.5.)

However, the output domains are usually different, i.e., $\mathcal{Y}_q \neq \mathcal{Y}_p$. For example, $\mathcal{Y}_p$ might be Imagenet labels and $\mathcal{Y}_q$ might be medical labels (e.g., types of diabetic retinopathy [Arc+19]). In this case, we need to "translate" the output of the pre-trained model to the new domain. This is easy to do with neural networks: we simply "chop off" the final layer of the original model, and add a new "head" to model the new class labels, as illustrated in Figure 19.2. For example, suppose $p(y|\boldsymbol{x}, \boldsymbol{\theta}_p) = \text{softmax}(y|\mathbf{W}_2\boldsymbol{h}(\boldsymbol{x}; \boldsymbol{\theta}_1) + \boldsymbol{b}_2)$, where $\boldsymbol{\theta}_p = (\mathbf{W}_2, \boldsymbol{b}_2, \boldsymbol{\theta}_1)$. Then we can construct $q(y|\boldsymbol{\theta}_q) = \text{softmax}(y|\mathbf{W}_3\boldsymbol{h}(\boldsymbol{x}; \boldsymbol{\theta}_1) + \boldsymbol{b}_3)$, where $\boldsymbol{\theta}_q = (\mathbf{W}_3, \boldsymbol{b}_3, \boldsymbol{\theta}_1)$ and $\boldsymbol{h}(\boldsymbol{x}; \boldsymbol{\theta}_1)$ is the shared nonlinear feature extractor.

After performing this "model surgery", we can fine-tune the new model with parameters $\boldsymbol{\theta}_q = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_3)$, where $\boldsymbol{\theta}_1$ parameterizes the feature extractor, and $\boldsymbol{\theta}_3$ parameterizes the final linear layer that maps features to the new set of labels. If we treat $\boldsymbol{\theta}_1$ as "**frozen parameters**", then the resulting model $q(y|\boldsymbol{x}, \boldsymbol{\theta}_q)$ is linear in its parameters, so we have a convex optimization problem for which many simple and efficient fitting methods exist (see Part II). This is particularly helpful in the long-tail setting, where some classes are very rare [Kan+20]. However, a linear "decoder" may be too limiting, so we can also allow $\boldsymbol{\theta}_1$ to be fine-tuned as well, but using a lower learning rate, to prevent the values moving too far from the values estimated on $\mathcal{D}_p$.

19.2.2 Adapters

One disadvantage of fine-tuning all the model parameters of a pre-trained model is that it can be slow, since there are often many parameters, and we may need to use a small learning rate to prevent the low-level feature extractors from diverging too far from their prior values. In addition, every new task requires a new model to be trained, making task sharing hard. An alternative approach is to keep the pre-trained model untouched, but to add new parameters to modify its internal behavior to

customize the feature extraction process for each task. This idea is called **adapters**, and has been explored in several papers (e.g., [RBV17; RBV18; Hou+19]).

Figure 19.3a illustrates adapters for transformer networks (Section 15.5), as proposed in [Hou+19]. The basic idea is to insert two shallow bottleneck MLPs inside each transformer layer, one after the multi-head attention and once after the feed-forward layers. Note that these MLPs have skip connections, so that they can be initialized to implement the identity mapping. If the transformer layer has features of dimensionality D, and the adapter uses a bottleneck of size M, this introduces $O(DM)$ new parameters per layer. These adapter MLPs, as well as the layer norm parameters and final output head, are trained for each new task, but the all remaining parameters are frozen. Empirically on several NLP benchmarks, this is found to give better performance than fine tuning, while only needing about 1-10% of the original parameters.

Figure 19.3b illustrates adapters for residual networks (Section 14.3.4), as proposed in [RBV17; RBV18]. The basic idea is to add a 1x1 convolution layer $\boldsymbol{\alpha}$, which is analogous to the MLP adapter in the transformer case, to the internal layers of the CNN. This can be added in series or in parallel, as shown in the diagram. If we denote the adapter layer by $\rho(\boldsymbol{x})$, we can define the series adapter to be

$$\rho(\boldsymbol{x}) = \boldsymbol{x} + \text{diag}_1(\boldsymbol{\alpha}) \circledast \boldsymbol{x} = \text{diag}_1(\mathbf{I} + \boldsymbol{\alpha}) \circledast \boldsymbol{x} \tag{19.4}$$

where $\text{diag}_1(\boldsymbol{\alpha}) \in \mathbb{R}^{1\times 1\times C\times D}$ reshapes a matrix $\boldsymbol{\alpha} \in \mathbb{R}^{C\times D}$ into a matrix that can be applied to each spatial location in parallel. (We have omitted batch normalization for simplicity.) If we insert this after a regular convolution layer $\boldsymbol{f} \circledast \boldsymbol{x}$ we get

$$\boldsymbol{y} = \rho(\boldsymbol{f} \circledast \boldsymbol{x}) = (\text{diag}_1(\mathbf{I} + \boldsymbol{\alpha}) \circledast \boldsymbol{f}) \circledast \boldsymbol{x} \tag{19.5}$$

This can be interpreted as a low-rank multiplicative perturbation to the original filter $\boldsymbol{f}$. The parallel adapter can be defined by

$$\boldsymbol{y} = \boldsymbol{f} \circledast \boldsymbol{x} + \text{diag}_1(\boldsymbol{\alpha}) \circledast \boldsymbol{x} = (\boldsymbol{f} + \text{diag}_L(\boldsymbol{\alpha})) \circledast \boldsymbol{x} \tag{19.6}$$

This can be interpreted as a low-rank additive perturbation to the original filter $\boldsymbol{f}$. In both cases, setting $\boldsymbol{\alpha} = \mathbf{0}$ ensures the adapter layers can be initialized to the identity transformation. In addition, both methods required $O(C^2)$ parameters per layer.

19.2.3 Supervised pre-training

The pre-training task may be supervised or unsupervised; the main requirements are that it can teach the model basic structure about the problem domain and that it is sufficiently similar to the downstream fine-tuning task. The notion of task similarity is not rigorously defined, but in practice the domain of the pre-training task is often more broad than that of the fine-tuning task (e.g., pre-train on all bird species and fine-tune on endangered ones).

The most straightforward form of transfer learning is the case where a large labeled dataset is suitable for pre-training. For example, it is very common to use the ImageNet dataset (Section 1.5.1.2) to pretrain CNNs, which can then be used for a variety of downstream tasks and datasets (see e.g., [Kol+19]). Imagenet has 1.28 million natural images, each associated with a label from one of 1,000 classes. The classes constitute a wide variety of different concepts, including animals, foods, buildings, musical instruments, clothing, and so on. The images themselves are diverse in the sense

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

that they contain objects from many angles and in many sizes with a wide variety of backgrounds. This diversity and scale may partially explain why it has become a de-facto pre-training task for transfer learning in computer vision. (See finetune_cnn_jax.ipynb for some example code.)

However, Imagenet pre-training has been shown to be less helpful when the domain of the fine-tuning task is quite different from natural images (e.g. medical images [Rag+19]). And in some cases where it is helpful (e.g., training object detection systems), it seems to be more of a speedup trick (by warm-starting optimization at a good point) rather than something that is essential, in the sense that one can achieve comparable performance on the downstream task when training from scratch, if done for long enough [HGD19].

Supervised pre-training is somewhat less common in non-vision applications. One notable exception is to pre-train on natural language inference data (i.e. whether a sentence implies or contradicts another) to learn vector representations of sentences [Con+17], though this approach has largely been supplanted by unsupervised methods (Section 19.2.4). Another non-vision application of transfer learning is to pre-train a speech recognition on a large English-labeled corpus before fine-tuning on low-resource languages [Ard+20].

19.2.4 Unsupervised pre-training (self-supervised learning)

It is increasingly common to use **unsupervised pre-training**, because unlabeled data is often easy to acquire, e.g., unlabeled images or text documents from the web.

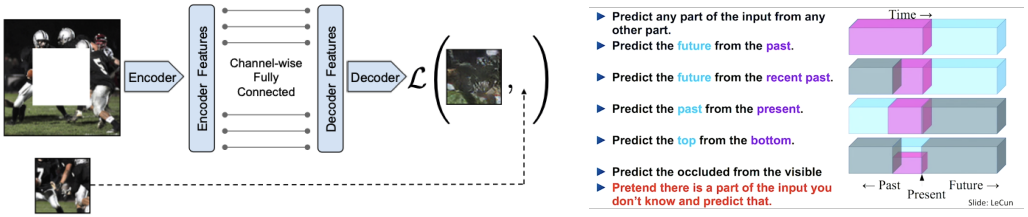
For a short period of time it was common to pre-train deep neural networks using an unsupervised objective (e.g., reconstruction error, as discussed in Section 20.3) over the labeled dataset (i.e. ignoring the labels) before proceeding with standard supervised training [HOT06; Vin+10b; Erh+10]. While this technique is also called unsupervised pre-training, it differs from the form of pre-training for transfer learning we discuss in this section, which uses a (large) unlabeled dataset for pre-training before fine-tuning on a different (smaller) labeled dataset.

Pre-training tasks that use unlabeled data are often called **self-supervised** rather than unsupervised. This term is used because the labels are created by the algorithm, rather than being provided externally by a human, as in standard supervised learning. Both supervised and self-supervised learning are discriminative tasks, since they require predicting outputs given inputs. By contrast, other unsupervised approaches, such as some of those discussed in Chapter 20, are generative, since they predict outputs unconditionally.

There are many different self-supervised learning heuristics that have been tried (see e.g., [GR18; JT19; Ren19] for a review, and https://github.com/jason718/awesome-self-supervised-learning for an extensive list of papers). We can identify at least three main broad groups, which we discuss below.

19.2.4.1 Imputation tasks

One approach to self-supervised learning is to solve **imputation tasks**. In this approach, we partition the input vector $\boldsymbol{x}$ into two parts, $\boldsymbol{x} = (\boldsymbol{x}_h, \boldsymbol{x}_v)$, and then try to predict the hidden part $\boldsymbol{x}_h$ given the remaining visible part, $\boldsymbol{x}_v$, using a model of the form $\hat{\boldsymbol{x}}_h = f(\boldsymbol{x}_v, \boldsymbol{x}_h = \mathbf{0})$. We can think of this as a "**fill-in-the-blank**" task; in the NLP community, this is called a **cloze task**. See Figure 19.4 for some visual examples, and Section 15.7.1.2 for some NLP examples.



(a) (b)

Figure 19.4: (a) Context encoder for self-supervised learning. From [Pat+16]. Used with kind permission of Deepak Pathak. (b) Some other proxy tasks for self-supervised learning. From [LeC18]. Used with kind permission of Yann LeCun.

19.2.4.2 Proxy tasks

Another approach to SSL is to solve **proxy tasks**, also called **pretext tasks**. In this setup, we create pairs of inputs, $(\boldsymbol{x}_1, \boldsymbol{x}_2)$, and then train a Siamese network classifier (Figure 16.5a) of the form $p(y|\boldsymbol{x}_1, \boldsymbol{x}_2) = p(y|r[f(\boldsymbol{x}_1), f(\boldsymbol{x}_2)])$, where $f(\boldsymbol{x})$ is some function that performs "**representation learning**" [BCV13], and y is some label that captures the relationship between $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, which is predicted by $r(\boldsymbol{f}_1, \boldsymbol{f}_2)$. For example, suppose $\boldsymbol{x}_1$ is an image patch, and $\boldsymbol{x}_2 = t(\boldsymbol{x}_1)$ is some transformation of $\boldsymbol{x}_1$ that we control, such as a random rotation; then we define y to be the rotation angle that we used [GSK18].

19.2.4.3 Contrastive tasks

The currently most popular approach to self-supervised learning is to use various kinds of **contrastive tasks**. The basic idea is to create pairs of examples that are semantically similar to each other, using data augmentation methods (Section 19.1), and then to ensure that the distance between their representations is closer (in embedding space) than the distance between two unrelated examples. This is exactly the same idea that is used in deep metric learning (Section 16.2.2) — the only difference is that the algorithm creates its own similar pairs, rather than relying on an externally provided measure of similarity, such as labels. We give some examples of this in Section 19.2.4.4 and Section 19.2.4.5.

19.2.4.4 SimCLR

In this section, we discuss **SimCLR**, which stands for "Simple contrastive learning of visual representations" [Che+20b; Che+20c]. This has shown state of the art performance on transfer learning and semi-supervised learning. The basic idea is as follows. Each input $\boldsymbol{x} \in \mathbb{R}^D$ is converted to two augmented "views' $\boldsymbol{x}_1 = t_1(\boldsymbol{x})$, $\boldsymbol{x}_2 = t_2(\boldsymbol{x})$, which are "semantically equivalent" versions of the input generated by some transformations t_1, t_2. For example, if $\boldsymbol{x}$ is an image, these could be small perturbations to the image, such as random crops, as discussed in Section 19.1. In addition, we sample "negative" examples $\boldsymbol{x}_1^-, \ldots, \boldsymbol{x}_n^- \in N(\boldsymbol{x})$ from the dataset which represent "semantically different" images (in practice, these are the other examples in the minibatch). Next we define some feature mapping $F : \mathbb{R}^D \to \mathbb{R}^E$, where D is the size of the input, and E is the size of the embedding.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

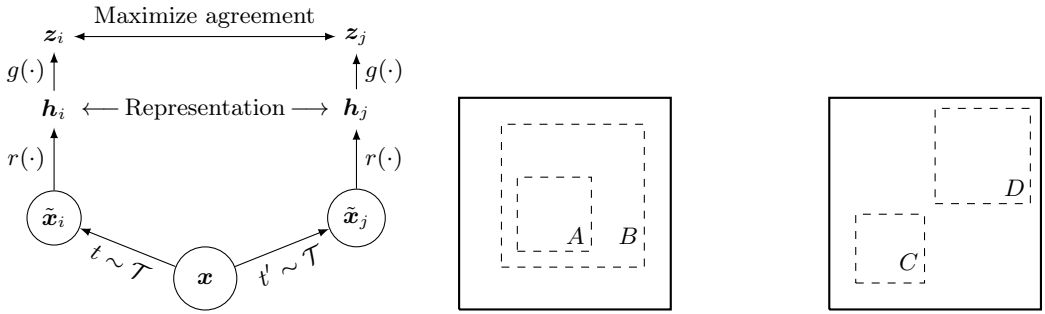


Figure 19.5: (a) Illustration of SimCLR training. $\mathcal{T}$ is a set of stochastic semantics-preserving transformations (data augmentations). (b-c) Illustration of the benefit of random crops. Solid rectangles represent the original image, dashed rectangles are random crops. In (b), the model is forced to predict the local view A from the global view B (and vice versa). In (c), the model is forced to predict the appearance of adjacent views (C,D). From Figures 2–3 of [Che+20b]. Used with kind permission of Ting Chen.

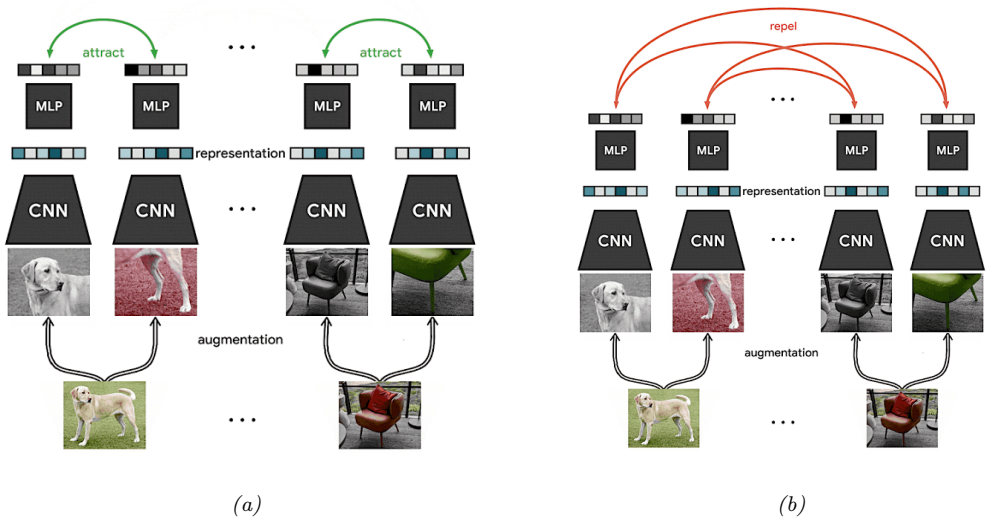


Figure 19.6: Visualization of SimCLR training. Each input image in the minibatch is randomly modified in two different ways (using cropping (followed by resize), flipping, and color distortion), and then fed into a Siamese network. The embeddings (final layer) for each pair derived from the same image is forced to be close, whereas the embeddings for all other pairs are forced to be far. From https://ai.googleblog.com/2020/04/advancing-self-supervised-and-semi.html. Used with kind permission of Ting Chen.

We then try to maximize the similarity of the similar views, while minimizing the similarity of the different views, for each input $\boldsymbol{x}$:

$$J = F(t_1(\boldsymbol{x}))^\mathsf{T} F(t_2(\boldsymbol{x})) - \log \sum_{\boldsymbol{x}_i^- \in N(\boldsymbol{x})} \exp\left[F(\boldsymbol{x}_i^-)^\mathsf{T} F(t_1(\boldsymbol{x}))\right] \tag{19.7}$$

In practice, we use cosine similarity, so we ℓ_2-normalize the representations produced by F before taking inner products, but this is omitted in the above equation. See Figure 19.5a for an illustration. (In this figure, we assume $F(\boldsymbol{x}) = g(r(\boldsymbol{x}))$, where the intermediate representation $\boldsymbol{h} = r(\boldsymbol{x})$ is the one that will be later used for fine-tuning, and g is an additional transformation applied during training.)

Interestingly, we can interpret this as a form of conditional **energy based model** of the form

$$p(\boldsymbol{x}_2|\boldsymbol{x}_1) = \frac{\exp[-\mathcal{E}(\boldsymbol{x}_2|\boldsymbol{x}_1)]}{Z(\boldsymbol{x}_1)} \tag{19.8}$$

where $\mathcal{E}(\boldsymbol{x}_2|\boldsymbol{x}_1) = -F(\boldsymbol{x}_2)^\mathsf{T} F(\boldsymbol{x}_1)$ is the energy, and

$$Z(\boldsymbol{x}) = \int \exp[-\mathcal{E}(\boldsymbol{x}^-|\boldsymbol{x})] d\boldsymbol{x}^- = \int \exp[F(\boldsymbol{x}^-)^\mathsf{T} F(\boldsymbol{x})] d\boldsymbol{x}^- \tag{19.9}$$

is the normalization constant, known as the **partition function**. The conditional log likelihood under this model has the form

$$\log p(\boldsymbol{x}_2|\boldsymbol{x}_1) = F(\boldsymbol{x}_2)^\mathsf{T} F(\boldsymbol{x}_1) - \log \int \exp[F(\boldsymbol{x}^-)^\mathsf{T} F(\boldsymbol{x}_1)] d\boldsymbol{x}^- \tag{19.10}$$

The only difference from Equation (19.7) is that we replace the integral with a Monte Carlo upper bound derived from the negative samples. Thus we can think of contrastive learning as approximate maximum likelihood estimation of a conditional energy based generative model [Gra+20]. More details on such models can be found in the sequel to this book, [Mur23].

A critical ingredient to the success of SimCLR is the choice of data augmentation methods. By using random cropping, they can force the model to predict local views from global views, as well as to predict adjacent views of the same image (see Figure 19.5). After cropping, all images are resized back to the same size. In addition, they randomly flip the image some fraction of the time.[2]

SimCLR relies on large batch training, in order to ensure a sufficiently diverse set of negatives. When this is not possible, we can use a memory bank of past (negative) embeddings, which can be updated using exponential moving averaging (Section 4.4.2.2). This is known as **momentum contrastive learning** or **MoCo** [He+20].

19.2.4.5 CLIP

In this section, we describe **CLIP**, which stands for "Contrastive Language-Image Pre-training" [Rad+]. This is a contrastive approach to representation learning which uses a massive corpus of

2. It turns out that distinguishing positive crops (from the same image) from negative crops (from different images) is often easy to do just based on color histograms. To prevent this kind of "cheating", they also apply a random color distortion, thus cutting off this "short circuit". The combination of random cropping and color distortion is found to work better than either method alone.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

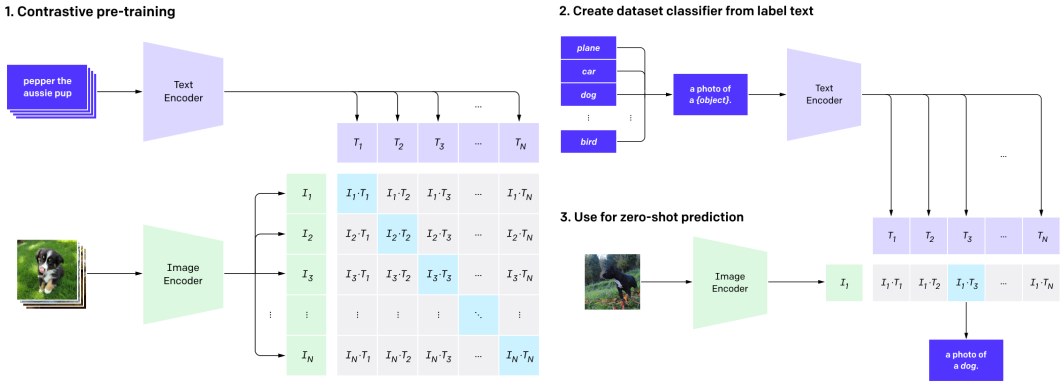


Figure 19.7: Illustration of the CLIP model. From Figure 1 of [Rad+]. Used with kind permission of Alec Radford.

400M (image, text) pairs extracted from the web. Let $\boldsymbol{x}_i$ be the i'th image and $\boldsymbol{y}_i$ be its matching text. Rather than trying to predict the exact words associated with the image, it is simpler to just determine if $\boldsymbol{y}_i$ is more likely to be the correct text compared to $\boldsymbol{y}_j$, for some other text string j in the minibatch. Similarly, the model can try to determine if image $\boldsymbol{x}_i$ is more likely to be matched than $\boldsymbol{x}_j$ to a given text $\boldsymbol{y}_i$.

More precisely, let $\boldsymbol{f}_I(\boldsymbol{x}_i)$ be the embedding of the image, $\boldsymbol{f}_T(\boldsymbol{y}_j)$ be the embedding of the text, $\mathbf{I}_i = \boldsymbol{f}_I(\boldsymbol{x}_i)/||\boldsymbol{f}_I(\boldsymbol{x}_i)||_2$ be the unit-norm version of the image embedding, and $\mathbf{T}_j = \boldsymbol{f}_T(\boldsymbol{y}_j)/||\boldsymbol{f}_T(\boldsymbol{y}_j)||_2$ be the unit-norm version of the text embedding. Define the vector of pairwise logits (similarity scores) to be

$$L_{ij} = \mathbf{I}_i^{\mathsf{T}} \mathbf{T}_j \tag{19.11}$$

We now train the parameters of the two embedding functions $\boldsymbol{f}_I$ and $\boldsymbol{f}_T$ to minimize the following loss, averaged over minibatches of size N:

$$J = \frac{1}{2}\left[\sum_{i=1}^{N} \mathrm{CE}(\mathbf{L}_{i,:}, \mathbf{1}_i) + \sum_{j=1}^{N} \mathrm{CE}(\mathbf{L}_{:,j}, \mathbf{1}_j)\right] \tag{19.12}$$

where CE is the cross entropy loss

$$\mathrm{CE}(\boldsymbol{p}, \boldsymbol{q}) = -\sum_{k=1}^{K} p_k \log q_k \tag{19.13}$$

and $\mathbf{1}_i$ is a one-hot encoding of label i. See Figure 19.7a for an illustration. (In practice, the normalized embeddings are scaled by a temperature parameter which is also learned; this controls the sharpness of the softmax.)

In their paper, they considered using a ResNet (Section 14.3.4) and a vision transformer (Section 15.5.6) for the function $\boldsymbol{f}_I$, and a text transformer (Section 15.5) for $\boldsymbol{f}_T$. They used a very large minibatch of $N \sim 32k$, and trained for many days on 100s of GPUs.

After the model is trained, it can be used for **zero-shot classification** of an image $\boldsymbol{x}$ as follows. First each of the K possible class labels for a given dataset is converted into a text string $\boldsymbol{y}_k$ that might occur on the web. For example, "dog" becomes "a photo of a dog". Second, we compute the normalized embeddings $\mathbf{I} \propto \boldsymbol{f}_I(\boldsymbol{x})$ and $\mathbf{T}_k \propto \boldsymbol{f}_T(\boldsymbol{y}_k)$. Third, we compute the softmax probabilites

$$p(y = k|\boldsymbol{x}) = \text{softmax}([\mathbf{I}^\intercal \mathbf{T}_1, \ldots, \mathbf{I}^\intercal \mathbf{T}_k])_k \tag{19.14}$$

See Figure 19.7b for an illustration. (A similar approach was adopted in the visual n-grams paper [Li+17].)

Remarkably, this approach can perform as well as standard supervised learning on tasks such as ImageNet classification, without ever being explicitly trained on specific labeled datasets. Of course, the images in ImageNet come from the web, and were found using text-based web-search, so the model has seen similar data before. Nevertheless, its generalization to new tasks, and robustness to distribution shift, are quite impressive (see the paper for examples).

One drawback of the approach, however, is that it is sensitive to how class labels are converted to textual form. For example, to make the model work on food classification, it is necessary to use text strings of the form "a photo of guacamole, a type of food", "a photo of ceviche, a type of food", etc. Disambiguating phrases such as "a type of food" are currently added by hand, on a per-dataset basis. This is called **prompt engineering**, and is needed since the raw class names can be ambiguous across (and sometimes within) a dataset.

19.2.5 Domain adaptation

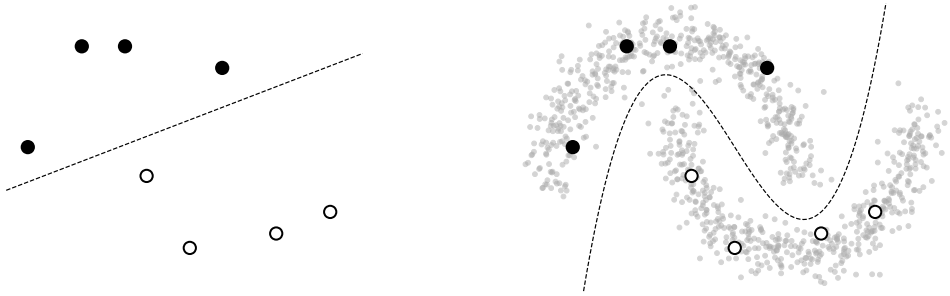
Consider a problem in which we have inputs from different domains, such as a **source domain** $\mathcal{X}_s$ and **target domain** $\mathcal{X}_t$, but a common set of output labels, $\mathcal{Y}$. (This is the "dual" of transfer learning, since the input domains are different, but the output domains the same.) For example, the domains might be images from a computer graphics system and real images, or product reviews and movie reviews. We assume we do not have labeled examples from the target domain. Our goal is to fit the model on the source domain, and then modify its parameters so it works on the target domain. This is called (unsupervised) **domain adaptation** (see e.g., [KL21] for a review).

A common approach to this problem is to train the source classifier in such a way that it cannot distinguish whether the input is coming from the source or target distribution; in this case, it will only be able to use features that are common to both domains. This is called **domain adversarial learning** [Gan+16]. More formally, let $d_n \in \{s, t\}$ be a label that specifies if the data example n comes from domain s or t. We want to optimize

$$\min_{\boldsymbol{\phi}} \max_{\boldsymbol{\theta}} \frac{1}{N_s + N_t} \sum_{n \in \mathcal{D}_s, \mathcal{D}_t} \ell(d_n, f_{\boldsymbol{\theta}}(\boldsymbol{x}_n)) + \frac{1}{N_s} \sum_{m \in \mathcal{D}_s} \ell(y_m, g_{\boldsymbol{\phi}}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_m))) \tag{19.15}$$

where $N_s = |\mathcal{D}_s|$, $N_t = |\mathcal{D}_t|$, f maps $\mathcal{X}_s \cup \mathcal{X}_t \to \mathcal{H}$, and g maps $\mathcal{H} \to \mathcal{Y}_t$. The objective in Equation (19.15) minimizes the loss on the desired task of classifying y, but *maximizes* the loss on the auxiliary task of classifying the source domain d. This can be implemented by the **gradient sign reversal** trick, and is related to GANs (generative adversarial networks). See e.g., [Csu17; Wu+19] for some other approaches to domain adaptation.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 19.8: Illustration of the benefits of semi-supervised learning for a binary classification problem. Labeled points from each class are shown as black and white circles respectively. (a) Decision boundary we might learn given only labeled data. (b) Decision boundary we might learn if we also had a lot of unlabeled data points, shown as smaller grey circles.

19.3 Semi-supervised learning

This section is co-authored with Colin Raffel.

Many recent successful applications of machine learning are in the supervised learning setting, where a large dataset of labeled examples are available for training a model. However, in many practical applications it is expensive to obtain this labeled data. Consider the case of automatic speech recognition: Modern datasets contain thousands of hours of audio recordings [Pan+15; Ard+20]. The process of annotating the words spoken in a recording is many times slower than realtime, potentially resulting in a long (and costly) annotation process. To make matters worse, in some applications data must be labeled by an expert (such as a doctor in medical applications) which can further increase costs.

Semi-supervised learning can alleviate the need for labeled data by taking advantage of unlabeled data. The general goal of semi-supervised learning is to allow the model to learn the high-level structure of the data distribution from unlabeled data and only rely on the labeled data for learning the fine-grained details of a given task. Whereas in standard supervised learning we assume that we have access to samples from the joint distribution of data and labels $\boldsymbol{x}, y \sim p(\boldsymbol{x}, y)$, semi-supervised learning assumes that we additionally have access to samples from the marginal distribution of $\boldsymbol{x}$, namely $\boldsymbol{x} \sim p(\boldsymbol{x})$, as illustrated in Figure 19.8. Further, it is generally assumed that we have many more of these unlabeled samples since they are typically cheaper to obtain. Continuing the example of automatic speech recognition, it is often much cheaper to simply record people talking (which would produce unlabeled data) than it is to transcribe recorded speech. Semi-supervised learning is a good fit for the scenario where a large amount of unlabeled data has been collected and the practitioner would like to avoid having to label all of it.

19.3.1 Self-training and pseudo-labeling

An early and straightforward approach to semi-supervised learning is **self-training** [Scu65; Agr70; McL75]. The basic idea behind self-training is to use the model itself to infer predictions on unlabeled

data, and then treat these predictions as labels for subsequent training. Self-training has endured as a semi-supervised learning method because of its simplicity and general applicability; i.e. it is applicable to any model that can generate predictions for the unlabeled data. Recently, it has become common to refer to this approach as "**pseudo-labeling**" [Lee13] because the inferred labels for unlabeled data are only "pseudo-correct" in comparison with the true, ground-truth targets used in supervised learning.

Algorithmically, self-training typically follows one of the following two procedures. In the first approach, pseudo-labels are first predicted for the entire collection of unlabeled data and the model is re-trained (possibly from scratch) to convergence on the combination of the labeled and (pseudo-labeled) unlabeled data. Then, the unlabeled data is re-labeled by the model and the process repeats itself until a suitable solution is found. The second approach instead continually generates predictions on randomly-chosen batches of unlabeled data and immediately trains the model against these pseudo-labels. Both approaches are currently common in practice; the first "offline" variant has been shown to be particularly successful when leveraging giant collections of unlabeled data [Yal+19; Xie+20] whereas the "online" approach is often used as one component of more sophisticated semi-supervised learning methods [Soh+20]. Neither variant is fundamentally better than the other. Offline self-training can result in training the model on "stale" pseudo-labels, since they are only updated each time the model converges. However, online pseudo-labeling can incur larger computational costs since it involves constantly "re-labeling" unlabeled data.

Self-training can suffer from an obvious problem: If the model generates incorrect predictions for unlabeled data and then is re-trained on these incorrect predictions, it can become progressively worse and worse at the intended classification task until it eventually learns a totally invalid solution. This issue has been dubbed **confirmation bias** [TV17] because the model is continually confirming its own (incorrect) bias about the decision rule.

A common way to mitigate confirmation bias is to use a "selection metric" [RHS05] which heuristically tries to only retain pseudo-labels that are correct. For example, assuming that a model outputs probabilities for each possible class, a frequently-used selection metric is to only retain pseudo-labels whose largest class probability is above a threshold [Yar95; RHS05]. If the model's class probability estimates are well-calibrated, then this selection metric will only retain labels that are highly likely to be correct (according to the model, at least). More sophisticated selection metrics can be designed according to the problem domain.

19.3.2 Entropy minimization

Self-training has the implicit effect of encouraging the model to output low-entropy (i.e. high-confidence) predictions. This effect is most apparent in the online setting with a cross-entropy loss, where the model minimizes the following loss function $\mathcal{L}$ on unlabeled data:

$$\mathcal{L} = -\max_c \log p_\theta(y = c|\boldsymbol{x}) \tag{19.16}$$

where $p_\theta(y|\boldsymbol{x})$ is the model's class probability distribution given input $\boldsymbol{x}$. This function is minimized when the model assigns all of its class probability to a single class c^*, i.e. $p(y = c^*|\boldsymbol{x}) = 1$ and $p(y \neq c^*|\boldsymbol{x}) = 0$.

A closely-related semi-supervised learning method is **entropy minimization** [GB05], which

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

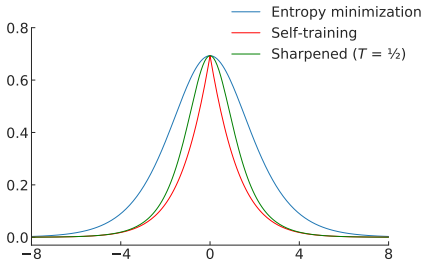


Figure 19.9: Comparison of the entropy minimization, self-training, and "sharpened" entropy minimization loss functions for a binary classification problem.

minimizes the following loss function:

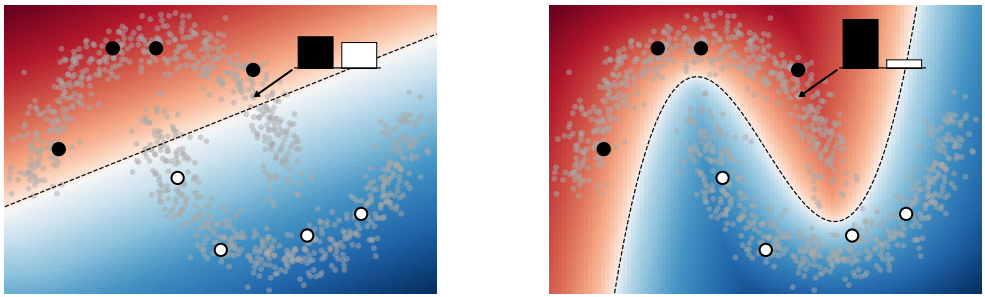
$$\mathcal{L} = -\sum_{c=1}^{C} p_\theta(y=c|\boldsymbol{x}) \log p_\theta(y=c|\boldsymbol{x}) \tag{19.17}$$

Note that this function is also minimized when the model assigns all of its class probability to a single class. We can make the entropy-minimization loss in Equation (19.17) equivalent to the online self-training loss in Equation (19.16) by replacing the first $p_\theta(y=c|\boldsymbol{x})$ term with a "one-hot" vector that assigns a probability of 1 for the class that was assigned the highest probability. In other words, online self-training minimizes the cross-entropy between the model's output and the "hard" target $\arg\max p_\theta(y|\boldsymbol{x})$, whereas entropy minimization uses the the "soft" target $p_\theta(y|\boldsymbol{x})$. One way to trade off between these two extremes is to adjust the "temperature" of the target distribution by raising each probability to the power of $1/T$ and renormalizing; this is the basis of the **mixmatch** method of [Ber+19b; Ber+19a; Xie+19]. At $T = 1$, this is equivalent to entropy minimization; as $T \to 0$, it becomes hard online self-training. A comparison of these loss functions is shown in Figure 19.9.

19.3.2.1 The cluster assumption

Why is entropy minimization a good idea? A basic assumption of many semi-supervised learning methods is that the decision boundary between classes should fall in a low-density region of the data manifold. This effectively assumes that the data corresponding to different classes are clustered together. A good decision boundary, therefore, should not pass through clusters; it should simply separate them. Semi-supervised learning methods that make the "**cluster assumption**" can be thought of as using unlabeled data to estimate the shape of the data manifold and moving the decision boundary away from it.

Entropy minimization is one such method. To see why, first assume that the decision boundary between two classes is "smooth", i.e. the model does not abruptly change its class prediction anywhere in its domain. This is true in practice for simple and/or regularized models. In this case, if the decision boundary passes through a high-density region of data, it will by necessity produce high-entropy predictions for some samples from the data distribution. Entropy minimization will therefore encourage the model to place its decision boundary in low-density regions of the input space to



(a) (b)

Figure 19.10: Visualization demonstrating how entropy minimization enforces the cluster assumption. The classifier assigns a higher probability to class 1 (black dots) or 2 (white dots) in red or blue regions respectively. The predicted class probabilities for one particular unlabeled datapoint is shown in the bar plot. In (a), the decision boundary passes through high-density regions of data, so the classifier is forced to output high-entropy predictions. In (b), the classifier avoids high-density regions and is able to assign low-entropy predictions to most of the unlabeled data.

avoid transitioning from one class to another in a region of space where data may be sampled. A visualization of this behavior is shown in Figure 19.10.

19.3.2.2 Input-output mutual information

An alternative justification for the entropy minimization objective was proposed by Bridle, Heading, and MacKay [BHM92], where it was shown that it naturally arises from maximizing the mutual information (Section 6.3) between the data and the label (i.e. the input and output of a model). Denoting $\boldsymbol{x}$ as the input and y as the target, the input-output mutual information can be written as

$$\mathcal{I}(y;\boldsymbol{x}) = \iint p(y,\boldsymbol{x}) \log \frac{p(y,\boldsymbol{x})}{p(y)p(\boldsymbol{x})} dyd\boldsymbol{x} \tag{19.18}$$

$$= \iint p(y|\boldsymbol{x})p(\boldsymbol{x}) \log \frac{p(y,\boldsymbol{x})}{p(y)p(\boldsymbol{x})} dyd\boldsymbol{x} \tag{19.19}$$

$$= \int p(\boldsymbol{x}) \int p(y|\boldsymbol{x}) \log \frac{p(y|\boldsymbol{x})}{p(y)} dyd\boldsymbol{x} \tag{19.20}$$

$$= \int p(\boldsymbol{x}) \int p(y|\boldsymbol{x}) \log \frac{p(y|\boldsymbol{x})}{\int p(\boldsymbol{x})p(y|\boldsymbol{x})d\boldsymbol{x}} dyd\boldsymbol{x} \tag{19.21}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Note that the first integral is equivalent to taking an expectation over $\boldsymbol{x}$, and the second integral is equivalent to summing over all possible values of the class y. Using these relations, we obtain

$$\mathcal{I}(y;\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{x}}\left[\sum_{i=1}^{L} p(y_i|\boldsymbol{x}) \log \frac{p(y_i|\boldsymbol{x})}{\mathbb{E}_{\boldsymbol{x}}[p(y_i|\boldsymbol{x})]}\right] \tag{19.22}$$

$$= \mathbb{E}_{\boldsymbol{x}}\left[\sum_{i=1}^{L} p(y_i|\boldsymbol{x}) \log p(y_i|\boldsymbol{x})\right] - \mathbb{E}_{\boldsymbol{x}}\left[\sum_{i=1}^{L} p(y_i|\boldsymbol{x}) \log \mathbb{E}_{\boldsymbol{x}}[p(y_i|\boldsymbol{x})]\right] \tag{19.23}$$

$$= \mathbb{E}_{\boldsymbol{x}}\left[\sum_{i=1}^{L} p(y_i|\boldsymbol{x}) \log p(y_i|\boldsymbol{x})\right] - \sum_{i=1}^{L} \mathbb{E}_{\boldsymbol{x}}[p(y_i|\boldsymbol{x}) \log \mathbb{E}_{\boldsymbol{x}}[p(y_i|\boldsymbol{x})]] \tag{19.24}$$

Since we had initially sought to *maximize* the mutual information, and we typically *minimize* loss functions, we can convert this to a suitable loss function by negating it:

$$\mathcal{L}(y;\boldsymbol{x}) = -\mathbb{E}_{\boldsymbol{x}}\left[\sum_{i=1}^{L} p(y_i|\boldsymbol{x}) \log p(y_i|\boldsymbol{x})\right] + \sum_{i=1}^{L} \mathbb{E}_{\boldsymbol{x}}[p(y_i|\boldsymbol{x}) \log \mathbb{E}_{\boldsymbol{x}}[p(y_i|\boldsymbol{x})]] \tag{19.25}$$

The first term is exactly the entropy minimization objective in expectation. The second term specifies that we should maximize the entropy of the expected class prediction, i.e. the average class prediction over our training set. This encourages the model to predict each possible class with equal probability, which is only appropriate when we know a priori that all classes are equally likely.

19.3.3 Co-training

Co-training [BM98] is also similar to self-training, but makes an additional assumption that there are two complementary "views" (i.e. independent sets of features) of the data, both of which can be used separately to train a reasonable model. After training two models separately on each view, unlabeled data is classified by each model to obtain candidate pseudo-labels. If a particular pseudo-label receives a low-entropy prediction (indicating high confidence) from one model and a high-entropy prediction (indicating low confidence) from the other, then that pseudo-labeled datapoint is added to the training set for the low-confidence model. Then, the process is repeated with the new, larger training datasets. The procedure of only retaining pseudo-labels when one of the models is confident ideally builds up the training sets with correctly-labeled data.

Co-training makes the strong assumption that there are two informative-but-independent views of the data, which may not be true for many problems. The **Tri-Training** algorithm [ZL05] circumvents this issue by instead using *three* models that are first trained on independently-sampled (with replacement) subsets of the labeled data. Ideally, initially training on different collections of labeled data results in models that do not always agree on their predictions. Then, pseudo-labels are generated for the unlabeled data independently by each of the three models. For a given unlabeled datapoint, if two of the models agree on the pseudo-label, it is added to the training set for the third model. This can be seen as a selection metric, because it only retains pseudo-labels where two (differently initialized) models agree on the correct label. The models are then re-trained on the combination of the labeled data and the new pseudo-labels, and the whole process is repeated iteratively.

19.3.4 Label propagation on graphs

If two datapoints are "similar" in some meaningful way, we might expect that they share a label. This idea has been referred to as the **manifold assumption**. **Label propagation** is a semi-supervised learning technique that leverages the manifold assumption to assign labels to unlabeled data. Label propagation first constructs a graph where the nodes are the data examples and the edge weights represent the degree of similarity. The node labels are known for nodes corresponding to labeled data but are unknown for unlabeled data. Label propagation then propagates the known labels across edges of the graph in such a way that there is minimal disagreement in the labels of a given node's neighbors. This provides label guesses for the unlabeled data, which can then be used in the usual way for supervised training of a model.

More specifically, the basic label propagation algorithm [ZG02] proceeds as follows: First, let $w_{i,j}$ denote a non-negative edge weight between $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ that provides a measure of similarity for the two (labeled or unlabeled) datapoints. Assuming that we have M labeled datapoints and N unlabeled datapoints, define the $(M+N) \times (M+N)$ transition matrix $\mathbf{T}$ as having entries

$$\mathbf{T}_{i,j} = \frac{w_{i,j}}{\sum_k w_{k,j}} \tag{19.26}$$

$\mathbf{T}_{i,j}$ represents the probability of propagating the label for node j to node i. Further, define the $(M+N) \times C$ label matrix $\mathbf{Y}$, where C is the number of possible classes. The ith row of $\mathbf{Y}$ represents the class probability distribution of datapoint i. Then, repeat the following steps until the values in $\mathbf{Y}$ do not change significantly: First, use the transition matrix $\mathbf{T}$ to propagate labels in $\mathbf{Y}$ by setting $\mathbf{Y} \leftarrow \mathbf{TY}$. Then, re-normalize the rows of Y by setting $\mathbf{Y}_{i,c} \leftarrow \mathbf{Y}_{i,c} / \sum_k \mathbf{Y}_{i,k}$. Finally, replace the rows of $\mathbf{Y}$ corresponding to labeled datapoints with their one-hot representation (i.e. $\mathbf{Y}_{i,c} = 1$ if datapoint i has ground-truth label c and 0 otherwise). After convergence, guessed labels are chosen based on the highest class probability for each datapoint in $\mathbf{Y}$.

This algorithm iteratively uses the similarity of datapoints (encoded in the weights used to construct the transition matrix) to propagate information from the (fixed) labels onto the unlabeled data. At each iteration, the label distribution for a given datapoint is computed as the weighted average of the label distributions for all of its connected datapoints, where the weighting corresponds to the edge weights in $\mathbf{T}$. It can be shown that this procedure converges to a single fixed point, whose computational cost mainly involves the inversion of the matrix of unlabled-to-unlabled transition probabilities [ZG02].

The overall approach can be seen as a form of **transductive learning**, since it is learning to predict labels for a fixed unlabeled dataset, rather than learning a model that generalizes. However, given the induced labeling. we can perform **inductive learning** in the usual way.

The success of label propagation depends heavily on the notion of similarity used to construct the weights between different nodes (datapoints). For simple data, measuring the Euclidean distance between datapoints can be sufficient. However, for complex and high-dimensional data the Euclidean distance might not meaningfully reflect the likelihood that two datapoints share the same class. The similarity weights can also be set arbitrarily according to problem-specific knowledge. For a few examples of different ways of constructing the similarity graph, see Zhu [Zhu05, chapter 3]. For some recent papers that use this approach in conjunction with deep learning, see e.g., [BRR18; Isc+19].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

19.3.5 Consistency regularization

Consistency regularization leverages the simple idea that perturbing a given datapoint (or the model itself) should not cause the model's output to change dramatically. Since measuring consistency in this way only makes use of the model's outputs (and not ground-truth labels), it is readily applicable to unlabeled data and therefore can be used to create appropriate loss functions for semi-supervised learning. This idea was first proposed under the framework of "learning with pseudo-ensembles" [BAP14], with similar variants following soon thereafter [LA16; SJT16].

In its most general form, both the model $p_\theta(y|\boldsymbol{x})$ and the transformations applied to the input can be stochastic. For example, in computer vision problems we may transform the input by using data augmentation like randomly rotating or adding noise the input image, and the network may include stochastic components like dropout (Section 13.5.4) or weight noise [Gra11]. A common and simple form of consistency regularization first samples $\boldsymbol{x}' \sim q(\boldsymbol{x}'|\boldsymbol{x})$ (where $q(\boldsymbol{x}'|\boldsymbol{x})$ is the distribution induced by the stochastic input transformations) and then minimizes the loss $\|p_\theta(y|\boldsymbol{x}) - p_\theta(y|\boldsymbol{x}')\|^2$. In practice, the first term $p_\theta(y|\boldsymbol{x})$ is typically treated as fixed (i.e. gradients are not propagated through it). In the semi-supervised setting, the combined loss function over a batch of labeled data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_M, y_M)$ and unlabeled data $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N$ is

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{M} \log p_\theta(y = y_i|\boldsymbol{x}_i) + \lambda \sum_{j=1}^{N} \|p_\theta(y|\boldsymbol{x}_j) - p_\theta(y|\boldsymbol{x}'_j)\|^2 \tag{19.27}$$

where λ is a scalar hyperparameter that balances the importance of the loss on unlabeled data and, for simplicity, we write $\boldsymbol{x}'_j$ to denote a sample drawn from $q(\boldsymbol{x}'|\boldsymbol{x}_j)$.

The basic form of consistency regularization in Equation (19.27) reveals many design choices that impact the success of this semi-supervised learning approach. First, the value chosen for the λ hyperparameter is important. If it is too large, then the model may not give enough weight to learning the supervised task and will instead start to reinforce its own bad predictions (as with confirmation bias in self-training). Since the model is often poor at the start of training before it has been trained on much labeled data, it is common in practice to initially set λ to zero and increase its value over the course of training.

A second important consideration are the random transformations applied to the input, i.e., $q(\boldsymbol{x}'|\boldsymbol{x})$. Generally speaking, these transformations should be designed so that they do not change the label of $\boldsymbol{x}$. As mentioned above, a common choice is to use domain-specific data augmentations. It has recently been shown that using strong data augmentations that heavily corrupt the input (but, arguably, still do not change the label) can produce particularly strong results [Xie+19; Ber+19a; Soh+20].

The use of data augmentation requires expert knowledge to determine what kinds of transformations are label-preserving and appropriate for a given problem. An alternative technique, called **virtual adversarial training** (VAT), instead transforms the input using an analytically-found perturbation designed to maximally change the model's output. Specifically, VAT computes a perturbation $\boldsymbol{\delta}$ that approximates $\boldsymbol{\delta} = \text{argmax}_{\boldsymbol{\delta}} D_{\mathbb{KL}}\left(p_\theta(y|\boldsymbol{x}) \parallel p_\theta(y|\boldsymbol{x} + \boldsymbol{\delta})\right)$. The approximation is done by sampling $\boldsymbol{d}$ from a multivariate Gaussian distribution, initializing $\boldsymbol{\delta} = \boldsymbol{d}$, and then setting

$$\boldsymbol{\delta} \leftarrow \nabla_{\boldsymbol{\delta}} D_{\mathbb{KL}}\left(p_\theta(y|\boldsymbol{x}) \parallel p_\theta(y|\boldsymbol{x} + \boldsymbol{\delta})\right)|_{\boldsymbol{\delta}=\xi\boldsymbol{d}} \tag{19.28}$$

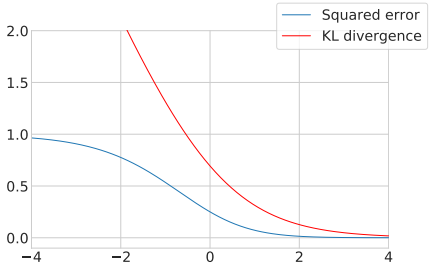


Figure 19.11: Comparison of the squared error and KL divergence losses for a consistency regularization. This visualization is for a binary classification problem where it is assumed that the model's output for the unperturbed input is 1. The figure plots the loss incurred for a particular value of the logit (i.e. the pre-activation fed into the output sigmoid nonlinearity) for the perturbed input. As the logit grows towards infinity, the model predicts a class label of 1 (in agreement with the prediction for the unperturbed input); as it grows towards negative infinity, the model predicts class 0. The squared error loss saturates (and has zero gradients) when the model predicts one class or the other with high probability, but the KL divergence grows without bound as the model predicts class 0 with more and more confidence.

where ξ is a small constant, typically 10^{-6}. VAT then sets

$$\boldsymbol{x}' = \boldsymbol{x} + \epsilon \frac{\boldsymbol{\delta}}{\|\boldsymbol{\delta}\|_2} \tag{19.29}$$

and proceeds as usual with consistency regularization (as in Equation (19.27)), where ϵ is a scalar hyperparameter that sets the L2-norm of the perturbation applied to $\boldsymbol{x}$.

Consistency regularization can also profoundly affect the geometry properties of the training objective, and the trajectory of SGD, such that performance can particularly benefit from non-standard training procedures. For example, the Euclidean distances between weights at different training epochs is significantly larger for objectives that use consistency regularization. Athiwaratkun et al. [Ath+19] show that a variant of **stochastic weight averaging** (SWA) [Izm+18] can achieve state-of-the-art performance on semi-supervised learning tasks by exploiting the geometric properties of consistency regularization.

A final consideration when using consistency regularization is the function used to measure the difference between the network's output with and without perturbations. Equation (19.27) uses the squared L2 distance (also referred to as the Brier score), which is a common choice [SJT16; TV17; LA16; Ber+19b]. It is also common to use the KL divergence $D_{\mathbb{KL}}\left(p_\theta(y|\boldsymbol{x}) \parallel p_\theta(y|\boldsymbol{x}')\right)$ in analogy with the cross-entropy loss (i.e. KL divergence between ground-truth label and prediction) used for labeled examples [Miy+18; Ber+19a; Xie+19]. The gradient of the squared-error loss approaches zero as the model's predictions on the perturbed and unperturbed input differ more and more, assuming the model uses a softmax nonlinearity on its output. Using the squared-error loss therefore has a possible advantage that the model is not updated when its predictions are very unstable. However, the KL divergence has the same scale as the cross-entropy loss used for labeled data, which makes for more intuitive tuning of the unlabeled loss hyperparameter λ. A comparison of the two loss functions is shown in Figure 19.11.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license